



Logo Fast and Mudry - AI generated

RAPPORT FAST & MUDRY

Projet de POO

Axel Schneider

Helder Ribeiro

11 juin 2026

Table des matières

1 – Introduction	3
2 – Architecture du projet	3
3 – Génération de la route et de la carte	5
3.1 – La ligne centrale	5
3.2 – De la géométrie à l'image	6
3.3 – Les biomes et les objets	6
4 – La machine à états	6
4.1 – Les états et les évènements	6
4.2 – Le fonctionnement	7
5 – Musique et gestion du son	7
5.1 – La musique	7
5.2 – Les bruitages	8
6 – Le rendu pseudo-3D avec le Mode 7	8
6.1 – Fonctionnement du rendu Mode 7 avec le GPU	8
6.2 – Affichage des objets et calcul de profondeur	10
7 – Défi et problématique	11
7.1 – Rendu graphique	11
8 – Conclusion	11
9 – Annexes	12
9.1 – Liens et références	12
9.2 – Code source du jeu	12

Note : Le rapport a été générée avec l'assistance d'une intelligence artificielle. Le contenu a été revu et validé par l'auteur avant son intégration au document final.

1 – Introduction

Dans le cadre du projet de programmation orientée objet, nous avons développé Fast & Mudry, un jeu de course en 2D écrit en Scala avec la bibliothèque gdx2d. Le principe est simple : le joueur contrôle une voiture qui doit traverser un circuit généré aléatoirement, en évitant les obstacles et en faisant attention à ne pas casser son véhicule en chemin.

Le développement ne s'est pas fait en ligne droite. La plus grosse évolution du projet a été le changement complet du système de rendu : les premières versions utilisaient une perspective calculée à la main dans le code, et nous l'avons remplacé par un rendu de type Mode 7 basé sur un shader qui tourne sur le GPU. Ce changement nous a obligé à réorganiser une partie du moteur de rendu, mais la logique du jeu elle-même a pu être conservée presque telle quelle.

Ce rapport présente d'abord l'architecture générale du projet, puis la génération de la route et de la carte, la machine à états qui gère le déroulement du jeu, la gestion de la musique et des sons, le fonctionnement du rendu pseudo-3D avec le Mode 7 et l'affichage des objets, et enfin les défis qu'on a rencontrés sur le rendu graphique.

2 – Architecture du projet

Le projet est organisé autour de trois grands axes : la logique de jeu, la gestion des écrans et le rendu graphique. Cette séparation n'était pas parfaite dès le début, mais nous avons essayé de la garder le plus possible pour que le code reste lisible et qu'on puisse travailler à deux sans se marcher dessus.

Les trois schémas ci-dessous montrent le système sous trois angles complémentaires : le flux d'application, le flux de données et le flux de rendu.

Le premier schéma montre le parcours dans l'application. Le programme commence par initialiser la fenêtre principale, puis enchaîne les écrans les uns après les autres : le menu, le chargement, la partie, le garage, l'écran de fin... L'application ne se résume donc pas à un programme linéaire : elle fonctionne comme une machine à états où chaque écran représente une étape du jeu (on y revient plus en détail dans la partie dédiée).

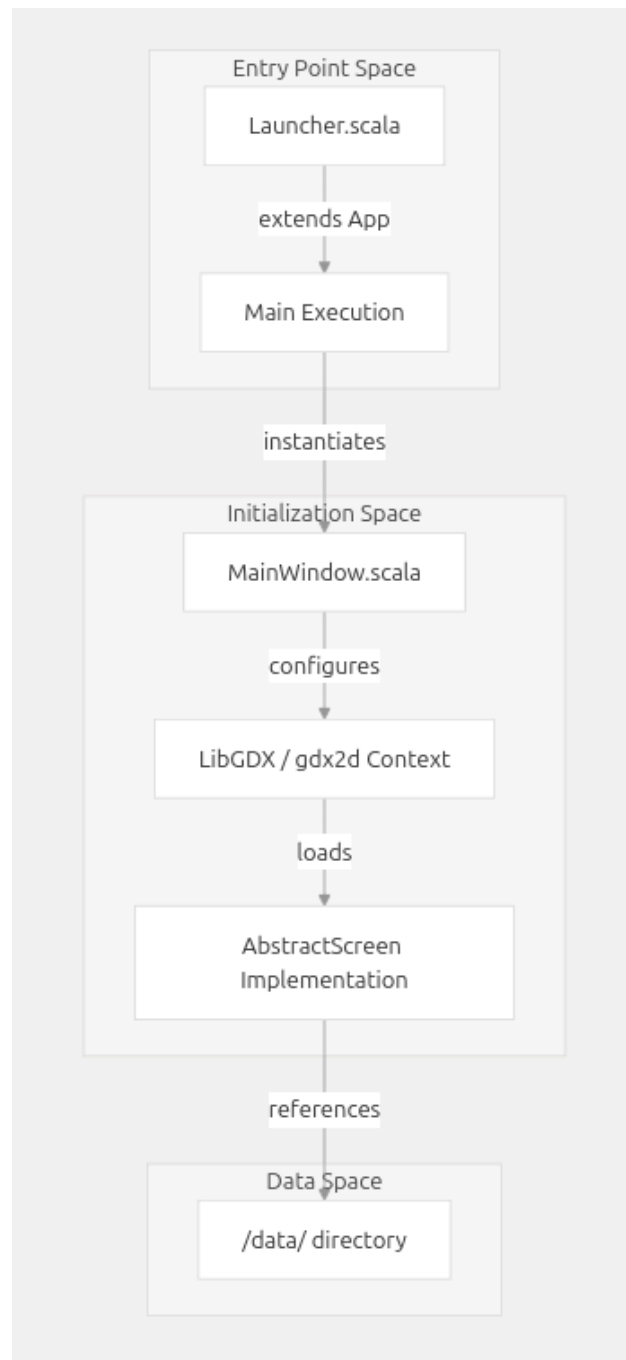


Figure 1 - Flux d'application du projet.

Le deuxième schéma se concentre sur les données. Tout tourne autour du monde du jeu : les entrées du joueur, les images, les sons et la progression alimentent un état global partagé. C'est à partir de cet état que la logique met à jour la position de la voiture, la progression du joueur, les événements du circuit et les objets interactifs. Ce qu'on voulait montrer ici, c'est que le cœur du projet n'est pas dans l'affichage, mais bien dans la gestion cohérente des informations du jeu.

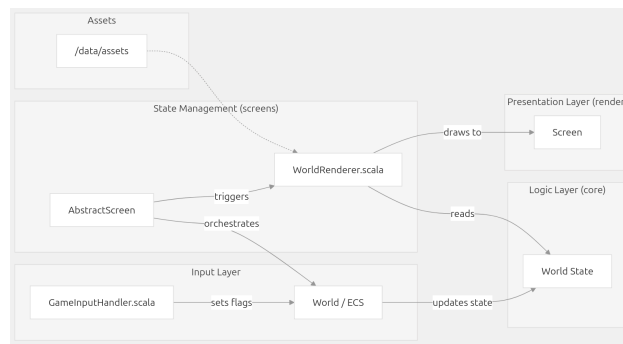


Figure 2 - Flux de données du projet.

Le troisième schéma explique comment cet état est ensuite transformé en image. Le module de rendu lit les informations du monde et les passe aux différents sous-systèmes de dessin : le fond, la piste, les objets, l'interface et le HUD. Pour la piste, le rendu repose sur une projection pseudo-3D réalisée par un shader Mode 7. L'affichage n'est donc pas une simple copie de l'état du jeu, c'est plutôt une interprétation visuelle de celui-ci.

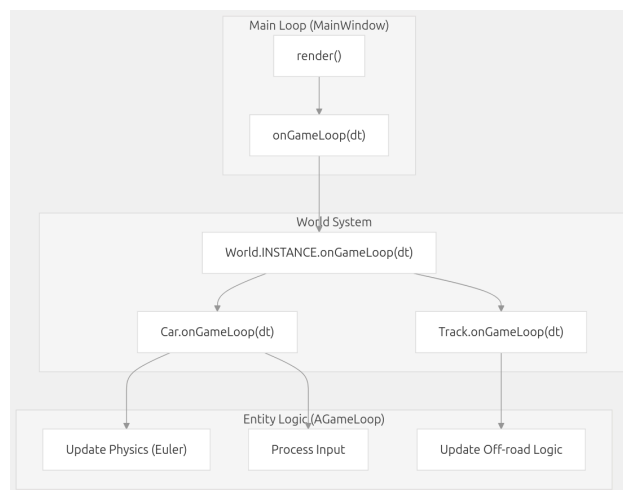


Figure 3 - Flux de rendu du projet.

Au final, ces trois vues se complètent bien : l'application gère les états, le cœur du jeu met à jour les données, et le moteur graphique transforme ces données en image. C'est cette séparation entre logique, données et rendu qui rend le projet modulaire et plutôt facile à faire évoluer.

3 – Génération de la route et de la carte

Un des objectifs du projet était que chaque partie soit un peu différente. Plutôt que de dessiner les circuits à la main, nous avons choisi de générer la route de manière procédurale au début de chaque journée de jeu. À chaque chargement, le joueur découvre donc un circuit qu'il n'a jamais vu.

3.1 – La ligne centrale

Tout part de la classe TrackGeometry, qui construit la ligne centrale de la route. Elle reçoit un point de départ, un point d'arrivée et un nombre de points de contrôle (20 dans notre cas). Les points sont d'abord répartis régulièrement entre le départ et l'arrivée, puis chacun reçoit un décalage aléatoire vertical et horizontal, sauf le premier et le dernier qui restent fixes pour garder un départ et une arrivée propre. À ce stade, la route ressemble à une suite de segments anguleux, ce qui n'est pas très agréable à conduire. Les points sont donc ensuite lissés avec une spline de Catmull-Rom, qui interpole 30 sous-points entre chaque paire de points de contrôle. Le résultat est une courbe fluide avec des virages naturels.

```
1 val yNoise = ((Math.random - 0.5) * 2.0 * randomHeight).toFloat
```

```

2  val h: Float = start.y + height * i + yNoise
3  val xNoise = if (i == 0 || i == nPoints - 1) 0f
4              else ((Math.random - 0.5) * 2.0 * width * 0.4).toFloat
5  val p: Vector2 = new Vector2(start.x + width * i + xNoise, h)

```

Listing 1 - Génération des points de contrôle aléatoires de la route

Cette ligne centrale sert ensuite de référence pour tout le reste. La géométrie expose une méthode qui calcule la distance entre un point quelconque et la ligne, ce qui permet de savoir si la voiture est sur la route ou non. Comme ce calcul est appelé énormément de fois, chaque segment possède une boîte englobante qui permet d'ignorer rapidement les segments trop éloignés. La ligne d'arrivée est simplement définie comme un cercle autour du dernier point de la courbe.

3.2 – De la géométrie à l'image

La carte affichée à l'écran est en réalité une grande texture générée par TrackTexture. L'objet parcourt chaque pixel de l'image et calcule sa distance à la ligne centrale. En fonction de cette distance, le pixel devient soit la ligne du milieu de la route, soit la route elle-même, soit le bas-côté, soit le terrain hors-piste. Les pixels proches du point d'arrivée sont dessinés en noir pour matérialiser la ligne d'arrivée.

Pour éviter d'avoir des surfaces complètement uniformes, les couleurs ne sont pas fixes. Chaque biome fournit une couleur claire et une couleur foncée pour chaque type de surface, et un bruit de Perlin vient mélanger les deux. Ça donne des textures avec des couleurs un peu différentes d'un pixel à l'autre, ce qui rend le sol beaucoup plus vivant. Le même principe est utilisé pour faire varier légèrement la largeur du bas-côté, comme ça le bord de la route n'est pas une ligne parfaite.

3.3 – Les biomes et les objets

Chaque journée du jeu possède son propre biome : la forêt, le désert et la neige. Le biome définit les couleurs de la carte, la musique, le décor en arrière-plan et aussi la physique quand on sort de la route. C'est également lui qui place les objets sur la carte, comme les arbres, les rochers ou les cactus. Pour chaque objet, une position est tirée aléatoirement sur la carte, et on recommence le tirage tant que la position ne convient pas, par exemple pour éviter qu'un arbre se retrouve au milieu de la route. Le bâtiment de la HES, lui, est toujours posé exactement sur le point d'arrivée.

Comme la génération de la texture est assez lourde (chaque pixel demande un calcul de distance), elle est lancée dans un thread séparé pendant que l'écran de chargement affiche une petite animation. Ça nous a permis d'éviter de bloquer la fenêtre du jeu pendant la génération. Une fois le calcul terminé, la texture est envoyée au GPU, la voiture est placée sur le premier point de la ligne centrale et orientée vers le deuxième, et la partie peut commencer.

4 – La machine à états

Le jeu est composé de plusieurs phases bien distinctes : le menu, le chargement, la course, les cinématiques, le quiz, le garage... Au début, les changements d'écrans étaient faits un peu à la main, mais on s'est vite rendu compte que ça devenait difficile de savoir qui avait le droit de passer où. Pour mettre de l'ordre, nous avons centralisé toute cette logique dans une machine à état.

4.1 – Les états et les événements

Chaque phase du jeu est représentée par un état, défini dans le trait scellé GameState. On y retrouve par exemple Menu, Loading, Playing, Quiz, Garage, Dead ou encore FinalCinematic. Certains états transportent en plus le jour courant de la partie (Day1, Day2 ou Day3), ce qui permet de savoir où le joueur en est dans sa progression sans avoir besoin d'une variable globale à côté.

Les transitions sont déclenchées par des événements, comme StartGame, MapLoaded, FinishLineCrossed ou CarBroke. L'idée centrale est que chaque état décide lui-même de sa transition : il implémente

une méthode `next` qui reçoit un évènement et retourne le prochain état. Si l'évènement ne le concerne pas, l'état se retourne simplement lui-même et rien ne se passe.

```

1  final case class Playing(day: Day) extends GameState {
2      def next(event: GameEvent): GameState = {
3          event match {
4              case FinishLineCrossed if day.next.isEmpty => FinalCinematic
5              case FinishLineCrossed => EndDayCinematic(day)
6              case CarBroke => Dead
7              case _ => this
8          }
9      }
10 }

```

Listing 2 - L'état Playing et ses transitions possibles

4.2 – Le fonctionnement

L'objet `GameStateMachine` garde en mémoire l'état courant. Quand un évènement arrive, il demande à l'état courant quel est le prochain état. Si l'état change, la machine active l'écran qui correspond via le gestionnaire d'écrans : l'état Menu affiche l'écran du menu, l'état Playing affiche l'écran de jeu, et ainsi de suite. Si l'état ne change pas, l'évènement est simplement ignoré. C'est un gros avantage : un évènement envoyé au mauvais moment ne peut pas casser le jeu ni provoquer des comportements bizarres.

Les évènements sont envoyés depuis un peu partout dans le code. Les boutons du menu envoient `StartGame` ou `OpenCarSelector`, l'écran de chargement envoie `MapLoaded` quand la carte est prête, la piste envoie `FinishLineCrossed` quand la voiture franchit la ligne d'arrivée ou `CarBroke` quand le véhicule est cassé, et les cinématiques envoient leur évènement de fin une fois leur durée écoulée. Aucun de ces endroits n'a besoin de connaître l'écran suivant, ils signalent juste ce qui vient de se passer.

Une partie complète suit donc un cycle assez naturel : depuis le menu, on passe au chargement puis à la course du premier jour. Quand la ligne d'arrivée est franchie, une cinématique de fin de journée se lance, suivie du quiz puis du garage où le joueur peut réparer sa voiture. Une cinématique de début de journée enchaîne ensuite sur le chargement du jour suivant. Au troisième jour, franchir la ligne d'arrivée déclenche la cinématique finale qui ramène au menu. Et si la voiture casse en route, peu importe le jour, on passe directement à l'écran de mort.

Ce découpage nous a beaucoup aidé sur la fin du projet : pour rajouter une nouvelle phase, comme le sélecteur de voiture, il suffisait de rajouter un état, ses évènements et son écran, sans toucher au restant de la logique.

5 – Musique et gestion du son

Un jeu de course sans son, ça ne donne pas grand chose. Nous avons donc ajouté de la musique et des bruitages, et toute cette partie est centralisée dans un seul objet, `AudioManager`. C'est lui qui possède les lecteurs de musique et les effets sonores, et le reste du code se contente de lui demander de jouer quelque chose, sans savoir comment ça marche derrière.

5.1 – La musique

Chaque morceau du jeu est représenté par un objet du trait scellé `MusicTrack` : il y en a un pour le menu, le quiz, le garage, les cinématiques, et un par biome pour la course (forêt, désert et neige). Quand un écran démarre, il demande simplement à `AudioManager` de jouer son morceau. Pendant la course, c'est le biome du jour qui fournit sa propre musique, donc l'ambiance sonore change en même temps que le décor.

La méthode `playTrack` s'occupe de la transition : elle arrête le morceau en cours et lance le nouveau en boucle. Si on lui demande le morceau qui est déjà en train de jouer, elle ne fait rien du tout, ce qui évite

que la musique soit relancer à zéro quand on revient sur le même écran. Les lecteurs sont gardés dans une map et ne sont créés qu'à la première utilisation d'un morceau, comme ça on ne charge pas tous les fichiers audio au lancement du jeu. Il y a aussi un petit cas particulier : la cinématique finale ne change pas de musique et garde celle du dernier biome, ce qu'on trouvait plus sympa pour la fin du jeu.

5.2 – Les bruitages

Le bruitage le plus important est celui du moteur. C'est un son joué en boucle dont la hauteur (le pitch) est recalculée à chaque frame en fonction de la vitesse de la voiture : plus on va vite, plus le moteur monte dans les tours. Le ratio entre la vitesse actuelle et la vitesse maximale est converti en un pitch compris entre une valeur basse et une valeur haute, ce qui donne l'impression d'accélérer alors qu'on joue toujours le même fichier son.

```

1 var ratio = Math.abs(speed) / maxSpeed
2 if (ratio > 1f) ratio = 1f
3
4 var pitch = AUDIO.SFX.ENGINE_LOW_PITCH + ratio * (AUDIO.SFX.ENGINE_TOP_PITCH -
5 AUDIO.SFX.ENGINE_LOW_PITCH)
6 if (pitch < 0.5f) pitch = 0.5f
7 if (pitch > 2f) pitch = 2f
8 engine.modifyPitch(pitch, engineId)

```

Listing 3 - Calcul du pitch du moteur en fonction de la vitesse

Notre petit plaisir du projet, c'est le backfire. Quand le joueur relâche les gaz à haute vitesse, le pot d'échappement lâche une série de petit pops, comme sur une vraie voiture de course. Le monde du jeu surveille la vitesse : si elle a dépassé un certain seuil puis commence à redescendre, il déclenche le backfire. AudioManager joue alors entre 4 et 5 pops, avec un délai et un pitch tirés aléatoirement pour chaque pop, comme ça la séquence n'est jamais exactement la même et ça sonne beaucoup moins artificiel.

Les derniers effets sont liés aux problèmes de la voiture : un son d'explosion quand un pneu lâche et un bruit de crash quand on percute un obstacle. Comme pour la musique, ces sons sont chargé seulement à la première utilisation. Et quand on ferme le jeu, la fenêtre principale demande à AudioManager de libérer toutes les ressources audio proprement.

6 – Le rendu pseudo-3D avec le Mode 7

6.1 – Fonctionnement du rendu Mode 7 avec le GPU

Dans l'architecture actuelle, c'est TrackRenderer qui est responsable de créer l'image affichée. Il joue un rôle d'intermédiaire : il récupère la texture du circuit, place une caméra virtuelle en fonction de la position du véhicule, envoie une série de paramètres au shader et lance le rendu. Cette séparation est pratique car la logique du jeu reste dans les classes métier, pendant que la partie graphique s'occupe juste de transformer ces informations en image.

Le fichier Mode7.scala ne dessine rien lui-même. Il sert surtout de configuration pour le shader : les noms des paramètres, les chemins vers le programme graphique et les valeurs par défaut. TrackRenderer transmet ensuite tout ça au shader : la résolution de l'écran, la position de la caméra, son angle, l'axe de rotation, la distance du plan de projection et le facteur d'échelle de la texture. L'avantage, c'est que la projection reste dans un module spécialisé, complètement indépendant de la physique de conduite ou de la gestion du jeu.

La position et l'orientation de la voiture influencent directement le rendu. À chaque image, TrackRenderer met à jour la caméra à partir des coordonnées du véhicule et de sa rotation, puis envoie ces valeurs au shader qui recalcule l'affichage du circuit pour chaque pixel. Du coup, quand la voiture tourne, la

texture du circuit est réinterprété sous un autre angle, ce qui donne l'illusion qu'on se déplace sur une vraie route en perspective. On peut donc voir le véhicule comme une entrée du rendu, au même titre que la taille de l'écran ou la résolution de la texture.

```

1 cameraPosition.y = World.INSTANCE.CAR.Coordinates.y
2 cameraPosition.x = World.INSTANCE.CAR.Coordinates.x
3 cameraAngle = World.INSTANCE.CAR.Rotation
4
5 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.POSITION, cameraPosition)
6 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.ANGLE, cameraAngle)

```

Listing 4 - Transmission des informations de déplacement au shader

Côté shader, le calcul part des coordonnées du fragment, c'est à dire du pixel à afficher à l'écran. Pour chaque pixel, le shader construit un rayon de vue dans un espace virtuel, applique une rotation selon l'angle de la caméra, puis retrouve la position correspondante dans la texture du circuit. En combinant la position de la caméra, la distance du plan de projection et le facteur de mise à l'échelle, on obtient au final une coordonnée de texture comprise entre 0 et 1.

$$\begin{aligned}
 r &= (x_s, y_s, d) \\
 r' &= R(a)r \\
 p &= c + (r'_x, r'_y) * \left(\frac{c_z}{r'_z} \right) \\
 t &= (p - o) * f
 \end{aligned}$$

auto 4 - Formulation générale de la projection du fragment

Cette écriture résume le calcul fait par le shader : un fragment, défini par ses coordonnées écran x_s et y_s , correspond d'abord à un rayon r dans un espace virtuel. Ce rayon est tourné selon l'angle a de la caméra, puis projeté sur un plan de référence pour obtenir une position p dans le repère de la carte. La coordonnée de texture t est enfin obtenue en appliquant une mise à l'échelle à partir de l'origine o et du facteur f . Si cette coordonnée tombe sur le circuit, la couleur de cette zone est lue dans la texture et affichée à l'écran. Au final c'est de la géométrie assez simple, mais appliqué à chaque pixel par le GPU.

```

1 vec3 pixelRay;
2 pixelRay.xz = (gl_FragCoord.xy / resolution.xy) * vec2(1.0, -1.0) + vec2(-0.5,
3 0.5);
4 pixelRay.y = screenPlanDistance;
5 pixelRay *= rotationMatrix(cameraAxis, cameraAngle);
6
7 vec2 hitPosition = cameraPosition.xy + pixelRay.xy * (cameraPosition.z /
8 pixelRay.z) + vec2(0.5, 0);
9 hitPosition = (hitPosition - mapOrigin) * renderingFactor;
10 gl_FragColor = texture2D(backbuffer, hitPosition);

```

Listing 5 - Projection du fragment vers une coordonnée de texture

Le GPU est parfait pour ce genre de traitement, parce qu'il peut faire les calculs sur des millions de pixels en parallèle. Contrairement au CPU qui traiterait les pixels un par un, le shader les traite tous en même temps avec la même logique de projection. Le processeur graphique s'occupe donc de la transformation géométrique et de la lecture de la texture, ce qui laisse le CPU se concentrer sur la logique du jeu.

Cette méthode est simple et efficace pour un jeu de course en 2D/2.5D, mais elle a ses limites : tout dépend de la résolution de la texture, du réglage de la caméra et de l'alignement entre la piste et les objets du décor. Il a aussi fallu régler le shader avec soin pour éviter les artefacts au bord du circuit, là où la projection devient instable. C'est un compromis qui nous convenait bien : un rendu fluide sans rajouter une complexité de calcul trop importante.

6.2 – Affichage des objets et calcul de profondeur

La même logique de projection est réutilisée pour afficher les objets du jeu, comme les obstacles ou les éléments du décor, par-dessus le circuit. Chaque item possède une position dans le monde, exprimée dans les coordonnées du circuit, et il faut le projeter dans l'espace de l'écran en fonction de la position de la caméra et de son orientation. Pour ça, on calcule d'abord un vecteur entre la position de l'objet et celle de la caméra, puis on fait tourner ce vecteur selon l'angle du véhicule. Le résultat permet de savoir si l'objet se trouve devant ou derrière la caméra virtuelle : si la composante de profondeur est négative ou trop petite, on ne le projete pas du tout, ce qui évite d'afficher des objets qui sont derrière nous ou hors du champ de vision.

Quand l'objet est visible, sa position à l'écran est obtenue par une projection perspective simple. La position horizontale vient du rapport entre la distance latérale de l'objet et sa profondeur, et la position verticale dépend à la fois de la hauteur de la caméra et de l'inclinaison du point de vue. L'échelle de l'objet est elle aussi calculée à partir de la profondeur, avec une relation de la forme suivante.

$$s = \frac{d_p}{p_y}$$

auto 5 - Échelle de l'objet calculée à partir de sa profondeur relative

Grâce à cette relation, les objets proches paraissent gros et les objets lointain tout petits, ce qui donne une impression de profondeur cohérente entre tous les éléments de la scène.

Dans le code, tout ce calcul est centralisé dans ItemsRenderer. La méthode de projection retourne les coordonnées d'affichage, un facteur d'échelle et une distance, puis les objets sont triés selon cette distance avant d'être dessinés. Les coordonnées à l'écran sont obtenues avec les deux relations suivantes.

$$x_e = w * \left(0.5 + \frac{d_p * p_x}{p_y}\right)$$

auto 6 - Coordonnée horizontale de projection sur l'écran

$$y_e = h * \left(0.5 + t - \frac{d_p * z_c}{p_y}\right)$$

auto 7 - Coordonnée verticale de projection sur l'écran

Ce tri par distance est important : sans lui, des éléments du décor ou des obstacles pouvaient se superposer n'importe comment et la scène devenait illisible.

```

1  val differentialVector = new Vector2(position.x, position.y)
2  differentialVector.sub(cameraPosition.x + 0.5f, cameraPosition.y)
3
4  val primeVector = new Vector2(
5      differentialVector.x * cosAlpha - differentialVector.y * sinAlpha,
6      differentialVector.x * sinAlpha + differentialVector.y * cosAlpha
7  )
8
9  if (primeVector.y ≤ 0.01f) return None
10
11  val screenX = screenW * (0.5f + (screenPlanDistance * primeVector.x) /
12      primeVector.y)
13  val screenY = screenH * (0.5f + pitch - (screenPlanDistance * cameraPosition.z) /
14      primeVector.y)
15
16  val scale = screenPlanDistance / primeVector.y

```

Listing 6 - Projection d'un item dans l'espace de l'écran

Pour résumer le flux de données : l'état du véhicule est transmis sous forme d'uniforms par TrackRenderer, le shader GPU échantillonne la texture du circuit, et l'image finale s'affiche à l'écran. Pour les items, il y a juste une projection en plus, qui transforme leur position dans le monde en coordonnée d'écran et en échelle visuelle.

7 – Défi et problématique

7.1 – Rendu graphique

Un des plus gros défis du projet a été le rendu de la piste. Les premières versions du jeu utilisaient un affichage de type arcade, avec une perspective simulée directement dans le code. C'était relativement rapide à mettre en place et ça donnait déjà une impression de route avec des virages, mais on a vite vu les limites de cette approche.

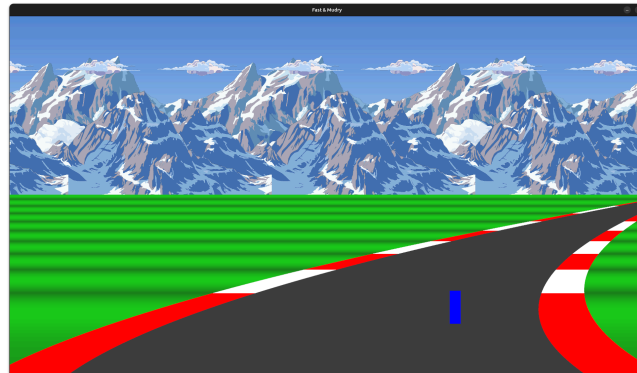


Figure 8 - Première version du rendu

En pratique, les virages étaient presque impossible à anticiper. La route manquait de profondeur et les changements de direction arrivaient d'un coup, sans prévenir. Pour le joueur c'était frustrant : on ne voyait que la portion de route juste devant la voiture, donc difficile de préparer une trajectoire ou d'éviter un obstacle qu'on découvre au dernier moment.

Pour corriger ça, nous avons repensé le système de rendu autour d'une approche de type Mode 7, la technique utilisée à l'époque par des jeux comme Mario Kart sur Super Nintendo. L'idée est de projeter une texture représentant le circuit de façon à créer une illusion de profondeur, tout en restant dans un environnement essentiellement 2D. Par rapport à l'ancien rendu, on voit une portion beaucoup plus grande du circuit devant le véhicule.



Figure 9 - Rendu avec mode 7

Ce changement a vraiment amélioré la lisibilité du jeu. Le joueur distingue maintenant les virages à venir, repère les obstacles situés plus loin sur la piste et peut voir certains éléments importants du décor, comme la HES qui marque la ligne d'arrivée. La conduite devient plus naturelle : on anticipe au lieu de simplement réagir à ce qui apparaît à l'écran.

8 – Conclusion

Ce projet nous a permis de construire un jeu complet, de l'écran de menu jusqu'à la cinématique finale, en passant par la génération procédurale de la carte, une machine à états, un rendu pseudo-3D sur le GPU et la gestion de la musique et des sons. Au départ, l'objectif était simplement de faire rouler une

voiture sur une route ; au final, on se retrouve avec trois biomes, un garage, un quiz et des pneus qui explosent. Au-delà du jeu lui-même, le projet nous a apporté beaucoup de connaissance sur la programmation orientée objet.

Sur le plan technique, la plus grande leçon du projet est probablement de ne pas avoir peur de jeter du code. Nous avons passé pas mal de temps sur le premier système de rendu, et le remplacer entièrement par le Mode 7 n'était pas une décision facile. Mais c'est ce changement qui a rendu le jeu réellement agréable à jouer.

Le travail à deux s'est bien passé, notamment grâce au découpage du code en modules et à l'utilisation de branches git avec des pull requests. Chacun pouvait avancer sur sa partie sans bloquer l'autre, et les conflits de merge sont restés rares.

Évidemment, tout n'est pas parfait. Certaines parties aurait mérité un peu plus de polissage, comme les réglages du shader qui restent un peu fragile au bord du circuit. Et si le projet devait continuer, on aimerait ajouter plus de journées avec de nouveaux biomes, enrichir la physique des collisions et pourquoi pas un mode deux joueurs. Mais dans l'ensemble, nous sommes plutôt fier du résultat : un jeu qui se lance, qui se joue, et qui donne envie de refaire une partie pour voir un circuit différent.

9 – Annexes

9.1 – Liens et références

Voici les différentes ressources qui nous ont aidé pendant le projet. La plupart sont aussi citées en commentaire directement dans le code, aux endroits concernés.

- [Programming Pseudo 3D Planes aka MODE7 \(javidx9\)](#) : la vidéo qui nous a servi de référence pour comprendre le fonctionnement du rendu Mode 7.
- [Code-It-Yourself! Retro Arcade Racing Game \(javidx9\)](#) : l'inspiration principale de la toute première version du rendu, de type arcade.
- [Let's make 16 games in C++ : Outrun \(FamTrinli\)](#) : un autre exemple de jeu de course en pseudo-3D qui nous a inspiré au début du projet.
- [Programming & Using Splines - Part#1 \(javidx9\)](#) : une vidéo sur les splines, utile pour comprendre le lissage de la ligne centrale de la route.
- [perlin-noise \(GitHub\)](#) : le dépôt d'où vient la majorité du code de génération de bruit de Perlin, utilisé pour les textures des biomes.
- [Scala Algorithms : StateMachine](#) : la structure sur laquelle est basé notre machine à états.
- [Stack Overflow : distance point-segment](#) : l'algorithme de calcul de distance entre un point et un segment, utilisé pour déterminer si la voiture est sur la route.
- [Documentation LibGDX : Scene2D](#) : la documentation utilisée pour la construction de l'interface utilisateur.

9.2 – Code source du jeu

Cette annexe contient l'intégralité du code source Scala du jeu, ainsi que le shader GLSL utilisé pour le rendu Mode 7.

9.2.1 – AudioManager.scala

```

1 package ch.hevs.fastandmudry
2 package core.audio
3
4 import ch.hevs.fastandmudry.utils.Constant.AUDIO
5 import ch.hevs.gdx2d.components.audio.{SoundSample, VolumeMusicPlayer}
6
7 import java.util
8 import scala.util.Random

```

```

9
10 object AudioManager {
11     private val players = new util.HashMap[MusicTrack, VolumeMusicPlayer]()
12     private var current: MusicTrack = _
13
14     private var engine: SoundSample = _
15     private var engineId: Long = -1
16
17     private val random = new Random()
18     private var backfire: SoundSample = _
19
20     private var popsLeft: Int = 0
21     private var nextPopTimer: Float = 0f
22
23     private var tireExplosion: SoundSample = _
24     private var collision: SoundSample = _
25
26     private def playerFor(track: MusicTrack): VolumeMusicPlayer = {
27         var player = players.get(track)
28         if (player == null) {
29             player = new VolumeMusicPlayer(track.path)
30             player.setVolume(AUDIO.MUSIC_VOLUME)
31             players.put(track, player)
32         }
33         return player
34     }
35
36     def playTrack(track: MusicTrack): Unit = {
37         if (current == track) return
38         if (current != null) players.get(current).stop()
39         playerFor(track).loop()
40         current = track
41     }
42
43     def stop(): Unit = {
44         if (current != null) players.get(current).stop()
45         current = null
46     }
47
48     def startEngine(): Unit = {
49         if (engine == null) {
50             engine = new SoundSample(AUDIO.SFX.ENGINE)
51             engine.setVolume(AUDIO.SFX.ENGINE_VOLUME)
52         }
53         if (engineId == -1) {
54             engine.setPitch(AUDIO.SFX.ENGINE_LOW_PITCH)
55             engineId = engine.loop()
56         }
57     }
58
59     def updateEngine(speed: Float, maxSpeed: Float): Unit = {
60         if (engineId == -1) return
61
62         var ratio = Math.abs(speed) / maxSpeed
63         if (ratio > 1f) ratio = 1f
64
65         var pitch = AUDIO.SFX.ENGINE_LOW_PITCH + ratio * (AUDIO.SFX.ENGINE_TOP_PITCH -
66 AUDIO.SFX.ENGINE_LOW_PITCH)
67         if (pitch < 0.5f) pitch = 0.5f
68         if (pitch > 2f) pitch = 2f
69
70         engine.modifyPitch(pitch, engineId)
71     }
72
73     def stopEngine(): Unit = {
74         if (engine != null) engine.stop()
75         engineId = -1
76     }
77 }

```

```

76
77 private def playPop(): Unit = {
78     val pitchRange = AUDIO.SFX.BACKFIRE_MAX_PITCH - AUDIO.SFX.BACKFIRE_MIN_PITCH
79     backfire.setPitch(AUDIO.SFX.BACKFIRE_MIN_PITCH + random.nextFloat() *
pitchRange)
80     backfire.play()
81 }
82
83 private def scheduleNextPop(): Unit = {
84     val gapRange = AUDIO.SFX.BACKFIRE_MAX_GAP - AUDIO.SFX.BACKFIRE_MIN_GAP
85     nextPopTimer = AUDIO.SFX.BACKFIRE_MIN_GAP + random.nextFloat() * gapRange
86 }
87
88 def triggerBackfire(): Unit = {
89     if (backfire == null) {
90         backfire = new SoundSample(AUDIO.SFX.BACKFIRE)
91         backfire.setVolume(AUDIO.SFX.BACKFIRE_VOLUME)
92     }
93
94     val popCount = AUDIO.SFX.BACKFIRE_MIN_POPS +
random.nextInt(AUDIO.SFX.BACKFIRE_MAX_POPS - AUDIO.SFX.BACKFIRE_MIN_POPS + 1)
95
96     playPop()
97     popsLeft = popCount - 1
98     scheduleNextPop()
99 }
100
101 def updateBackfire(elapsedTime: Float): Unit = {
102     if (popsLeft ≤ 0) return
103
104     nextPopTimer -= elapsedTime
105     if (nextPopTimer ≤ 0f) {
106         playPop()
107         popsLeft -= 1
108         if (popsLeft > 0) scheduleNextPop()
109     }
110 }
111
112 def playTireExplosion(): Unit = {
113     if (tireExplosion == null) {
114         tireExplosion = new SoundSample(AUDIO.SFX.TIRE_EXPLOSION)
115         tireExplosion.setVolume(AUDIO.SFX.TIRE_EXPLOSION_VOLUME)
116     }
117     tireExplosion.play()
118 }
119
120 def playCollision(): Unit = {
121     if (collision == null) {
122         collision = new SoundSample(AUDIO.SFX.COLLISION)
123         collision.setVolume(AUDIO.SFX.COLLISION_VOLUME)
124     }
125     collision.play()
126 }
127
128 def dispose(): Unit = {
129     val values = players.values.iterator
130     while (values.hasNext) {
131         values.next.dispose()
132     }
133     players.clear()
134     current = null
135
136     if (engine ≠ null) {
137         engine.dispose()
138         engine = null
139     }
140     engineId = -1

```

```

141
142     if (backfire ≠ null) {
143         backfire.dispose()
144         backfire = null
145     }
146     popsLeft = 0
147
148     if (tireExplosion ≠ null) {
149         tireExplosion.dispose()
150         tireExplosion = null
151     }
152
153     if (collision ≠ null) {
154         collision.dispose()
155         collision = null
156     }
157 }
158 }

```

9.2.2 – MusicTrack.scala

```

1 package ch.hevs.fastandmudry
2 package core.audio
3
4 import ch.hevs.fastandmudry.utils.Constant.AUDIO.MUSIC
5
6 sealed trait MusicTrack {
7     def path: String
8 }
9
10 object MusicTrack {
11     case object Menu extends MusicTrack { val path: String = MUSIC.MENU }
12     case object Cinematic extends MusicTrack { val path: String = MUSIC.CINEMATIC }
13     case object Quiz extends MusicTrack { val path: String = MUSIC.QUIZ }
14     case object Garage extends MusicTrack { val path: String = MUSIC.GARAGE }
15     case object Forest extends MusicTrack { val path: String = MUSIC.FOREST }
16     case object Desert extends MusicTrack { val path: String = MUSIC.DESERT }
17     case object Snow extends MusicTrack { val path: String = MUSIC.SNOW }
18 }

```

9.2.3 – VolumeMusicPlayer.scala

```

1 package ch.hevs.gdx2d.components.audio
2
3 class VolumeMusicPlayer(file: String) extends MusicPlayer(file) {
4     override def setVolume(v: Float): Unit = {
5         super.setVolume(v)
6         s.setVolume(v)
7     }
8 }

```

9.2.4 – CarSkin.scala

```

1 package ch.hevs.fastandmudry
2 package core.car
3
4 import com.badlogic.gdx.graphics.Color
5
6 import scala.collection.mutable.ArrayBuffer
7
8 final case class WheelPosition(x: Float, y: Float)
9
10 final case class SpeedometerConfig(x: Float, y: Float, radius: Float,

```



```

11         backgroundColor: Color, ticksColor: Color,
12         needleColor: Color)
13     final case class CarSkin(pilotName: String, carName: String,
14                             threeQuarterImagePath: String, sideImagePath: String,
15                             openHoodImagePath: String,
16                             interiorImagePath: String, steeringWheelImagePath: String,
17                             wheelPosition: WheelPosition,
18                             speedometer: SpeedometerConfig,
19                             skinBugged: Boolean
20     )
21     object CarSkins {
22         private val prefix = "data/images/skins/"
23         private val threeQuarterImagePathSuffix = "/three_quarter.png"
24         private val sideImagePathSuffix = "/side.png"
25         private val openHoodImagePathSuffix = "/open_hood.png"
26         private val interiorImagePathSuffix = "/interior.png"
27         private val steeringWheelImagePathSuffix = "/steering_wheel.png"
28
29         val all: ArrayBuffer[CarSkin] = ArrayBuffer(
30             CarSkin("Default", "BMW E36 M3",
31                 prefix + "e36" + threeQuarterImagePathSuffix, prefix + "e36" +
32                 sideImagePathSuffix, prefix + "e36" + openHoodImagePathSuffix,
33                 prefix + "e36" + interiorImagePathSuffix, prefix + "e36" +
34                 steeringWheelImagePathSuffix,
35                 WheelPosition(0.35f, 0.10f),
36                 SpeedometerConfig(0.295f, 0.22f, 100f, Color.BLACK, Color.WHITE, Color.RED),
37                 true
38             ),
39             CarSkin("Mudry", "Renault Zoe",
40                 prefix + "zoe" + threeQuarterImagePathSuffix, prefix + "zoe" +
41                 sideImagePathSuffix, prefix + "zoe" + openHoodImagePathSuffix,
42                 prefix + "zoe" + interiorImagePathSuffix, prefix + "zoe" +
43                 steeringWheelImagePathSuffix,
44                 WheelPosition(0.350f, 0f),
45                 SpeedometerConfig(0.25f, 0.1f, 80f, Color.BLACK, Color.WHITE, Color.CYAN),
46                 false
47             ),
48             CarSkin("Ryan Gosling", "Chevrolet Chevelle Malibu 1973",
49                 prefix + "chevrolet" + threeQuarterImagePathSuffix, prefix + "chevrolet" +
50                 sideImagePathSuffix, prefix + "chevrolet" + openHoodImagePathSuffix,
51                 prefix + "chevrolet" + interiorImagePathSuffix, prefix + "chevrolet" +
52                 steeringWheelImagePathSuffix,
53                 WheelPosition(0.35f, 0.18f),
54                 SpeedometerConfig(0.28f, 0.19f, 80f, Color.BLACK, Color.WHITE, Color.RED),
55                 true
56             ),
57             CarSkin("Jacquemet", "Car Postal",
58                 prefix + "bus" + threeQuarterImagePathSuffix, prefix + "bus" +
59                 sideImagePathSuffix, prefix + "bus" + openHoodImagePathSuffix,
60                 prefix + "bus" + interiorImagePathSuffix, prefix + "bus" +
61                 steeringWheelImagePathSuffix,
62                 WheelPosition(0.19f, -0.1f),
63                 SpeedometerConfig(0.1f, 0f, 80f, Color.BLACK, Color.WHITE, Color.RED),
64                 false
65             ),
66             CarSkin("Paul Walker", "Toyota Supra MK4",
67                 prefix + "supra" + threeQuarterImagePathSuffix, prefix + "supra" +
68                 sideImagePathSuffix, prefix + "supra" + openHoodImagePathSuffix,
69                 prefix + "supra" + interiorImagePathSuffix, prefix + "supra" +
70                 steeringWheelImagePathSuffix,
71                 WheelPosition(0.22f, 0.02f),
72                 SpeedometerConfig(0.187f, 0.127f, 80f, Color.BLACK, Color.ORANGE,
73                 Color.ORANGE),

```



```

63     false
64   ),
65 )
66
67 val default: CarSkin = all(0)
68 }

```

9.2.5 – AGameLoop.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 abstract class AGameLoop {
5     def onGameLoop(elapsedTime: Float): Unit
6 }

```

9.2.6 – Axle.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components
2
3 object Axle extends Enumeration {
4     type Axle = Value
5     val Front, Rear = Value
6 }

```

9.2.7 – CircleCollision.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.ecs.components.Locatable
4
5 trait CircleCollision extends Collisional {
6     protected var Size = 10;
7     override def CheckCollision(location: Locatable): Boolean = {
8         location.Coordinates.dst2(Coordinates) ≤ Size
9     }
10 }

```

9.2.8 – Collisional.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.audio.AudioManager
4 import ch.hevs.fastandmudry.core.ecs.components.Locatable
5 import ch.hevs.fastandmudry.core.ecs.systems.Car
6
7 trait Collisional extends Locatable with DefectableCollisional {
8     def onCollision(obj: Car): Unit = {
9         obj.Speed = 0
10        AudioManager.playCollision()
11        defectCar(obj)
12    }
13    def CheckCollision(location: Locatable): Boolean
14 }

```

9.2.9 – DefectableCollisional.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.ecs.systems.Car
4

```

```

5 trait DefectableCollisional {
6   def defectCar(obj: Car): Unit
7 }

```

9.2.10 – DefectChassis.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.ecs.systems.Car
4
5 trait DefectChassis extends DefectableCollisional {
6   override def defectCar(obj: Car): Unit = {
7     obj.ChassisProblem.IsDefected = true
8   }
9 }

```

9.2.11 – NoDefect.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.ecs.systems.Car
4
5 trait NoDefect extends DefectableCollisional {
6   override def defectCar(obj: Car): Unit = {}
7 }

```

9.2.12 – PassThoughtCollision.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.Collision
2
3 import ch.hevs.fastandmudry.core.ecs.components.Locatable
4
5 trait PassThoughtCollision extends Collisional {
6   override def CheckCollision(location: Locatable): Boolean = false
7 }

```

9.2.13 – GodMode.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components
2
3 import com.badlogic.gdx.{Gdx, Input}
4
5 trait GodMode {
6   private var _IsGodModeEnable: Boolean = false
7   private var isPressed = false
8
9   def IsGodModeEnable: Boolean = _IsGodModeEnable
10
11   def checkGodMode(): Unit = {
12     if (Gdx.input.isKeyPressed(Input.Keys.F8)) {
13       if (!isPressed) {
14         _IsGodModeEnable = !_IsGodModeEnable
15         isPressed = true
16       }
17     } else {
18       isPressed = false
19     }
20   }
21 }

```

9.2.14 – HasTires.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 trait HasTires {
5     private var _isRightTirePerforated: Boolean = false
6     private var _isLeftTirePerforated: Boolean = false
7
8     def IsRightTirePerforated: Boolean = _isRightTirePerforated
9     def IsLeftTirePerforated: Boolean = _isLeftTirePerforated
10
11     def IsRightTirePerforated_=(value: Boolean): Unit = {
12         _isRightTirePerforated = value
13     }
14
15     def IsLeftTirePerforated_=(value: Boolean): Unit = {
16         _isLeftTirePerforated = value
17     }
18 }

```

9.2.15 – Locatable.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 import com.badlogic.gdx.math.Vector2
5
6 trait Locatable {
7     private var _Coordinates: Vector2 = new Vector2(0f, 0f)
8
9     def Coordinates: Vector2 = _Coordinates
10
11     def Coordinates_=(value: Vector2): Unit = {
12         _Coordinates = value
13     }
14 }

```

9.2.16 – Moveable.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 trait Moveable extends Locatable with Orientable {
5     private var _speed = 0f
6     private var _maxSpeed = 1f
7     private var _isGoingBackward: Boolean = false
8
9     private def MinSpeed = -_maxSpeed/4f
10
11     def Speed: Float = _speed
12     def MaxSpeed: Float = _maxSpeed
13
14     def Speed_=(value: Float): Unit = {
15         _speed =
16             if(value > _maxSpeed) _maxSpeed
17             else if(IsGoingBackward && value < MinSpeed) MinSpeed
18             else if(!IsGoingBackward && value < 0) 0
19             else value
20     }
21     def MaxSpeed_=(value: Float): Unit = {
22         _maxSpeed = value
23         if (_speed > _maxSpeed) _speed = _maxSpeed
24     }
25
26     def IsGoingBackward: Boolean = _isGoingBackward

```

```

27 def IsGoingBackward_=(value: Boolean): Unit = {
28   _isGoingBackward = value
29 }
30
31 def Moving(elapsedTime: Float, factor: Float = 10f): Unit = {
32   Coordinates.y += math.cos(Rotation).toFloat * Speed * elapsedTime * factor
33   Coordinates.x += math.sin(Rotation).toFloat * Speed * elapsedTime * factor
34 }
35 }

```

9.2.17 – Orientable.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 trait Orientable {
5   private var _rotation: Float = 0f;
6
7   def Rotation: Float = _rotation
8
9   def Rotation_=(value: Float): Unit = {
10    _rotation = value
11  }
12 }

```

9.2.18 – Critical.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.problems
2
3 trait Critical

```

9.2.19 – Problem.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.problems
2
3 import ch.hevs.fastandmudry.core.ecs.systems.Car
4
5 trait Problem {
6   private var _isDefected = false;
7   def IsDefected: Boolean = _isDefected
8   def IsDefected_=(value: Boolean): Unit = { _isDefected = value }
9
10  def impactCar(elapsedTime: Float, car: Car): Unit
11
12  def reset(): Unit = {
13    IsDefected = false;
14  }
15 }

```

9.2.20 – Repairable.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components.problems
2
3 trait Repairable extends Problem {
4   private var _Title: String = ""
5   private var _ReparationPrice: Int = 0
6   private var _IconPath: String = "data/images/item/SimpleRock.png"
7   private var _ButtonText: String = "Repare"
8
9   def Title: String = _Title
10  protected def Title_=(value: String): Unit = {
11    _Title = value

```

```

12 }
13
14 def ReparationPrice: Int = _ReparationPrice
15 protected def ReparationPrice_=(value: Int): Unit = {
16     _ReparationPrice = value
17 }
18
19 def IconPath: String = _IconPath
20 protected def IconPath_=(value: String): Unit = {
21     _IconPath = value
22 }
23
24 def ButtonText: String = _ButtonText
25 private def ButtonText_=(value: String): Unit = {
26     _ButtonText = value
27 }
28
29 }

```

9.2.21 – Side.scala

```

1 package ch.hevs.fastandmudry.core.ecs.components
2
3 object Side extends Enumeration {
4     type Side = Value
5     val Left, Right = Value
6 }

```

9.2.22 – Steerable.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 import com.badlogic.gdx.Input
5
6 trait Steerable {
7     private var _wheelAngle: Float = 0f
8     private var _IsSteeringWheelReturnEnable: Boolean = false
9
10    var leftKey = Input.Keys.LEFT
11    var rightKey = Input.Keys.RIGHT
12
13    def IsSteeringWheelReturnEnable: Boolean = _IsSteeringWheelReturnEnable
14
15    def IsSteeringWheelReturnEnable_=(value: Boolean): Unit = {
16        _IsSteeringWheelReturnEnable = value
17    };
18
19    def WheelAngle: Float = _wheelAngle
20
21    def WheelAngle_=(value: Float): Unit = {
22        _wheelAngle = value
23    }
24 }

```

9.2.23 – Temperable.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.components
3
4 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR
5
6 trait Temperable {
7     private var _temperature: Float = 0f

```

```

8   private var _minTemp: Float = CAR.FACTOR.MIN_TEMPERATURE
9   private var _maxTemp: Float = CAR.FACTOR.MAX_TEMPERATURE
10
11  def Temperature: Float = _temperature
12  def MinTemperature: Float = _minTemp
13  def MaxTemperature: Float = _maxTemp
14
15  def Temperature_=(value: Float): Unit = {
16    _temperature = value
17  }
18 }

```

9.2.24 – AItem.scala

```

1  package ch.hevs.fastandmudry
2  package core.ecs.entities.Item
3
4  import ch.hevs.fastandmudry.core.ecs.components.Collision.Collisional
5  import ch.hevs.fastandmudry.core.ecs.components.Locatable
6  import ch.hevs.fastandmudry.core.ecs.systems.track.TrackGeometry
7  import ch.hevs.fastandmudry.core.world.World
8  import com.badlogic.gdx.graphics.Texture
9
10 abstract class AItem extends Locatable {
11   private var isColliding: Boolean = false
12
13   def isOnTrackImportant = true
14   def getIsOnTrack: Boolean = false
15   def getTexture: Texture
16   def getMaxSize: (Int, Int) = (getTexture.getWidth, getTexture.getHeight)
17   def checkStats(): Unit = {
18     if(isInstanceOf[Collisional]) {
19       val collisionnable = this.asInstanceOf[Collisional]
20       val collides = collisionnable.CheckCollision(World.INSTANCE.CAR)
21       if(collides && !isColliding) collisionnable.onCollision(World.INSTANCE.CAR)
22       isColliding = collides
23     }
24   }
25
26   final def isGenerationAllowed(geometry: TrackGeometry): Boolean = {
27     isOnTrackImportant && geometry.isOffRoad(this.Coordinates) == getIsOnTrack
28   }
29 }

```

9.2.25 – Cactus.scala

```

1  package ch.hevs.fastandmudry.core.ecs.entities.Item.desert
2
3  import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4  DefectChassis}
5  import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6  import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.DESERT.CACTUS
7  import com.badlogic.gdx.graphics.Texture
8
9  class Cactus extends AItem with CircleCollision with DefectChassis {
10    this.Size = 50
11
12    private lazy val texture = ItemTextures.Desert.Cactus
13
14    override def getTexture: Texture = texture
15
16    override def getMaxSize: (Int, Int) = (getTexture.getWidth *
    CACTUS.RENDERING_FACTOR, getTexture.getHeight * CACTUS.RENDERING_FACTOR)
17  }

```

9.2.26 – DeserticRock.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.Item.desert
2
3 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4 NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.DESERT.DESERTIC_ROCK
7 import com.badlogic.gdx.graphics.Texture
8
9 class DeserticRock extends AItem with CircleCollision with NoDefect {
10     override def isOnTrackImportant: Boolean = false
11     override def getIsOnTrack: Boolean = true
12     private lazy val texture = ItemTextures.Desert.DeserticRock
13     override def getTexture: Texture = texture
14     override def getMaxSize: (Int, Int) =
15     (getTexture.getWidth*DESERTIC_ROCK.RENDERING_FACTOR,
16      getTexture.getHeight*DESERTIC_ROCK.RENDERING_FACTOR)
17 }

```

9.2.27 – DeserticSkeleton.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.Item.desert
2
3 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4 NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.DESERT.DESERTIC_SKELETON
7 import com.badlogic.gdx.graphics.Texture
8
9 class DeserticSkeleton extends AItem with CircleCollision with NoDefect {
10     override def getIsOnTrack: Boolean = false
11     private lazy val texture = ItemTextures.Desert.DeserticSkeleton
12     override def getTexture: Texture = texture
13     override def getMaxSize: (Int, Int) =
14     (getTexture.getWidth*DESERTIC_SKELETON.RENDERING_FACTOR,
15      getTexture.getHeight*DESERTIC_SKELETON.RENDERING_FACTOR)
16 }

```

9.2.28 – SimpleBush.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.Item.forest
2
3 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4 NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.FOREST.SIMPLE_BUSH
7 import com.badlogic.gdx.graphics.Texture
8
9 class SimpleBush extends AItem with CircleCollision with NoDefect {
10     this.Size = 25
11     private lazy val texture = ItemTextures.Forest.SimpleBush
12     override def getTexture: Texture = texture
13     override def getMaxSize: (Int, Int) = (getTexture.getWidth *
14     SIMPLE_BUSH.RENDERING_FACTOR, getTexture.getHeight * SIMPLE_BUSH.RENDERING_FACTOR)
15 }

```

9.2.29 – SimpleRock.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.entities.Item.forest
3

```

```

4 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import
7 ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.FOREST.SIMPLE_ROCK.RENDERING_FACTOR
8 import com.badlogic.gdx.graphics.Texture
9
10 class SimpleRock extends AItem with CircleCollision with NoDefect {
11     override def isOnTrackImportant: Boolean = false
12     override def getIsOnTrack: Boolean = true
13     private lazy val texture = ItemTextures.Forest.SimpleRock
14     override def getTexture: Texture = texture
15     override def getMaxSize: (Int, Int) = (getTexture.getWidth*RENDERING_FACTOR,
getTexture.getHeight*RENDERING_FACTOR)
16 }

```

9.2.30 – SimpleTree.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.entities.Item.forest
3
4 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
DefectChassis}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import
7 ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.FOREST.SIMPLE_TREE.RENDERING_FACTOR
8 import com.badlogic.gdx.graphics.Texture
9
10 class SimpleTree extends AItem with CircleCollision with DefectChassis {
11     this.Size = 50
12
13     private lazy val texture = ItemTextures.Forest.SimpleTree
14
15     override def getTexture: Texture = texture
16
17     override def getMaxSize: (Int, Int) = (getTexture.getWidth * RENDERING_FACTOR,
getTexture.getHeight * RENDERING_FACTOR)
18 }

```

9.2.31 – HES.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.entities.Item
3
4 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.HES.RENDERING_FACTOR
5 import com.badlogic.gdx.graphics.Texture
6
7
8 class HES extends AItem {
9     override def getTexture: Texture = ItemTextures.HES
10     override def getMaxSize: (Int, Int) = (getTexture.getWidth*RENDERING_FACTOR,
getTexture.getHeight*RENDERING_FACTOR)
11 }

```

9.2.32 – ItemTextures.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.entities.Item
3
4 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS
5 import com.badlogic.gdx.graphics.Texture
6
7 import java.io.File

```



```

8  import scala.util.Random
9
10
11  object ItemTextures {
12      private lazy val _random = new Random()
13      private def getAllTexture(src: String): List[Texture] = {
14          val folder = new File(src)
15          folder.listFiles()
16              .filter(file => file.isFile && file.getName.toLowerCase.endsWith(".png"))
17              .map[Texture](f => new Texture(f.getPath)).toList
18      }
19      object Forest {
20          private lazy val SimpleTrees =
21              getAllTexture(ITEMS.FOREST.SIMPLE_TREE.IMAGE_SOURCE)
22          private lazy val SimpleRocks =
23              getAllTexture(ITEMS.FOREST.SIMPLE_ROCK.IMAGE_SOURCE)
24          private lazy val SimpleBushs =
25              getAllTexture(ITEMS.FOREST.SIMPLE_BUSH.IMAGE_SOURCE)
26
27          def SimpleTree: Texture = SimpleTrees(_random.nextInt(SimpleTrees.length))
28          def SimpleRock: Texture = SimpleRocks(_random.nextInt(SimpleRocks.length))
29          def SimpleBush: Texture = SimpleBushs(_random.nextInt(SimpleBushs.length))
30      }
31
32      object Desert {
33          private lazy val Cactuss = getAllTexture(ITEMS.DESERT.CACTUS.IMAGE_SOURCE)
34          private lazy val DeserticRocks =
35              getAllTexture(ITEMS.DESERT.DESERTIC_ROCK.IMAGE_SOURCE)
36          private lazy val DeserticSkeletons =
37              getAllTexture(ITEMS.DESERT.DESERTIC_SKELETON.IMAGE_SOURCE)
38
39          def Cactus: Texture = Cactuss(_random.nextInt(Cactuss.length))
40          def DeserticRock: Texture =
41              DeserticRocks(_random.nextInt(DeserticRocks.length))
42          def DeserticSkeleton: Texture =
43              DeserticSkeletons(_random.nextInt(DeserticSkeletons.length))
44      }
45
46      object Snow {
47          private lazy val PinTrees = getAllTexture(ITEMS.SNOW.PIN_TREE.IMAGE_SOURCE)
48          private lazy val SnowyRocks = getAllTexture(ITEMS.SNOW.SNOWY_ROCK.IMAGE_SOURCE)
49          private lazy val SnowPiles = getAllTexture(ITEMS.SNOW.SNOW_PILE.IMAGE_SOURCE)
50          def PinTree: Texture = PinTrees(_random.nextInt(PinTrees.length))
51          def SnowyRock: Texture = SnowyRocks(_random.nextInt(SnowyRocks.length))
52          def SnowPile: Texture = SnowPiles(_random.nextInt(SnowPiles.length))
53      }
54
55      lazy val HES = new Texture(ITEMS.HES.IMAGE_SOURCE)
56  }

```

9.2.33 – PinTree.scala

```

1  package ch.hevs.fastandmudry.core.ecs.entities.Item.snow
2
3  import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4      DefectChassis}
5  import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6  import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.SNOW.PIN_TREE
7  import com.badlogic.gdx.graphics.Texture
8
9  class PinTree extends AItem with CircleCollision with DefectChassis {
10      this.Size = 50
11
12      private lazy val texture = ItemTextures.Snow.PinTree

```

```

12
13     override def getTexture: Texture = texture
14
15     override def getMaxSize: (Int, Int) = (getTexture.getWidth *
16     PIN_TREE.RENDERING_FACTOR, getTexture.getHeight * PIN_TREE.RENDERING_FACTOR)
17 }

```

9.2.34 – SnowPile.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.Item.snow
2
3 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4 NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.SNOW.SNOW_PILE
7 import com.badlogic.gdx.graphics.Texture
8
9 class SnowPile extends AItem with CircleCollision with NoDefect {
10     this.Size = 25
11     override def isOnTrackImportant: Boolean = false
12     private lazy val texture = ItemTextures.Snow.SnowPile
13     override def getTexture: Texture = texture
14     override def getMaxSize: (Int, Int) = (getTexture.getWidth *
15     SNOW_PILE.RENDERING_FACTOR, getTexture.getHeight * SNOW_PILE.RENDERING_FACTOR)
16 }

```

9.2.35 – SnowyRock.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.Item.snow
2
3 import ch.hevs.fastandmudry.core.ecs.components.Collision.{CircleCollision,
4 NoDefect}
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, ItemTextures}
6 import ch.hevs.fastandmudry.utils.Constant.MapTexture.ITEMS.SNOW.SNOWY_ROCK
7 import com.badlogic.gdx.graphics.Texture
8
9 class SnowyRock extends AItem with CircleCollision with NoDefect {
10     override def isOnTrackImportant: Boolean = false
11     override def getIsOnTrack: Boolean = true
12     private lazy val texture = ItemTextures.Snow.SnowyRock
13     override def getTexture: Texture = texture
14     override def getMaxSize: (Int, Int) =
15     (getTexture.getWidth*SNOWY_ROCK.RENDERING_FACTOR,
16     getTexture.getHeight*SNOWY_ROCK.RENDERING_FACTOR)
17 }

```

9.2.36 – ChassisProblem.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.problems
2
3 import ch.hevs.fastandmudry.core.ecs.components.problems.{Problem, Repairable}
4 import ch.hevs.fastandmudry.core.ecs.systems.Car
5
6 class ChassisProblem extends Problem with Repairable {
7     Title = "Chassis"
8     ReparationPrice = 100
9     IconPath = "data/images/problems/chassis.png"
10
11     private var hasChanged = false
12     override def impactCar(elapsedTime: Float, car: Car): Unit = {
13         if(IsDefected && !hasChanged) {
14             car.MaxSpeed -= 10f
15             hasChanged = true
16         }
17     }
18 }

```

```

16     }
17   }
18 }

```

9.2.37 – TemperatureProblem.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.problems
2
3 import ch.hevs.fastandmudry.core.ecs.components.problems.{Critical, Problem}
4 import ch.hevs.fastandmudry.core.ecs.systems.Car
5 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR
6
7 class TemperatureProblem(val Car: Car) extends Problem with Critical {
8   override def impactCar(elapsedTime: Float, car: Car): Unit = {
9     }
10
11   def updateBroken(elapsedTime: Float, factor: Float): Unit = {
12     Car.Temperature += factor * elapsedTime
13
14     if(Car.Temperature ≤ CAR.FACTOR.MIN_TEMPERATURE || Car.Temperature ≥
15     CAR.FACTOR.MAX_TEMPERATURE) IsDefected = true
16   }
17 }

```

9.2.38 – TireProblem.scala

```

1 package ch.hevs.fastandmudry.core.ecs.entities.problems
2
3 import ch.hevs.fastandmudry.core.audio.AudioManager
4 import ch.hevs.fastandmudry.core.ecs.components
5 import ch.hevs.fastandmudry.core.ecs.components.Axle.Axle
6 import ch.hevs.fastandmudry.core.ecs.components.Side.Side
7 import ch.hevs.fastandmudry.core.ecs.components.problems.{Problem, Repairable}
8 import ch.hevs.fastandmudry.core.ecs.systems.Car
9 import ch.hevs.fastandmudry.ui.hud.DebugHUD
10 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR.FACTOR
11
12 import scala.util.Random
13
14 class TireProblem(val Side: Side, val Axle: Axle) extends Problem with Repairable {
15   Title = s"$Axle $Side tire"
16   ReparationPrice = 25
17   IconPath = "data/images/problems/wheel.png"
18
19   override def impactCar(elapsedTime: Float, car: Car): Unit = {
20     DebugHUD.setLogVar(s"Tire - ${Axle} ${Side} Perforated", IsDefected)
21
22     if(IsDefected) {
23       if(Axle == components.Axle.Front) {
24         if((Side == components.Side.Right && car.WheelAngle ≥ 0) || (Side ==
25         components.Side.Left && car.WheelAngle ≤ 0)){
26           car.WheelAngle += (if(Side == components.Side.Right) 1 else -1) *
27           FACTOR.WHEEL_ROTATION * elapsedTime * 0.1f
28           car.IsSteeringWheelReturnEnable = false
29         }
30       } else {
31         car.MaxSpeed *= 0.5f
32       }
33     }
34
35     def updateBroken(prob: Float): Unit = {
36       if(!IsDefected && Random.nextFloat() < prob) {
37         IsDefected = true
38       }
39     }
40   }
41 }

```

```

37     AudioManager.playTireExplosion()
38   }
39 }
40 }

```

9.2.39 – TireSlippageProblem.scala

```

1  package ch.hevs.fastandmudry.core.ecs.entities.problems
2
3  import ch.hevs.fastandmudry.core.ecs.components.problems.Problem
4  import ch.hevs.fastandmudry.core.ecs.systems.Car
5  import com.badlogic.gdx.Input
6
7  class TireSlippageProblem extends Problem {
8
9      override def impactCar(elapsedTime: Float, car: Car): Unit = {
10         if(IsDefected) {
11             car.rightKey = Input.Keys.LEFT
12             car.leftKey = Input.Keys.RIGHT
13         } else {
14             car.rightKey = Input.Keys.RIGHT
15             car.leftKey = Input.Keys.LEFT
16         }
17     }
18 }

```

9.2.40 – Car.scala

```

1  package ch.hevs.fastandmudry
2  package core.ecs.systems
3
4  import ch.hevs.fastandmudry.core.ecs.components._
5  import ch.hevs.fastandmudry.core.ecs.components.problems.{Critical, Problem,
6  Repairable}
7  import ch.hevs.fastandmudry.core.ecs.entities.problems.{ChassisProblem,
8  TemperatureProblem, TireProblem, TireSlippageProblem}
9  import ch.hevs.fastandmudry.ui.hud.DebugHUD
10 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR.FACTOR
11 import com.badlogic.gdx.{Gdx, Input}
12
13 import scala.collection.mutable.ArrayBuffer
14
15 class Car extends AGameLoop with Orientable with Moveable with Steerable with
16 Temperable with GodMode {
17     MaxSpeed = FACTOR.MAX_SPEED
18     var isBroken: Boolean = false
19
20     val FrontLeftTire = new TireProblem(Side.Left, Axle.Front)
21     val BackLeftTire = new TireProblem(Side.Left, Axle.Rear)
22     val FrontRightTire = new TireProblem(Side.Right, Axle.Front)
23     val BackRightTire = new TireProblem(Side.Right, Axle.Rear)
24     val TemperatureProblem = new TemperatureProblem(this)
25     val TireSlippage = new TireSlippageProblem
26     val ChassisProblem = new ChassisProblem
27
28     private val Problems = ArrayBuffer[Problem](
29         FrontLeftTire,
30         BackLeftTire,
31         FrontRightTire,
32         BackRightTire,
33         TemperatureProblem,
34         TireSlippage,
35         ChassisProblem
36     )
37 }

```

```

34
35 def getReparableProblems: ArrayBuffer[Reparable] = Problems.collect[Reparable] {
36   case r: Reparable => r
37 }
38
39 override def onGameLoop(elapsedTime: Float): Unit = {
40   checkGodMode()
41   IsSteeringWheelReturnEnable = true;
42   if (Gdx.input.isKeyPressed(Input.Keys.UP)) {
43     Speed += FACTOR.ACCELERATION * elapsedTime * (if(IsGoingBackward) 4f else 1f)
44     if(Speed ≥ 0) IsGoingBackward = false
45   }
46   else if (Gdx.input.isKeyPressed(Input.Keys.DOWN)) {
47     IsGoingBackward = true
48     Speed -= FACTOR.ACCELERATION * elapsedTime * (if(Speed > 0) 4f else 1f)
49   }
50   else {
51     Speed -= (if(IsGoingBackward) -1 else 1) * FACTOR.DECCELERATION * elapsedTime
52     if(Speed ≥ 0) IsGoingBackward = false
53   }
54
55   if (Gdx.input.isKeyPressed(leftKey)) {
56     WheelAngle -= FACTOR.WHEEL_ROTATION * elapsedTime
57     IsSteeringWheelReturnEnable = false
58   }
59   if (Gdx.input.isKeyPressed(rightKey)) {
60     WheelAngle += FACTOR.WHEEL_ROTATION * elapsedTime
61     IsSteeringWheelReturnEnable = false
62   }
63
64   if(!IsGodModeEnable) Problems.foreach(_.impactCar(elapsedTime, this))
65
66   if(IsSteeringWheelReturnEnable) {
67     WheelAngle *= Math.pow(FACTOR.WHEEL_RETURN, elapsedTime.toDouble).toFloat
68     if (Math.abs(WheelAngle) < 0.001f) WheelAngle = 0f
69   }
70
71   if(WheelAngle > FACTOR.WHEEL_MAX_ANGLE) WheelAngle = FACTOR.WHEEL_MAX_ANGLE
72   if(WheelAngle < -FACTOR.WHEEL_MAX_ANGLE) WheelAngle = -FACTOR.WHEEL_MAX_ANGLE
73
74   if (Math.abs(Speed) > 0.01f) {
75     Rotation += Speed * WheelAngle * elapsedTime
76     Rotation %= (Math.PI * 2).toFloat
77   }
78
79   Temperature += 0.5f * elapsedTime
80   isBroken = checkCarState()
81
82   DebugHUD.setLogVar("Car - Rotation", Rotation)
83   DebugHUD.setLogVar("Car - WheelAngle", WheelAngle)
84   DebugHUD.setLogVar("Car - Speed", Speed)
85   DebugHUD.setLogVar("Car - Temperature", Temperature)
86 }
87
88 def reset(): Unit = {
89   MaxSpeed = FACTOR.MAX_SPEED
90   Speed = 0f
91   WheelAngle = 0f
92   Temperature = 0f
93   isBroken = false
94 }
95
96 def checkCarState(): Boolean = {
97   Problems.filter(_.isInstanceOf[Critical]).exists(_.IsDefected)
98 }
99 }

```

9.2.41 – MapGenerator.scala

```

1  package ch.hevs.fastandmudry
2  package core.ecs.systems
3
4  import ch.hevs.gdx2d.lib.utils.catmull.CatmullRomUtils
5  import com.badlogic.gdx.math.Vector2
6
7  import scala.collection.mutable.ArrayBuffer
8  /*
9   The CatmullChain code has been taken from the
10  ch.hevs.gdx2d.components.physics.PhysicsChain lib
11  and been modified to adapt it to our situation.
12  */
13  class MapGenerator(protected var start: Vector2, protected var stop: Vector2,
14  nControlPoints: Int) {
15    val mainChain: ArrayBuffer[Vector2] = generateCatmullChain(nControlPoints)
16    val leftChain: ArrayBuffer[Vector2] = createChainWithOffset(10f, mainChain)
17    val rightChain: ArrayBuffer[Vector2] = createChainWithOffset(-10f, mainChain)
18
19    println("####")
20    println(s"x: ${mainChain(0).x} // y: ${mainChain(0).y}")
21    println(s"x: ${mainChain(1).x} // y: ${mainChain(1).y}")
22
23    /**
24     * Create a chain from {@link # start} to {@link # stop} using {@code nPoints}.
25     *
26     * @param nPoints The number of points from the start to the stop position
27     */
28    private def generateCatmullChain(nPoints: Int): ArrayBuffer[Vector2] = {
29      var vertices = randomVertices(nPoints, 0.8f)
30      // Interpolates new points with a Catmull-Rom spline, using 9 subdivisions per
31      segment
32      val spline: Array[Vector2] = CatmullRomUtils.subdividePoints(vertices.toArray,
33      9)
34      // Replace the existing vertices by the spline generated
35      vertices.clear()
36      for (i ← spline.indices) {
37        vertices.addOne(spline(i))
38      }
39      return vertices
40    }
41    private def randomVertices(nPoints: Int, randomHeight: Float):
42    ArrayBuffer[Vector2] = {
43      val vertices: ArrayBuffer[Vector2] = new ArrayBuffer[Vector2]
44      val width: Float = (stop.x - start.x) / (nPoints - 1)
45      val height: Float = (stop.y - start.y) / (nPoints - 1)
46      for (i ← 0 until nPoints) {
47        var h: Float = start.y + height * i
48        h += (randomHeight * (Math.random * h)).toFloat
49        val p: Vector2 = new Vector2(start.x + width * i, h)
50        vertices.addOne(p)
51      }
52      vertices
53    }
54    private def createChainWithOffset(offset: Float, chain: ArrayBuffer[Vector2]):
55    ArrayBuffer[Vector2] = {
56      val newChain: ArrayBuffer[Vector2] = new ArrayBuffer[Vector2]
57      for (i ← chain.indices) {
58        val v = chain(i)
59        newChain.addOne(new Vector2(v.x, v.y + offset))
60      }
61      return newChain
62    }
63  }
64
65  // AI-generated just in test phase for previsualisation

```

```

60 def printAsciiPreview(cols: Int = 120, rows: Int = 30): Unit = {
61   if (mainChain.isEmpty) { println("(empty chain)"); return }
62   var minX = Float.PositiveInfinity; var maxX = Float.NegativeInfinity
63   var minY = Float.PositiveInfinity; var maxY = Float.NegativeInfinity
64   for (i ← mainChain.indices) {
65     val v = mainChain(i)
66     if (v.x < minX) minX = v.x
67     if (v.x > maxX) maxX = v.x
68     if (v.y < minY) minY = v.y
69     if (v.y > maxY) maxY = v.y
70   }
71
72   // Aspect-ratio preserving scale:
73   // X gets the full column width; Y is mapped so 1 row covers the SAME world
74   // distance as 1 column (times a terminal-aspect fudge factor since character
75   // cells are taller than wide). This stops a small jitter from filling the
76   // whole vertical space and looking like giant pikes.
77   val charAspect = 2.0f
78   val rangeX = math.max(maxX - minX, 1e-3f)
79   val unitsPerCol = rangeX / (cols - 1)
80   val unitsPerRow = unitsPerCol * charAspect
81
82   // Pick just enough rows to hold the actual Y span (+ a 1-row pad each side).
83   val rangeY = math.max(maxY - minY, 1e-3f)
84   val rowsNeeded = math.ceil(rangeY / unitsPerRow).toInt + 2
85   val actualRows = math.max(1, math.min(rows, rowsNeeded))
86
87   // Center the view on the road's Y midline.
88   val centerY = (minY + maxY) / 2f
89   val halfH = (actualRows / 2f) * unitsPerRow
90   val viewMinY = centerY - halfH
91   val viewMaxY = centerY + halfH
92
93   val grid = Array.fill(actualRows, cols)(' ')
94   for (i ← mainChain.indices) {
95     val v = mainChain(i)
96     val gx = (((v.x - minX) / rangeX) * (cols - 1)).toInt
97     // Flip Y so larger Y appears at the top.
98     val gy = (((viewMaxY - v.y) / (viewMaxY - viewMinY)) * (actualRows -
99 1)).toInt
100    if (gx ≥ 0 && gx < cols && gy ≥ 0 && gy < actualRows) grid(gy)(gx) = '#'
101  }
102  for (i ← leftChain.indices) {
103    val v = leftChain(i)
104    val gx = (((v.x - minX) / rangeX) * (cols - 1)).toInt
105    // Flip Y so larger Y appears at the top.
106    val gy = (((viewMaxY - v.y) / (viewMaxY - viewMinY)) * (actualRows -
107 1)).toInt
108    if (gx ≥ 0 && gx < cols && gy ≥ 0 && gy < actualRows) grid(gy)(gx) = '*'
109  }
110  for (i ← rightChain.indices) {
111    val v = rightChain(i)
112    val gx = (((v.x - minX) / rangeX) * (cols - 1)).toInt
113    // Flip Y so larger Y appears at the top.
114    val gy = (((viewMaxY - v.y) / (viewMaxY - viewMinY)) * (actualRows -
115 1)).toInt
116    if (gx ≥ 0 && gx < cols && gy ≥ 0 && gy < actualRows) grid(gy)(gx) = ' '
117  }
118  val border = "+" + ("-" * cols) + "+"
119  println(f"MapGenerator preview (${mainChain.size} samples, x=[$minX%.1f..
120 $maxX%.1f] y=[$minY%.1f.. $maxY%.1f], ${unitsPerCol}%.1f px/col)")
121  println(border)
122  grid.foreach(row => println("|" + row.mkString + "|"))
123  println(border)
124 }

```



```

122
123 object MapGeneratorPreview {
124     def main(args: Array[String]): Unit = {
125         val gen = new MapGenerator(new Vector2(0f, 100f), new Vector2(1000f, 100f), 8)
126         gen.printAsciiPreview(800, 1600)
127     }
128 }

```

9.2.42 – TrackGeometry.scala

```

1  package ch.hevs.fastandmudry
2  package core.ecs.systems.track
3
4  import ch.hevs.fastandmudry.utils.Constant.MapTexture
5  import ch.hevs.gdx2d.lib.utils.catmull.CatmullRomUtils
6  import com.badlogic.gdx.math.{Rectangle, Vector2}
7
8  import scala.collection.mutable.ArrayBuffer
9
10 class TrackGeometry(start: Vector2, stop: Vector2, nControlPoints: Int) {
11     private val halfRoadWidth: Float = MapTexture.HALF_ROAD_WIDTH
12     private val halfShoulderWidth: Float = MapTexture.HALF_SHOULDER_WIDTH
13     private val centerLine: ArrayBuffer[Vector2] =
14 generateCatmullChain(nControlPoints)
15     private val maxRadius: Float = halfShoulderWidth + MapTexture.SHOULDER_JITTER
16
17     lazy val trackSize: Rectangle = calculateTrackSize()
18
19     def CenterLine: ArrayBuffer[Vector2] = centerLine
20     def FinishPoint: Vector2 = centerLine(centerLine.length - 1)
21     def HalfRoadWidth: Float = halfRoadWidth
22     def HalfShoulderWidth: Float = halfShoulderWidth
23
24     private case class SegmentBounds(startVector: Vector2, endVector: Vector2, minX:
25 Float, maxX: Float, minY: Float, maxY: Float)
26     private lazy val segmentBounds: Array[SegmentBounds] = {
27         val bounds = new ArrayBuffer[SegmentBounds]
28         for (i ← 0 until centerLine.length - 1) {
29             val startVector = centerLine(i)
30             val endVector = centerLine(i + 1)
31
32             // make a box tight to the segment
33             val innerMinX = math.min(startVector.x, endVector.x)
34             val innerMaxX = math.max(startVector.x, endVector.x)
35             val innerMinY = math.min(startVector.y, endVector.y)
36             val innerMaxY = math.max(startVector.y, endVector.y)
37
38             // add a radius so it also covers the road and shoulder
39             val minX = innerMinX - maxRadius
40             val maxX = innerMaxX + maxRadius
41             val minY = innerMinY - maxRadius
42             val maxY = innerMaxY + maxRadius
43
44             val box = SegmentBounds(startVector, endVector, minX, maxX, minY, maxY)
45             bounds.addOne(box)
46         }
47         bounds.toArray
48     }
49
50     def distToCenterLineSq(p: Vector2): Float =
51         distToCenterLineSq(p.x, p.y)
52
53     def distToCenterLineSq(px: Float, py: Float): Float = {
54         var minDist = 999999f
55         var i = 0

```



```

55 // while loop is more performant in this case (runned millions/billions of
times) than a for loop
56 while (i < segmentBounds.length) {
57     val b = segmentBounds(i)
58     // only do math if it is in the segment bound (increase performance)
59     if (px ≥ b.minX && px ≤ b.maxX && py ≥ b.minY && py ≤ b.maxY) {
60         val d = distToSegmentSquared(px, py, b.startVector, b.endVector)
61         if (d < minDist) minDist = d
62     }
63     i += 1
64 }
65 return minDist
66 }
67
68 // Taken from: https://stackoverflow.com/questions/849211/shortest-distance-
between-a-point-and-a-line-segment
69 private def sqr(x: Float): Float = x * x
70 private def dist2(v: Vector2, w: Vector2): Float = sqr(v.x - w.x) + sqr(v.y -
w.y)
71
72 private def distToSegmentSquared(px: Float, py: Float, v: Vector2, w: Vector2):
Float = {
73     val l2 = dist2(v, w)
74     if (l2 == 0f) return sqr(px - v.x) + sqr(py - v.y)
75     var t = ((px - v.x) * (w.x - v.x) + (py - v.y) * (w.y - v.y)) / l2
76     t = math.max(0f, math.min(1f, t))
77     val x = v.x + t * (w.x - v.x)
78     val y = v.y + t * (w.y - v.y)
79     return sqr(px - x) + sqr(py - y)
80 }
81
82 def isOffRoad(p: Vector2): Boolean =
83     distToCenterLineSq(p) > halfShoulderWidth * halfShoulderWidth
84
85 /**
86  * Create a chain from {@link # start} to {@link # stop} using {@code nPoints}.
87  *
88  * @param nPoints The number of points from the start to the stop position
89  */
90 private def generateCatmullChain(nPoints: Int): ArrayBuffer[Vector2] = {
91     val vertices = randomVertices(nPoints, 200f)
92     // Interpolates new points with a Catmull-Rom spline, using 30 subdivisions per
segment
93     val spline: Array[Vector2] = CatmullRomUtils.subdividePoints(vertices.toArray,
30)
94     // Replace the existing vertices by the spline generated
95     vertices.clear()
96     for (i ← spline.indices) {
97         vertices.addOne(spline(i))
98     }
99     return vertices
100 }
101 private def randomVertices(nPoints: Int, randomHeight: Float):
ArrayBuffer[Vector2] = {
102     val vertices: ArrayBuffer[Vector2] = new ArrayBuffer[Vector2]
103     val width: Float = (stop.x - start.x) / (nPoints - 1)
104     val height: Float = (stop.y - start.y) / (nPoints - 1)
105     for (i ← 0 until nPoints) {
106         val yNoise = ((Math.random - 0.5) * 2.0 * randomHeight).toFloat
107         val h: Float = start.y + height * i + yNoise
108         val xNoise = if (i == 0 || i == nPoints - 1) 0f else ((Math.random - 0.5) *
2.0 * width * 0.4).toFloat
109         val p: Vector2 = new Vector2(start.x + width * i + xNoise, h)
110         vertices.addOne(p)
111     }
112     vertices

```

```

113 }
114
115 def isFinishLine(p: Vector2): Boolean =
116     isFinishLine(p.x, p.y)
117
118 def isFinishLine(px: Float, py: Float): Boolean =
119     sqr(px - FinishPoint.x) + sqr(py - FinishPoint.y) ≤ halfRoadWidth *
halfRoadWidth
120
121 private def calculateTrackSize(): Rectangle = {
122     var minX = 999999999999f
123     var maxX = 0f
124     var minY = 999999999999f
125     var maxY = 0f
126
127     def checkMinMax(chain: ArrayBuffer[Vector2]): Unit = {
128         for (i ← chain.indices) {
129             val v = chain(i)
130             if (v.x < minX) minX = v.x
131             if (v.x > maxX) maxX = v.x
132             if (v.y < minY) minY = v.y
133             if (v.y > maxY) maxY = v.y
134         }
135     }
136
137     checkMinMax(centerLine)
138
139     return new Rectangle(minX, minY, maxX - minX, maxY - minY)
140 }
141 }

```

9.2.43 – Track.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.systems.track
3
4 import ch.hevs.fastandmudry.core.ecs.components.AGameLoop
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.{AItem, HES}
6 import ch.hevs.fastandmudry.core.ecs.systems.Car
7 import ch.hevs.fastandmudry.core.state.{CarBroke, FinishLineCrossed,
GameStateMachine}
8 import ch.hevs.fastandmudry.core.world.biome.{Biome, DesertBiome}
9 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR.FACTOR
10 import com.badlogic.gdx.graphics.Texture.TextureFilter
11 import com.badlogic.gdx.graphics.{Pixmap, PixmapIO, Texture}
12 import com.badlogic.gdx.math.Vector2
13 import com.badlogic.gdx.{Gdx, Input}
14
15 class Track(private val Car: Car) extends AGameLoop {
16     private var geometry: TrackGeometry = _
17     private var texture: Texture = _
18     private var finished: Boolean = false
19
20     var biome: Biome = new DesertBiome()
21
22     def Geometry: TrackGeometry = geometry
23     def Texture: Texture = texture
24
25     def setBiome(biome: Biome): Unit = {
26         this.biome = biome
27     }
28
29     def buildPixmap(): Pixmap = {
30         geometry = new TrackGeometry(new Vector2(0f, 100f), new Vector2(5000f, 100f),
20)
31         TrackTexture.generate(geometry, biome)

```

```

32 }
33
34 def installPixmap(pixmap: Pixmap): Unit = {
35     // for debug
36     PixmapIO.writePNG(Gdx.files.local("track_debug.png"), pixmap)
37
38     texture = new Texture(pixmap)
39     texture.setFilter(TextureFilter.Linear, TextureFilter.Linear)
40     pixmap.dispose()
41
42     val spawn = geometry.CenterLine(0)
43     val end = geometry.CenterLine(1)
44     Car.reset()
45     Car.Coordinates = new Vector2(spawn.x, spawn.y)
46     Car.Rotation = math.atan2(end.x - spawn.x, end.y - spawn.y).toFloat
47
48     generateMapItems()
49     finished = false
50 }
51
52 override def onGameLoop(elapsedTime: Float): Unit = {
53     if (geometry == null) return
54     if (finished) return
55     if (geometry.isFinishLine(Car.Coordinates)) {
56         println("Finish line crossed!")
57         finished = true
58         GameStateMachine.handle(FinishLineCrossed)
59         return
60     }
61     val isOffRoad = geometry.isOffRoad(Car.Coordinates)
62     Car.MaxSpeed = if (!isOffRoad || Car.IsGodModeEnable) FACTOR.MAX_SPEED else
63     FACTOR.MAX_SPEED * biome.offRoadDecreasingFactorSpeed
64     biome.updatePhysics(Car, isOffRoad, elapsedTime)
65
66     mapItems.foreach(i => i.checkStats())
67
68     if (Car.isBroken) {
69         finished = true
70         GameStateMachine.handle(CarBroke)
71         return
72     }
73
74     if (Gdx.input.isKeyJustPressed(Input.Keys.F7))
75         GameStateMachine.handle(FinishLineCrossed)
76
77     private val hesItem = new HES()
78     private var mapItems = List[AItem]()
79     def generateMapItems(): Unit = {
80         mapItems = biome.generateMapItems(Geometry)
81
82         hesItem.Coordinates = Geometry.FinishPoint
83         mapItems = mapItems.appended(hesItem)
84     }
85
86     def getMapItems: List[AItem] = mapItems
87 }

```

9.2.44 – TrackTexture.scala

```

1 package ch.hevs.fastandmudry
2 package core.ecs.systems.track
3
4 import ch.hevs.fastandmudry.core.world.biome.Biome
5 import ch.hevs.fastandmudry.utils.Constant.MapTexture
6 import com.badlogic.gdx.graphics.Pixmap.Format

```

```

7  import com.badlogic.gdx.graphics.{Color, Pixmap}
8
9  object TrackTexture {
10     def generate(geometry: TrackGeometry, biome: Biome): Pixmap = {
11         val trackRectangle = geometry.trackSize
12         val width = trackRectangle.width.toInt + 2 * MapTexture.MAP_PADDING
13         val height = trackRectangle.height.toInt + 2 * MapTexture.MAP_PADDING
14         val pixmap = new Pixmap(width, height, Format.RGBA8888)
15
16         biome.prepareTextures(width, height)
17
18         val halfRoadSq = geometry.HalfRoadWidth * geometry.HalfRoadWidth
19         val halfLineSq = MapTexture.HALF_LINE_WIDTH * MapTexture.HALF_LINE_WIDTH
20
21         val finishColor = Color.rgb8888(Color.BLACK)
22         val roadLineColor = Color.rgb8888(biome.getRoadLineColor())
23
24         for (y ← 0 until height) {
25             for (x ← 0 until width) {
26                 val worldX = trackRectangle.x - MapTexture.MAP_PADDING + x
27                 val worldY = trackRectangle.y - MapTexture.MAP_PADDING + y
28                 val distSq = geometry.distToCenterLineSq(worldX, worldY)
29                 val jitteredShoulder = geometry.HalfShoulderWidth +
30 biome.getShoulderJitter(x, y)
31                 val halfShoulderSq = jitteredShoulder * jitteredShoulder
32                 val onRoad = distSq ≤ halfRoadSq
33                 val onShoulder = !onRoad && distSq ≤ halfShoulderSq
34                 val onCenterLine = distSq ≤ halfLineSq
35                 val color = {
36                     if (onRoad && geometry.isFinishLine(worldX, worldY)) finishColor
37                     else if (onCenterLine) roadLineColor
38                     else if (onRoad) biome.getRoadColor(x, y)
39                     else if (onShoulder) biome.getShoulderColor(x, y)
40                     else biome.getOffRoadColor(x, y)
41                 }
42                 pixmap.drawPixel(x, y, color)
43             }
44         }
45         return pixmap
46     }
47 }

```

9.2.45 – GarageData.scala

```

1  package ch.hevs.fastandmudry
2  package core.garage
3
4  import scala.collection.mutable.ArrayBuffer
5
6  final case class GarageItem(text: String, imagePath: String, buttonText: String,
7 price: Int)
8
9  object GarageData {
10     val items: ArrayBuffer[GarageItem] = ArrayBuffer(
11         GarageItem("Pneus percés", "data/images/item/SimpleRock.png", "Réparer", 200),
12         GarageItem("Moteur en surchauffe", "data/images/item/SimpleTree.png",
13 "Réparer", 200),
14         GarageItem("Niveau d'huile insuffisant", "data/images/item/SimpleRock.png",
15 "Réparer", 200),
16         GarageItem("Freins cassés", "data/images/item/SimpleTree.png", "Réparer", 200),
17         GarageItem("Moteur en surchauffe", "data/images/item/SimpleTree.png",
18 "Réparer", 200),
19         GarageItem("Niveau d'huile insuffisant", "data/images/item/SimpleRock.png",
20 "Réparer", 200),

```

```

16     GarageItem("Freins cassés", "data/images/item/SimpleTree.png", "Réparer", 200),
17     GarageItem("Moteur en surchauffe", "data/images/item/SimpleTree.png",
18     "Réparer", 200),
19     GarageItem("Niveau d'huile insuffisant", "data/images/item/SimpleRock.png",
20     "Réparer", 200),
21     GarageItem("Freins cassés", "data/images/item/SimpleTree.png", "Réparer", 200),
22     GarageItem("Moteur en surchauffe", "data/images/item/SimpleTree.png",
23     "Réparer", 200),
24     GarageItem("Niveau d'huile insuffisant", "data/images/item/SimpleRock.png",
25     "Réparer", 200),
26     GarageItem("Freins cassés", "data/images/item/SimpleTree.png", "Réparer", 200),
27     GarageItem("Vitres fissurées", "data/images/item/SimpleRock.png", "Réparer",
28     200)
29 )
30 }

```

9.2.46 – Question.scala

```

1 package ch.hevs.fastandmudry
2 package core.quiz
3
4 final case class Question(text: String, answers: Array[String], correctIndex: Int)

```

9.2.47 – QuizData.scala

```

1 package ch.hevs.fastandmudry
2 package core.quiz
3
4 import scala.collection.mutable.ArrayBuffer
5 import scala.util.Random
6
7 object QuizData {
8     val QUESTIONS_PER_QUIZ_SESSION: Int = 3
9
10    val questions: ArrayBuffer[Question] = ArrayBuffer(
11        Question("Quelle est la capitale de l'Australie ?",
12            Array("Sydney", "Melbourne", "Canberra", "Perth"), 2),
13        Question("En quelle année a eu lieu la chute du mur de Berlin ?",
14            Array("1987", "1989", "1991", "1985"), 1),
15        Question("Quel pays compte le plus grand nombre de fuseaux horaires ?",
16            Array("Russie", "Etats-Unis", "Chine", "France"), 3),
17        Question("Combien de cordes possède un violon ?",
18            Array("4", "5", "6", "7"), 0),
19        Question("Quel pays a remporté la Coupe du Monde de football 2018 ?",
20            Array("Croatie", "France", "Bresil", "Allemagne"), 1),
21        Question("Quel artiste a peint 'La Nuit étoilée' ?",
22            Array("Claude Monet", "Vincent van Gogh", "Paul Cezanne", "Edgar Degas"), 1),
23        Question("Quelle est la plus haute montagne du monde ?",
24            Array("K2", "Mont Blanc", "Everest", "Kilimandjaro"), 2),
25        Question("Quel est le plus grand océan du monde ?",
26            Array("Atlantique", "Indien", "Arctique", "Pacifique"), 3),
27        Question("En quelle année l'homme a-t-il marché sur la Lune pour la première
28        fois ?",
29            Array("1965", "1969", "1972", "1959"), 1),
30        Question("Quelle ville a accueilli les premiers Jeux Olympiques modernes en
31        1896 ?",
32            Array("Paris", "Londres", "Athenes", "Rome"), 2),
33        Question("Quelle est la capitale du Sri Lanka ?",
34            Array("Paris", "Sri Jayawardenepura Kotte", "Sri Jayamadenapura Kotte", "Sri
35        Jayawardenepura Kotte"), 3),
36        Question("Que signifie l'acronyme HTML ?",

```

```

35     Array("HyperText Markup Language", "High Tech Modern Language", "HyperText
Machine Language", "Home Tool Markup Language"), 0),
36     Question("Quel symbole est utilise pour les commentaires sur une ligne en
Java ?",
37         Array("#", "//", "/*—", "%"), 1),
38     Question("Quelle structure de donnees fonctionne en FIFO (First In First
Out) ?",
39         Array("Pile (Stack)", "File (Queue)", "Arbre", "Tableau"), 1),
40     Question("En binaire, que vaut le nombre decimal 5 ?",
41         Array("100", "111", "101", "110"), 2),
42     Question("Quel langage est principalement utilise pour styler des pages
web ?",
43         Array("HTML", "CSS", "SQL", "PHP"), 1),
44     Question("Quelle est la complexite temporelle d'une recherche dichotomique ?",
45         Array("O(n)", "O(n^2)", "O(log n)", "O(1)"), 2),
46     Question("Que retourne l'expression 7 % 3 dans la plupart des langages ?",
47         Array("2", "1", "3", "0"), 1),
48     Question("Quel mot-cle declare une constante en Scala ?",
49         Array("var", "let", "val", "const"), 2),
50     Question("Quel protocole securise est utilise pour le web (HTTPS) ?",
51         Array("FTP", "SSH", "TLS/SSL", "SMTP"), 2),
52     Question("En programmation orientee objet, quel concept consiste a masquer les
details internes ?",
53         Array("Heritage", "Encapsulation", "Polymorphisme", "Abstraction"), 1)
54 )
55
56 private val usedQuestionsText: ArrayBuffer[String] = ArrayBuffer.empty
57
58 def nextSession(): ArrayBuffer[Question] = {
59     val unusedQuestions = ArrayBuffer.empty[Question]
60     for (question ← questions) {
61         if (!usedQuestionsText.contains(question.text)) {
62             unusedQuestions += question
63         }
64     }
65
66     val picked = ArrayBuffer.empty[Question]
67     var count = 0
68     while (count < QUESTIONS_PER_QUIZ_SESSION) {
69         if (unusedQuestions.nonEmpty) {
70             val r = new Random()
71             val randomIndex = r.nextInt(unusedQuestions.length)
72             val question = unusedQuestions(randomIndex)
73             picked += question
74             usedQuestionsText += question.text
75             unusedQuestions.remove(randomIndex)
76             count += 1
77         }
78     }
79
80     picked
81 }
82
83 def reset(): Unit = {
84     usedQuestionsText.clear()
85 }
86 }

```

9.2.48 – Quiz.scala

```

1 package ch.hevs.fastandmudry
2 package core.quiz
3
4 import scala.collection.mutable.ArrayBuffer
5

```

```

6 sealed trait QuizPhase
7 object QuizPhase {
8   case object ShowingQuestion extends QuizPhase
9   case object ShowingAnswers extends QuizPhase
10  case object Revealing extends QuizPhase
11 }
12
13 class Quiz(questions: ArrayBuffer[Question]) {
14   private var index: Int = 0
15   private var phase: QuizPhase = QuizPhase.ShowingQuestion
16   private var _isLastAnswerCorrect: Boolean = false
17
18   def getPhase: QuizPhase = phase
19   def currentQuestion: Question = questions(index)
20   def isLastAnswerCorrect: Boolean = _isLastAnswerCorrect
21
22   def revealAnswers(): Unit = {
23     if (phase == QuizPhase.ShowingQuestion) phase = QuizPhase.ShowingAnswers
24   }
25
26   def answer(chosenIndex: Int): Boolean = {
27     _isLastAnswerCorrect = chosenIndex == currentQuestion.correctIndex
28     phase = QuizPhase.Revealing
29     isLastAnswerCorrect
30   }
31
32   def doContinueQuestions(): Boolean = {
33     if (index < questions.length - 1) {
34       index += 1
35       phase = QuizPhase.ShowingQuestion
36       return true
37     } else {
38       return false
39     }
40   }
41 }

```

9.2.49 – Day.scala

```

1 package ch.hevs.fastandmudry
2 package core.state
3
4 import ch.hevs.fastandmudry.core.world.biome.{Biome, DesertBiome, ForestBiome,
5   SnowBiome}
6 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
7
8 sealed trait Day {
9   def next: Option[Day]
10  def newBiome: Biome
11  def endDayCinematicBackground: BitmapImage
12  def startDayCinematicBackground: BitmapImage
13 }
14
15 case object Day1 extends Day {
16   val next: Option[Day] = Some(Day2)
17   def newBiome: Biome = new ForestBiome()
18   def endDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
19   cinematic/day1/bg.png")
20   def startDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
21   cinematic/day1/bg.png")
22 }
23
24 case object Day2 extends Day {
25   val next: Option[Day] = Some(Day3)
26   def newBiome: Biome = new DesertBiome()
27 }

```



```

24     def endDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
cinematic/day2/bg.jpg")
25     def startDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
cinematic/day2/bg.jpg")
26 }
27
28 case object Day3 extends Day {
29     val next: Option[Day] = None
30     def newBiome: Biome = new SnowBiome()
31     def endDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
cinematic/day3/bg.jpg")
32     def startDayCinematicBackground: BitmapImage = new BitmapImage("data/images/
cinematic/day3/bg.jpg")
33 }

```

9.2.50 – GameEvent.scala

```

1 package ch.hevs.fastandmudry
2 package core.state
3
4 import ch.hevs.fastandmudry.core.car.CarSkin
5
6 sealed trait GameEvent
7
8 case object StartGame extends GameEvent
9 case object MapLoaded extends GameEvent
10 case object FinishLineCrossed extends GameEvent
11 case object EndDayCinematicEnded extends GameEvent
12 case object QuizCompleted extends GameEvent
13 case object GarageReady extends GameEvent
14 case object StartDayCinematicEnded extends GameEvent
15 case object FinalCinematicEnded extends GameEvent
16 case object CarBroke extends GameEvent
17 case object BackToMenu extends GameEvent
18 case object OpenCarSelector extends GameEvent
19 case object OpenCarDebug extends GameEvent
20 final case class CarSkinSelected(skin: CarSkin) extends GameEvent

```

9.2.51 – GameStateMachine.scala

```

1 package ch.hevs.fastandmudry
2 package core.state
3
4 import ch.hevs.fastandmudry.screens.CustomScreenManager
5
6 object GameStateMachine {
7     private var currentState: GameState = GameState.Menu
8
9     def getGameState: GameState = currentState
10
11     def handle(event: GameEvent): Unit = {
12         val nextState = currentState.next(event)
13         if (nextState != currentState) {
14             println(s"From $currentState to $nextState by $event")
15             currentState = nextState
16             onHandleNextState(nextState)
17         } else {
18             println(s"Stayed in $currentState ignoring $event")
19         }
20     }
21
22     private def onHandleNextState(state: GameState): Unit = {
23         val csm = CustomScreenManager.getInstance
24         state match {

```



```

25     case GameState.Menu => csm.activateScreen(CustomScreenManager.MENU)
26     case GameState.Loading(_) => csm.activateScreen(CustomScreenManager.LOADING)
27     case GameState.Playing(_) => csm.activateScreen(CustomScreenManager.GAME)
28     case GameState.StartDayCinematic(_) =>
csm.activateScreen(CustomScreenManager.CINEMATIC)
29     case GameState.EndDayCinematic(_) =>
csm.activateScreen(CustomScreenManager.CINEMATIC)
30     case GameState.Quiz(_) => csm.activateScreen(CustomScreenManager.QUIZ)
31     case GameState.Garage(_) => csm.activateScreen(CustomScreenManager.GARAGE)
32     case GameState.FinalCinematic =>
csm.activateScreen(CustomScreenManager.CINEMATIC)
33     case GameState.Dead => csm.activateScreen(CustomScreenManager.DEATH)
34     case GameState.CarSelector =>
csm.activateScreen(CustomScreenManager.CAR_SELECTOR)
35     case GameState.CarDebug => csm.activateScreen(CustomScreenManager.CAR_DEBUG)
36   }
37 }
38 }

```

9.2.52 – GameState.scala

```

1  package ch.hevs.fastandmudry
2  package core.state
3
4  // State machine structure taken from : https://www.scala-algorithms.com/
5  // StateMachine/
6  sealed trait GameState {
7    def next(event: GameEvent): GameState
8  }
9
10 object GameState {
11   case object Menu extends GameState {
12     def next(event: GameEvent): GameState = {
13       event match {
14         case StartGame => Loading(Day1)
15         case OpenCarSelector => CarSelector
16         case OpenCarDebug => CarDebug
17         // by default, does nothing, stay in Menu
18         case _ => this
19       }
20     }
21   }
22
23   case object Dead extends GameState {
24     def next(event: GameEvent): GameState = {
25       event match {
26         case BackToMenu => Menu
27         case _ => this
28       }
29     }
30   }
31
32   final case class Loading(day: Day) extends GameState {
33     def next(event: GameEvent): GameState = {
34       event match {
35         case MapLoaded => Playing(day)
36         case _ => this
37       }
38     }
39   }
40
41   final case class Playing(day: Day) extends GameState {
42     def next(event: GameEvent): GameState = {
43       event match {
44         case FinishLineCrossed if day.next.isEmpty => FinalCinematic

```

```

44     case FinishLineCrossed ⇒ EndDayCinematic(day)
45     case CarBroke ⇒ Dead
46     case _ ⇒ this
47   }
48 }
49 }
50
51 case object FinalCinematic extends GameState {
52   def next(event: GameEvent): GameState = {
53     event match {
54       case FinalCinematicEnded ⇒ Menu
55       case _ ⇒ this
56     }
57   }
58 }
59
60 final case class EndDayCinematic(day: Day) extends GameState {
61   def next(event: GameEvent): GameState = {
62     event match {
63       case EndDayCinematicEnded ⇒ Quiz(day)
64       case _ ⇒ this
65     }
66   }
67 }
68
69 final case class StartDayCinematic(day: Day) extends GameState {
70   def next(event: GameEvent): GameState = {
71     event match {
72       case StartDayCinematicEnded ⇒ Loading(day)
73       case _ ⇒ this
74     }
75   }
76 }
77
78 final case class Quiz(day: Day) extends GameState {
79   def next(event: GameEvent): GameState = {
80     event match {
81       case QuizCompleted ⇒ Garage(day)
82       case _ ⇒ this
83     }
84   }
85 }
86
87 final case class Garage(day: Day) extends GameState {
88   def next(event: GameEvent): GameState = {
89     event match {
90       case GarageReady ⇒ StartDayCinematic(day.next.get)
91       case _ ⇒ this
92     }
93   }
94 }
95
96 case object CarSelector extends GameState {
97   def next(event: GameEvent): GameState = {
98     event match {
99       case BackToMenu ⇒ Menu
100      case CarSkinSelected(_) ⇒ Menu
101      case _ ⇒ this
102    }
103  }
104 }
105
106 case object CarDebug extends GameState {
107   def next(event: GameEvent): GameState = {
108     event match {
109       case BackToMenu ⇒ Menu
110       case _ ⇒ this

```

```

111     }
112   }
113 }
114 }

```

9.2.53 – Wallet.scala

```

1  package ch.hevs.fastandmudry
2  package core.state
3
4  object Wallet {
5    private var _coins: Int = 0
6
7    def coins: Int = _coins
8
9    def add(coins: Int): Unit = {
10      if (coins > 0) _coins += coins
11    }
12
13    def spend(amount: Int): Boolean = {
14      if (amount > 0 && amount ≤ _coins) {
15        _coins -= amount
16        true
17      } else {
18        false
19      }
20    }
21
22    def reset(): Unit = {
23      _coins = 0
24    }
25  }

```

9.2.54 – Biome.scala

```

1  package ch.hevs.fastandmudry
2  package core.world.biome
3
4  import ch.hevs.fastandmudry.core.audio.MusicTrack
5  import ch.hevs.fastandmudry.core.ecs.entities.Item.AItem
6  import ch.hevs.fastandmudry.core.ecs.systems.Car
7  import ch.hevs.fastandmudry.core.ecs.systems.track.TrackGeometry
8  import ch.hevs.fastandmudry.utils.Constant.MapTexture
9  import ch.hevs.gdx2d.components.bitmaps.BitmapImage
10 import com.badlogic.gdx.graphics.Color
11
12 trait Biome {
13   private var offRoadNoiseField: Array[Array[Int]] = _
14   private var roadNoiseField: Array[Array[Int]] = _
15   private var shoulderNoiseField: Array[Array[Int]] = _
16   private var shoulderJitterField: Array[Array[Float]] = _
17
18   def musicTrack: MusicTrack
19
20   // The factor decreasing the car speed when the car is off the road
21   def offRoadDecreasingFactorSpeed: Float
22   def getRoadLineColor(): Color
23
24   def offRoadLowColor(): Color
25   def offRoadHighColor(): Color
26   def offRoadNoiseCell(): Int = 24
27
28   def roadLowColor(): Color
29   def roadHighColor(): Color
30   def roadNoiseCell(): Int = 4

```

```

31
32 def shoulderLowColor(): Color
33 def shoulderHighColor(): Color
34 def shoulderNoiseCell(): Int = 16
35
36 def getOffRoadColor(x: Int, y: Int): Int = offRoadNoiseField(x)(y)
37 def getRoadColor(x: Int, y: Int): Int = roadNoiseField(x)(y)
38 def getShoulderColor(x: Int, y: Int): Int = shoulderNoiseField(x)(y)
39
40 def getShoulderJitter(x: Int, y: Int): Float =
41     (shoulderJitterField(x)(y) * 2f - 1f) * MapTexture.SHOULDER_JITTER
42
43 def prepareTextures(width: Int, height: Int): Unit = {
44     offRoadNoiseField = BiomeTexture.generateNoiseField(
45         width, height, offRoadNoiseCell(), offRoadLowColor(), offRoadHighColor()
46     )
47     roadNoiseField = BiomeTexture.generateNoiseField(
48         width, height, roadNoiseCell(), roadLowColor(), roadHighColor()
49     )
50     shoulderNoiseField = BiomeTexture.generateNoiseField(
51         width, height, shoulderNoiseCell(), shoulderLowColor(), shoulderHighColor()
52     )
53     shoulderJitterField = BiomeTexture.generateScalarField(
54         width, height, MapTexture.SHOULDER_JITTER_NOISE_CELL
55     )
56 }
57
58 def parallaxLayers(): Array[ParallaxLayer]
59 def skyImage(): BitmapImage
60
61 // Update the car physics (tire explosion, motor overheat, ...)
62 def updatePhysics(car: Car, isOffRoad: Boolean, elapsedTime: Float): Unit
63
64 def generateMapItems(geometry: TrackGeometry): List[AItem] = {
65     List.empty
66 }
67 }

```

9.2.55 – BiomeTexture.scala

```

1 package ch.hevs.fastandmudry
2 package core.world.biome
3
4 import ch.hevs.fastandmudry.utils.noise.{Generator, Noise}
5 import com.badlogic.gdx.graphics.Color
6
7 object BiomeTexture {
8
9     // Most of the code come from this git repo: https://github.com/yatsukha/perlin-
10     noise
11     def generateNoiseField(width: Int, height: Int, noiseCell: Int, low: Color, high:
12     Color): Array[Array[Int]] = {
13         val r: Int = width
14         val c: Int = height
15         val t = noiseCell
16         val gf = Generator.gradientField(
17             new scala.util.Random(System.currentTimeMillis),
18             // we want the sand to be not too noisy, so we have to divide by NOISE_CELL
19             so it is smoother
20             (r / t + 2, c / t + 2)
21         )
22         val perlin =
23             (0 until r)
24                 .map(_._toDouble / t.toDouble)
25                 .map(

```

```

24     x =>
25       (0 until c)
26         .map(_.toDouble / t.toDouble)
27         .map(y => Noise.noise(gf, (x, y)))
28         // expand the average [0.3, 0.7] signal energy to full range
29         // .map(util.exaggerate(_))
30         .map(_ + 1) // move to [0, 2]
31         .map(_ / 2) // scale to [0, 1]
32     )
33
34   val perlinFlat = flatten(perlin, r, c)
35
36   val field = Array.ofDim[Int](width, height)
37
38   for (x <- 0 until width) {
39     for (y <- 0 until height) {
40       val noiseVal = Noise.turbulence(perlinFlat, r, c, 8, x, y)
41
42       val red = low.r + (high.r - low.r) * noiseVal
43       val g = low.g + (high.g - low.g) * noiseVal
44       val b = low.b + (high.b - low.b) * noiseVal
45
46       val finalR = math.min(1.0, math.max(0.0, red)).toFloat
47       val finalG = math.min(1.0, math.max(0.0, g)).toFloat
48       val finalB = math.min(1.0, math.max(0.0, b)).toFloat
49
50       field(x)(y) = Color.rgb8888(finalR, finalG, finalB, 1f)
51     }
52   }
53   field
54 }
55
56 def generateScalarField(width: Int, height: Int, noiseCell: Int):
57   Array[Array[Float]] = {
58     val r = width
59     val c = height
60     val t = noiseCell
61     val gf = Generator.gradientField(
62       new scala.util.Random(System.currentTimeMillis),
63       (r / t + 2, c / t + 2)
64     )
65
66     val perlin =
67       (0 until r)
68         .map(_.toDouble / t.toDouble)
69         .map(
70           x =>
71             (0 until c)
72               .map(_.toDouble / t.toDouble)
73               .map(y => Noise.noise(gf, (x, y)))
74               .map(_ + 1)
75               .map(_ / 2)
76         )
77
78     val perlinFlat = flatten(perlin, r, c)
79
80     val field = Array.ofDim[Float](width, height)
81     for (x <- 0 until width) {
82       for (y <- 0 until height) {
83         val n = Noise.turbulence(perlinFlat, r, c, 8, x, y)
84         field(x)(y) = math.min(1.0, math.max(0.0, n)).toFloat
85       }
86     }
87     field
88   }

```

```

89 // flatten the IndexedSeq[IndexedSeq[Double]] to a 1D array → improve
performances
90 private def flatten(perlin: IndexedSeq[IndexedSeq[Double]], rows: Int, cols:
Int): Array[Double] = {
91     val flat = new Array[Double](rows * cols)
92     for (i ← 0 until rows) {
93         for (j ← 0 until cols) {
94             flat(i * cols + j) = perlin(i)(j)
95         }
96     }
97     return flat
98 }
99 }

```

9.2.56 – DesertBiome.scala

```

1 package ch.hevs.fastandmudry
2 package core.world.biome
3
4 import ch.hevs.fastandmudry.core.audio.MusicTrack
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.AItem
6 import ch.hevs.fastandmudry.core.ecs.entities.Item.desert.{Cactus, DeserticRock,
DeserticSkeleton}
7 import ch.hevs.fastandmudry.core.ecs.systems.Car
8 import ch.hevs.fastandmudry.core.ecs.systems.track.TrackGeometry
9 import ch.hevs.fastandmudry.utils.Constant.MapTexture
10 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
11 import com.badlogic.gdx.graphics.Color
12
13 import scala.util.Random
14
15 class DesertBiome extends Biome {
16     override def musicTrack: MusicTrack = MusicTrack.Desert
17
18     private val sky: BitmapImage = new BitmapImage("data/parallax/desert/
desert_sky.png")
19     private val layers: Array[ParallaxLayer] = Array(
20         new ParallaxLayer(new BitmapImage("data/parallax/desert/desert_mountain.png"),
-400f, 0.30f, false),
21         new ParallaxLayer(new BitmapImage("data/parallax/desert/desert_moon.png"),
-800f, 0.20f, false),
22         new ParallaxLayer(new BitmapImage("data/parallax/desert/desert_dunemid.png"),
-1500f, 0.3f, false),
23         new ParallaxLayer(new BitmapImage("data/parallax/desert/desert_dunefront.png"),
-2500f, 0.15f, true)
24     )
25
26     override def offRoadDecreasingFactorSpeed: Float = 0.2f
27
28     override def getRoadLineColor(): Color = Color.YELLOW
29
30     override def offRoadLowColor(): Color = new Color(180f / 255f, 100f / 255f, 50f /
255f, 1f)
31     override def offRoadHighColor(): Color = new Color(245f / 255f, 220f / 255f,
150f / 255f, 1f)
32
33     override def roadLowColor(): Color = new Color(95f / 255f, 85f / 255f, 70f /
255f, 1f)
34     override def roadHighColor(): Color = new Color(140f / 255f, 125f / 255f, 100f /
255f, 1f)
35
36     override def shoulderLowColor(): Color = new Color(90f / 255f, 80f / 255f, 50f /
255f, 1f)

```

```

37     override def shoulderHighColor(): Color = new Color(190f / 255f, 175f / 255f,
38     150f / 255f, 1f)
39
40     override def parallaxLayers(): Array[ParallaxLayer] = layers
41
42     override def skyImage(): BitmapImage = sky
43
44     override def updatePhysics(car: Car, isOffRoad: Boolean, elapsedTime: Float):
Unit = {
45         if(isOffRoad) {
46             car.TemperatureProblem.updateBroken(elapsedTime, 4f)
47         } else {
48             }
49     }
50
51     override def generateMapItems(geometry: TrackGeometry): List[AItem] = {
52         val random = new Random()
53         val trackWidth = geometry.trackSize.getWidth + MapTexture.MAP_PADDING * 2
54         val trackHeight = geometry.trackSize.getHeight + MapTexture.MAP_PADDING * 2
55
56         val originX = geometry.trackSize.x - MapTexture.MAP_PADDING
57         val originY = geometry.trackSize.y - MapTexture.MAP_PADDING
58
59         val res = List.fill(1000) {
60             val item = new Cactus()
61
62             do {
63                 item.Coordinates.x = random.nextFloat() * trackWidth + originX
64                 item.Coordinates.y = random.nextFloat() * trackHeight + originY
65             }while(item.isGenerationAllowed(geometry))
66             item
67         }
68
69         val res2 = List.fill(500) {
70             val item = new DeserticRock()
71
72             do {
73                 item.Coordinates.x = random.nextFloat() * trackWidth + originX
74                 item.Coordinates.y = random.nextFloat() * trackHeight + originY
75             }while(item.isGenerationAllowed(geometry))
76             item
77         }
78
79         val res3 = List.fill(250) {
80             val item = new DeserticSkeleton()
81
82             do {
83                 item.Coordinates.x = random.nextFloat() * trackWidth + originX
84                 item.Coordinates.y = random.nextFloat() * trackHeight + originY
85             }while(item.isGenerationAllowed(geometry))
86             item
87         }
88
89         res.appendedAll(res2).appendedAll(res3)
90     }
91 }

```

9.2.57 – ForestBiome.scala

```

1 package ch.hevs.fastandmudry
2 package core.world.biome
3
4 import ch.hevs.fastandmudry.core.audio.MusicTrack
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.AItem

```

```

6 import ch.hevs.fastandmudry.core.ecs.entities.Item.forest.{SimpleBush, SimpleRock,
SimpleTree}
7 import ch.hevs.fastandmudry.core.ecs.systems.Car
8 import ch.hevs.fastandmudry.core.ecs.systems.track.TrackGeometry
9 import ch.hevs.fastandmudry.utils.Constant.MapTexture
10 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
11 import com.badlogic.gdx.graphics.Color
12
13 import scala.util.Random
14
15 class ForestBiome extends Biome {
16     override def musicTrack: MusicTrack = MusicTrack.Forest
17
18     private val sky: BitmapImage = new BitmapImage("data/parallax/forest/
forest_sky.png")
19     private val layers: Array[ParallaxLayer] = Array(
20         new ParallaxLayer(new BitmapImage("data/parallax/forest/forest_mountain.png"),
-400f, 0.30f, false),
21         new ParallaxLayer(new BitmapImage("data/parallax/forest/forest_back.png"),
-800f, 0.20f, false),
22         new ParallaxLayer(new BitmapImage("data/parallax/forest/forest_short.png"),
-2500f, 0.15f, true)
23     )
24
25     override def offRoadDecreasingFactorSpeed: Float = 0.5f
26     override def updatePhysics(car: Car, isOffRoad: Boolean, elapsedTime: Float):
Unit = {
27         if(isOffRoad) {
28             car.FrontLeftTire.updateBroken(0.005f)
29             car.BackLeftTire.updateBroken(0.005f)
30             car.FrontRightTire.updateBroken(0.005f)
31             car.BackRightTire.updateBroken(0.005f)
32         }
33     }
34
35     override def getRoadLineColor(): Color = Color.WHITE
36
37     override def offRoadLowColor(): Color = new Color(22f / 255f, 85f / 255f, 30f /
255f, 1f)
38     override def offRoadHighColor(): Color = new Color(50f / 255f, 170f / 255f, 70f /
255f, 1f)
39
40     override def roadLowColor(): Color = new Color(70f / 255f, 70f / 255f, 70f /
255f, 1f)
41     override def roadHighColor(): Color = new Color(110f / 255f, 110f / 255f, 110f /
255f, 1f)
42
43     override def shoulderLowColor(): Color = new Color(40f / 255f, 150f / 255f, 55f /
255f, 1f)
44     override def shoulderHighColor(): Color = new Color(50f / 255f, 40f / 255f, 10f /
255f, 1f)
45
46     override def skyImage(): BitmapImage = sky
47
48     override def parallaxLayers(): Array[ParallaxLayer] = layers
49
50     override def generateMapItems(geometry: TrackGeometry): List[AItem] = {
51         val random = new Random()
52         val trackWidth = geometry.trackSize.getWidth + MapTexture.MAP_PADDING * 2
53         val trackHeight = geometry.trackSize.getHeight + MapTexture.MAP_PADDING * 2
54
55         val originX = geometry.trackSize.x - MapTexture.MAP_PADDING
56         val originY = geometry.trackSize.y - MapTexture.MAP_PADDING
57
58         val res = List.fill(2500) {
59             val item = new SimpleTree()

```



```

60
61     do {
62         item.Coordinates.x = random.nextFloat() * trackWidth + originX
63         item.Coordinates.y = random.nextFloat() * trackHeight + originY
64     } while (item.isGenerationAllowed(geometry))
65     item
66 }
67
68 val res2 = List.fill(200) {
69     val item = new SimpleRock()
70     do {
71         item.Coordinates.x = random.nextFloat() * trackWidth + originX
72         item.Coordinates.y = random.nextFloat() * trackHeight + originY
73     } while (item.isGenerationAllowed(geometry))
74     item
75 }
76
77 val res3 = List.fill(300) {
78     val item = new SimpleBush()
79     do {
80         item.Coordinates.x = random.nextFloat() * trackWidth + originX
81         item.Coordinates.y = random.nextFloat() * trackHeight + originY
82     } while (item.isGenerationAllowed(geometry))
83     item
84 }
85
86 res.appendedAll(res2).appendedAll(res3)
87 }
88 }

```

9.2.58 – ParallaxLayer.scala

```

1 package ch.hevs.fastandmudry
2 package core.world.biome
3
4 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
5
6 class ParallaxLayer(val image: BitmapImage, val scrollOffset: Float, val
7 heightPercentage: Float, val isFloor: Boolean) {
8 }

```

9.2.59 – SnowBiome.scala

```

1 package ch.hevs.fastandmudry
2 package core.world.biome
3
4 import ch.hevs.fastandmudry.core.audio.MusicTrack
5 import ch.hevs.fastandmudry.core.ecs.entities.Item.AItem
6 import ch.hevs.fastandmudry.core.ecs.entities.Item.snow.{PinTree, SnowPile,
7 SnowyRock}
8 import ch.hevs.fastandmudry.core.ecs.systems.Car
9 import ch.hevs.fastandmudry.core.ecs.systems.track.TrackGeometry
10 import ch.hevs.fastandmudry.utils.Constant.MapTexture
11 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
12 import com.badlogic.gdx.graphics.Color
13
14 import scala.util.Random
15
16 class SnowBiome extends Biome {
17     override def musicTrack: MusicTrack = MusicTrack.Snow
18
19     private val sky: BitmapImage = new BitmapImage("data/parallax/skies/sky_sky.png")
20     private val layers: Array[ParallaxLayer] = Array(

```

```

20     new ParallaxLayer(new BitmapImage("data/parallax/skies/sky_back_mountain.png"),
-400f, 0.30f, false),
21     new ParallaxLayer(new BitmapImage("data/parallax/skies/sky_cloud_floor.png"),
-800f, 0.20f, false),
22     new ParallaxLayer(new BitmapImage("data/parallax/skies/
sky_front_mountain.png"), -1500f, 0.3f, false),
23     new ParallaxLayer(new BitmapImage("data/parallax/skies/sky_front_cloud.png"),
-2500f, 0.15f, true)
24 )
25
26 override def offRoadDecreasingFactorSpeed: Float = 0.7f
27
28 override def getRoadLineColor(): Color = Color.YELLOW
29
30 override def offRoadLowColor(): Color = new Color(180f / 255f, 205f / 255f,
240f / 255f, 1f)
31 override def offRoadHighColor(): Color = new Color(255f / 255f, 255f / 255f,
255f / 255f, 1f)
32
33 override def roadLowColor(): Color = new Color(90f / 255f, 95f / 255f, 110f /
255f, 1f)
34 override def roadHighColor(): Color = new Color(200f / 255f, 210f / 255f, 230f /
255f, 1f)
35
36 override def shoulderLowColor(): Color = new Color(115f / 255f, 130f / 255f,
150f / 255f, 1f)
37 override def shoulderHighColor(): Color = new Color(200f / 255f, 220f / 255f,
230f / 255f, 1f)
38
39 override def parallaxLayers(): Array[ParallaxLayer] = layers
40
41 override def skyImage(): BitmapImage = sky
42
43 override def updatePhysics(car: Car, isOffRoad: Boolean, elapsedTime: Float):
Unit = {
44     if(isOffRoad) car.TemperatureProblem.updateBroken(elapsedTime, -5f)
45     car.TireSlippage.IsDefected = isOffRoad
46 }
47
48 override def generateMapItems(geometry: TrackGeometry): List[AItem] = {
49     val random = new Random()
50     val trackWidth = geometry.trackSize.getWidth + MapTexture.MAP_PADDING * 2
51     val trackHeight = geometry.trackSize.getHeight + MapTexture.MAP_PADDING * 2
52
53     val originX = geometry.trackSize.x - MapTexture.MAP_PADDING
54     val originY = geometry.trackSize.y - MapTexture.MAP_PADDING
55
56     val res = List.fill(2500) {
57         val item = new PinTree()
58
59         do {
60             item.Coordinates.x = random.nextFloat() * trackWidth + originX
61             item.Coordinates.y = random.nextFloat() * trackHeight + originY
62         }while(item.isGenerationAllowed(geometry))
63         item
64     }
65
66     val res2 = List.fill(250) {
67         val item = new SnowyRock()
68
69         do {
70             item.Coordinates.x = random.nextFloat() * trackWidth + originX
71             item.Coordinates.y = random.nextFloat() * trackHeight + originY
72         }while(item.isGenerationAllowed(geometry))
73         item
74     }
75

```

```

76     val res3 = List.fill(250) {
77         val item = new SnowPile()
78
79         do {
80             item.Coordinates.x = random.nextFloat() * trackWidth + originX
81             item.Coordinates.y = random.nextFloat() * trackHeight + originY
82         } while (item.isGenerationAllowed(geometry))
83         item
84     }
85     res.appendedAll(res2).appendAll(res3)
86 }
87 }

```

9.2.60 – World.scala

```

1  package ch.hevs.fastandmudry
2  package core.world
3
4  import ch.hevs.fastandmudry.core.audio.AudioManager
5  import ch.hevs.fastandmudry.core.car.{CarSkin, CarSkins}
6  import ch.hevs.fastandmudry.core.ecs.components.AGameLoop
7  import ch.hevs.fastandmudry.core.ecs.systems.Car
8  import ch.hevs.fastandmudry.core.ecs.systems.track.Track
9  import ch.hevs.fastandmudry.utils.Constant.{AUDIO, GAME}
10
11 class World private () extends AGameLoop {
12     val CAR: Car = new Car
13     val TRACK: Track = new Track(CAR)
14
15     var selectedSkin: CarSkin = CarSkins.default
16
17     private var previousSpeed: Float = 0f
18     private var backfireArmed: Boolean = false
19
20     override def onGameLoop(elapsedTime: Float): Unit = {
21         CAR.onGameLoop(elapsedTime)
22         CAR.Moving(elapsedTime, GAME.CAR.FACTOR.SPEED)
23
24         AudioManager.updateEngine(CAR.Speed, GAME.CAR.FACTOR.MAX_SPEED)
25         checkBackfire()
26         AudioManager.updateBackfire(elapsedTime)
27
28         TRACK.onGameLoop(elapsedTime)
29     }
30
31     private def checkBackfire(): Unit = {
32         val speed = Math.abs(CAR.Speed)
33         val previous = Math.abs(previousSpeed)
34         val speedLimitForBackfire = GAME.CAR.FACTOR.MAX_SPEED *
35         AUDIO.SFX.BACKFIRE_SPEED_RATIO
36
37         // have to load the backfire → needs to increase speed
38         if (speed > previous && speed ≥ speedLimitForBackfire) backfireArmed = true
39
40         // backfire armed and speed lower than previous speed and bigger than the limit
41         for pops → triggers bakfire
42         if (backfireArmed && speed < previous && previous ≥ speedLimitForBackfire) {
43             AudioManager.triggerBackfire()
44             backfireArmed = false
45         }
46
47         previousSpeed = CAR.Speed
48     }
49 }
50
51 object World {

```

```

50 private lazy val _Instance: World = new World
51
52 def INSTANCE: World = _Instance
53 }

```

9.2.61 – DebugInput.scala

```

1 package ch.hevs.fastandmudry
2 package input
3
4 import ch.hevs.fastandmudry.utils.Constant
5 import com.badlogic.gdx.Gdx
6
7 object DebugInput {
8     private var isSetupKeyPressing = false
9     private var _isDebugEnabled = false
10    def IsDebugEnabled: Boolean = _isDebugEnabled
11
12    def debuggingKeyProcess(): Unit = {
13        if (Gdx.input.isKeyPressed(Constant.KEYS.ENABLING_DEBUG_KEY)) {
14            if (!isSetupKeyPressing) {
15                isSetupKeyPressing = true
16                _isDebugEnabled = !_isDebugEnabled
17            }
18            else isSetupKeyPressing = false
19        }
20    }
21 }

```

9.2.62 – Launcher.scala

```

1 package ch.hevs.fastandmudry
2
3 object Launcher extends App {
4     new MainWindow
5 }

```

9.2.63 – MainWindow.scala

```

1 package ch.hevs.fastandmudry
2
3 import ch.hevs.fastandmudry.core.audio.AudioManager
4 import ch.hevs.fastandmudry.screens.CustomScreenManager
5 import ch.hevs.fastandmudry.utils.Constant.Window
6 import ch.hevs.gdx2d.desktop.PortableApplication
7 import ch.hevs.gdx2d.lib.GdxGraphics
8
9 class MainWindow extends PortableApplication(Window.WIDTH, Window.HEIGHT) {
10    override def onInit(): Unit = {
11        setTitle("Fast & Mudry")
12        CustomScreenManager.getInstance.activateScreen(CustomScreenManager.MENU)
13    }
14
15    override def onGraphicRender(g: GdxGraphics): Unit = {
16        CustomScreenManager.getInstance.render(g);
17    }
18
19    override def onClick(x: Int, y: Int, button: Int): Unit = {
20        super.onClick(x, y, button)
21        CustomScreenManager.getInstance.getActiveScreen.onClick(x,y,button)
22    }
23
24    override def onDispose(): Unit = {
25        super.onDispose()
26        AudioManager.dispose()
27    }
28 }

```

```

27 }
28 }

```

9.2.64 – AbstractRenderer.scala

```

1 package ch.hevs.fastandmudry
2 package render
3
4 import ch.hevs.gdx2d.lib.GdxGraphics
5
6 trait AbstractRenderer {
7     def onGraphicRender(g: GdxGraphics): Unit
8     def onInit(): Unit = {}
9 }

```

9.2.65 – BackgroundRenderer.scala

```

1 package ch.hevs.fastandmudry
2 package render.background
3
4 import ch.hevs.fastandmudry.core.world.World
5 import ch.hevs.fastandmudry.render.AbstractRenderer
6 import ch.hevs.fastandmudry.render.Data.Game
7 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
8 import ch.hevs.gdx2d.lib.GdxGraphics
9
10 class BackgroundRenderer extends AbstractRenderer {
11     override def onGraphicRender(g: GdxGraphics): Unit = {
12         val screenW = g.getScreenWidth
13         val screenH = g.getScreenHeight
14         val horizon = screenH / 2f
15         val biome = World.INSTANCE.TRACK.biome
16
17         g.drawTransformedPicture(screenW / 2f, horizon + (screenH / 4f), 0, screenW,
18             screenH / 2f, biome.skyImage())
19
20         for(layer ← biome.parallaxLayers()) {
21             val y = if(layer.isFloor) g.getScreenHeight - 100 else horizon
22             drawConstrainedLayer(g, layer.image, Game.Rotation * layer.scrollOffset, y,
23                 layer.heightPercentage)
24         }
25
26         private def drawConstrainedLayer(g: GdxGraphics, image: BitmapImage,
27             scrollOffset: Float, yBase: Float, heightPercentage: Float): Unit = {
28             val texture = image.getImage
29             val imgW = texture.getWidth.toFloat
30             val imgH = texture.getHeight.toFloat
31
32             val drawH = g.getScreenHeight * heightPercentage
33             val drawW = imgW * (drawH / imgH)
34
35             val centerY = yBase + (drawH / 2f)
36
37             val offset = scrollOffset % drawW
38             var currentX = offset - drawW
39
40             while (currentX < g.getScreenWidth + drawW) {
41                 g.drawTransformedPicture(currentX, centerY, 0, drawW, drawH, image)
42                 currentX += drawW
43             }
44         }
45     }
46 }

```

9.2.66 – Data.scala

```

1 package ch.hevs.fastandmudry
2 package render
3
4 import ch.hevs.fastandmudry.core.world.World
5
6 object Data {
7   object Game {
8     def Rotation: Float = World.INSTANCE.CAR.Rotation
9   }
10 }

```

9.2.67 – GameRenderer.scala

```

1 package ch.hevs.fastandmudry
2 package render.game
3
4 import ch.hevs.fastandmudry.render.AbstractRenderer
5 import ch.hevs.gdx2d.lib.GdxGraphics
6
7 object GameRenderer extends AbstractRenderer {
8   override def onInit(): Unit = {
9     super.onInit()
10    TrackRenderer.onInit()
11  }
12
13  override def onGraphicRender(g: GdxGraphics): Unit = {
14    TrackRenderer.onGraphicRender(g)
15  }
16 }

```

9.2.68 – ItemsRenderer.scala

```

1 package ch.hevs.fastandmudry
2 package render.game
3
4 import ch.hevs.fastandmudry.core.ecs.entities.Item.AItem
5 import ch.hevs.fastandmudry.core.world.World
6 import ch.hevs.fastandmudry.render.AbstractRenderer
7 import ch.hevs.gdx2d.lib.GdxGraphics
8 import com.badlogic.gdx.math.Vector2
9
10 object ItemsRenderer extends AbstractRenderer {
11   case class ProjectionResult(screenX: Float, screenY: Float, scale: Float,
12     distance: Float)
13   private case class RenderItem(item: AItem, proj: ProjectionResult)
14   def projectToScreen(position: Vector2, screenW: Float, screenH: Float):
15     Option[ProjectionResult] = {
16       val cameraPosition = TrackRenderer.cameraPosition
17       val alpha = TrackRenderer.cameraAngle
18       val pitch = TrackRenderer.pitch
19       val fov = TrackRenderer.cameraFov
20
21       val screenPlanDistance = (0.5f / Math.atan(fov / 2.0)).toFloat
22
23       val differentialVector = new Vector2(position.x, position.y)
24       differentialVector.sub(cameraPosition.x + 0.5f, cameraPosition.y)
25
26       val cosAlpha = Math.cos(alpha).toFloat
27       val sinAlpha = Math.sin(alpha).toFloat
28
29       val primeVector = new Vector2(
30         differentialVector.x * cosAlpha - differentialVector.y * sinAlpha,
31         differentialVector.x * sinAlpha + differentialVector.y * cosAlpha
32       )
33     }
34 }

```

```

30     )
31
32     // Si l'objet est derrière
33     if (primeVector.y ≤ 0.01f) return None
34
35     val screenX = screenW * (0.5f + (screenPlanDistance * primeVector.x) /
primeVector.y)
36     val screenY = screenH * (0.5f + pitch - (screenPlanDistance *
cameraPosition.z) / primeVector.y)
37
38     val scale = screenPlanDistance / primeVector.y
39
40     Some(ProjectionResult(screenX, screenY, scale, primeVector.y))
41 }
42
43 override def onGraphicRender(g: GdxGraphics): Unit = {
44     World.INSTANCE.TRACK.getMapItems.flatMap((i) => {
45         val coord = i.Coordinates.cpy()
46         projectToScreen(coord, g.getScreenWidth.toFloat,
g.getScreenHeight.toFloat).map(p => RenderItem(i, p))
47     }).sortBy(_.proj.distance)(Ordering[Float].reverse).foreach(r => {
48         val (mw, mh) = r.item.getMaxSize
49         val rw = mw * r.proj.scale
50         val rh = mh * r.proj.scale
51         val rx = r.proj.screenX - rw/2f
52         val ry = r.proj.screenY
53         g.draw(r.item.getTexture, rx , ry, rw, rh)
54     })
55 }
56 }

```

9.2.69 – TrackRenderer.scala

```

1  package ch.hevs.fastandmudry
2  package render.game
3
4  import ch.hevs.fastandmudry.core.world.World
5  import ch.hevs.fastandmudry.render.AbstractRenderer
6  import ch.hevs.fastandmudry.render.shaders.Mode7
7  import ch.hevs.fastandmudry.ui.hud.DebugHUD
8  import ch.hevs.fastandmudry.utils.Common
9  import ch.hevs.fastandmudry.utils.Constant.MapTexture
10 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
11 import ch.hevs.gdx2d.lib.GdxGraphics
12 import com.badlogic.gdx.graphics.glutils.FrameBuffer
13 import com.badlogic.gdx.math.Vector2
14 import com.badlogic.gdx.{Gdx, Input}
15
16
17 object TrackRenderer extends AbstractRenderer {
18     var imageBackground : BitmapImage = null
19     var fbo: FrameBuffer = null
20
21     var cameraFov = Mode7.DEFAULT_VALUES.CAMERA.FOV
22     var cameraAngle = Mode7.DEFAULT_VALUES.CAMERA.ANGLE
23     val cameraPosition = Mode7.DEFAULT_VALUES.CAMERA.POSITION
24     val cameraAxis = Mode7.DEFAULT_VALUES.CAMERA.AXIS
25     var pitch = Mode7.DEFAULT_VALUES.PITCH
26     var renderingFactor = Mode7.DEFAULT_VALUES.RENDERING_FACTOR
27
28     override def onInit(): Unit = {
29         val texture = World.INSTANCE.TRACK.Texture
30         renderingFactor.set(1f / texture.getWidth, 1f / texture.getHeight)
31     }
32
33     override def onGraphicRender(g: GdxGraphics): Unit = {

```

```

34 graphicalSetup()
35
36 cameraPosition.y = World.INSTANCE.CAR.Coordinates.y
37 cameraPosition.x = World.INSTANCE.CAR.Coordinates.x
38 cameraAngle = World.INSTANCE.CAR.Rotation
39
40 // rendering
41
42 if(g.getShaderRenderer == null) {
43     g.setShader(Mode7.SHADER_PATH)
44 }
45
46 g.getShaderRenderer.setTexture(World.INSTANCE.TRACK.Texture, 0)
47 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.ENABLE,
Mode7.DEFAULT_VALUES.ENABLE)
48 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.POSITION,
cameraPosition)
49 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.AXIS, cameraAxis)
50 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.ANGLE, cameraAngle)
51 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.SCREEN.PLAN.DISTANCE, 0.5f /
Math.atan(cameraFov / 2.0).toFloat())
52 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.RESOLUTION, new
Vector2(g.getScreenWidth.toFloat(), g.getScreenHeight.toFloat()))
53 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.PITCH, pitch)
54 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.RENDERING_FACTOR,
renderingFactor)
55
56 val trackRectangle = World.INSTANCE.TRACK.Geometry.trackSize
57 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.MAP_ORIGIN, new
Vector2(trackRectangle.x - MapTexture.MAP_PADDING, trackRectangle.y -
MapTexture.MAP_PADDING))
58
59 g.drawShader()
60
61 // Item rendering
62 ItemsRenderer.onGraphicRender(g)
63 }
64
65 private def graphicalSetup(): Unit = {
66     val ELAPSED_TIME = Gdx.graphics.getDeltaTime;
67
68     if (!Common.Debugging.IsDebugEnabled) return;
69
70     cameraFov = updateValue(cameraFov, Input.Keys.Q, Input.Keys.A, 1f,
ELAPSED_TIME);
71     pitch = updateValue(pitch, Input.Keys.W, Input.Keys.S, 0.01f, ELAPSED_TIME);
72     cameraPosition.z = updateValue(cameraPosition.z, Input.Keys.T, Input.Keys.G,
0.1f, ELAPSED_TIME);
73
74     DebugHUD.setLogVar("Track Render - Camera Position (+T, -G)",
cameraPosition.toString)
75     DebugHUD.setLogVar("Track Render - Camera FOV (+Q, -A)", cameraFov)
76     DebugHUD.setLogVar("Track Render - Pitch (+W, -S)", pitch)
77     DebugHUD.setLogVar("Track Render - RenderingFactor", renderingFactor.toString)
78 }
79
80 private def updateValue(v: Float, keyAdd: Int, keySub: Int, factor: Float,
elapsedTime: Float): Float = {
81     var r = v
82     if(Gdx.input.isKeyPressed(keySub)) r -= factor * elapsedTime
83     if(Gdx.input.isKeyPressed(keyAdd)) r += factor * elapsedTime
84     r
85 }
86 }

```


9.2.70 – HudRenderer.scala

```

1 package ch.hevs.fastandmudry
2 package render.hud
3
4 import ch.hevs.fastandmudry.core.world.World
5 import ch.hevs.fastandmudry.render.AbstractRenderer
6 import ch.hevs.fastandmudry.ui.hud.{CarHUD, DebugHUD, SpeedometerHUD,
7 TemperatureHUD}
8 import ch.hevs.fastandmudry.utils.Constant.RENDERING
9 import ch.hevs.gdx2d.lib.GdxGraphics
10
11 class HudRenderer extends AbstractRenderer {
12     private val skin = World.INSTANCE.selectedSkin
13     val Car = World.INSTANCE.CAR
14
15     override def onInit(): Unit = {
16         super.onInit()
17     }
18
19     override def onGraphicRender(g: GdxGraphics): Unit = {
20         if(!skin.skinBugged) {
21             RENDERING.CAR.INTERIOR_SCALE = 1f
22             RENDERING.CAR.INTERIOR_VERTICAL_ANCHOR = 0f
23         }
24         CarHUD.drawInterior(g)
25         SpeedometerHUD.draw(g, Car)
26         CarHUD.drawSteeringWheel(g, Car)
27         TemperatureHUD.draw(g, Car)
28         DebugHUD.draw()
29     }
30 }

```

9.2.71 – Mode7.scala

```

1 package ch.hevs.fastandmudry
2 package render.shaders
3
4 import com.badlogic.gdx.math.{Vector2, Vector3}
5
6 object Mode7 {
7     val SHADER_PATH = "data/shaders/mode7.glsl"
8     object Parameter {
9         object KEY {
10             val ENABLE = "enabled"
11             object CAMERA {
12                 val POSITION = "cameraPosition"
13                 val AXIS = "cameraAxis"
14                 val ANGLE = "cameraAngle"
15             }
16             object SCREEN {
17                 object PLAN {
18                     val DISTANCE = "screenPlanDistance"
19                 }
20             }
21             val RESOLUTION = "resolution"
22             val PITCH = "pitch"
23             val RENDERING_FACTOR = "renderingFactor"
24             val MAP_ORIGIN = "mapOrigin"
25         }
26     }
27     object DEFAULT_VALUES {
28         val ENABLE = true
29         object CAMERA {
30             val FOV: Float = Math.PI.toFloat / 4f;
31             val ANGLE = 0f

```

```

32     val POSITION = new Vector3(0f, 0f, 10f)
33     val AXIS = new Vector3(0f, -0f, -0.1f)
34 }
35 val PITCH = 0f
36 val RENDERING_FACTOR = new Vector2(0.01f, 0.01f)
37 }
38 }

```

9.2.72 – WorldRenderer.scala

```

1  package ch.hevs.fastandmudry
2  package render
3
4  import ch.hevs.fastandmudry.render.background.BackgroundRenderer
5  import ch.hevs.fastandmudry.render.game.GameRenderer
6  import ch.hevs.fastandmudry.render.hud.HudRenderer
7  import ch.hevs.gdx2d.lib.GdxGraphics
8
9  class WorldRenderer extends AbstractRenderer {
10     private val backgroundRenderer = new BackgroundRenderer
11     private val hudRenderer = new HudRenderer
12
13
14     override def onInit(): Unit = {
15         super.onInit()
16         backgroundRenderer.onInit()
17         GameRenderer.onInit()
18     }
19
20     override def onGraphicRender(g: GdxGraphics): Unit = {
21         backgroundRenderer.onGraphicRender(g)
22         GameRenderer.onGraphicRender(g)
23         hudRenderer.onGraphicRender(g)
24     }
25 }

```

9.2.73 – AbstractScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens
3
4  import ch.hevs.gdx2d.components.bitmaps.BitmapImage
5  import ch.hevs.gdx2d.components.screen_management.RenderingScreen
6  import ch.hevs.gdx2d.lib.GdxGraphics
7  import com.badlogic.gdx.scenes.scene2d.Stage
8  import com.badlogic.gdx.utils.viewport.ScreenViewport
9
10 abstract class AbstractScreen extends RenderingScreen {
11     protected lazy val LOGO: BitmapImage = new BitmapImage("data/images/
FastAndMudry_logo.png")
12
13     // https://libgdx.com/wiki/graphics/2d/scene2d/scene2d
14     protected val stage: Stage = new Stage(new ScreenViewport())
15     protected def renderStage(g: GdxGraphics, dt: Float): Unit = {
16         g.end() // we have to close the first batch (gdx) to open the new batch (scene)
17         stage.act(dt)
18         stage.draw()
19     }
20 }

```

9.2.74 – CarSelectorScreen.scala

```

1  package ch.hevs.fastandmudry

```

```

2 package screens.carSelector
3
4 import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5 import ch.hevs.fastandmudry.core.car.{CarSkin, CarSkins}
6 import ch.hevs.fastandmudry.core.state.{BackToMenu, CarSkinSelected,
7 GameStateMachine}
8 import ch.hevs.fastandmudry.core.world.World
9 import ch.hevs.fastandmudry.screens.AbstractScreen
10 import ch.hevs.fastandmudry.ui.components.ButtonFactory
11 import ch.hevs.gdx2d.utils.Constant.CAR_SELECTOR
12 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
13 import ch.hevs.gdx2d.lib.GdxGraphics
14 import com.badlogic.gdx.graphics.g2d.BitmapFont
15 import com.badlogic.gdx.{Gdx, Input}
16
17 class CarSelectorScreen extends AbstractScreen {
18     private val background: BitmapImage = new
19     BitmapImage(CAR_SELECTOR.BACKGROUND_IMAGE)
20     private val titleFont: BitmapFont = new BitmapFont()
21     titleFont.getData().setScale(CAR_SELECTOR.FONT_SIZE)
22     private var currentIndex: Int = CarSkins.all.indexOf(World.INSTANCE.selectedSkin)
23     private def currentSkin: CarSkin = CarSkins.all(currentIndex)
24     private val carImages: Array[BitmapImage] = CarSkins.all.map(skin => new
25     BitmapImage(skin.threeQuarterImagePath)).toArray
26
27     private val btnPrevious = ButtonFactory.primary("<")
28     btnPrevious.setSize(CAR_SELECTOR.BUTTON_WIDTH, CAR_SELECTOR.BUTTON_HEIGHT)
29     btnPrevious.setPosition(
30         CAR_SELECTOR.SIDE_PADDING,
31         (Gdx.graphics.getHeight - btnPrevious.getHeight) / 2
32     )
33     btnPrevious.onClick(() => previous())
34
35     private val btnNext = ButtonFactory.primary(">")
36     btnNext.setSize(CAR_SELECTOR.BUTTON_WIDTH, CAR_SELECTOR.BUTTON_HEIGHT)
37     btnNext.setPosition(
38         Gdx.graphics.getWidth - CAR_SELECTOR.SIDE_PADDING - btnNext.getWidth,
39         (Gdx.graphics.getHeight - btnNext.getHeight) / 2
40     )
41     btnNext.onClick(() => next())
42
43     private val btnSelect = ButtonFactory.primary("Choisir")
44     btnSelect.setSize(CAR_SELECTOR.SELECT_BUTTON_WIDTH,
45     CAR_SELECTOR.SELECT_BUTTON_HEIGHT)
46     btnSelect.setPosition(
47         (Gdx.graphics.getWidth - btnSelect.getWidth) / 2,
48         CAR_SELECTOR.TOP_BOTTOM_PADDING
49     )
50     btnSelect.onClick(() => onSkinSelected(currentSkin))
51
52     private val btnExit = ButtonFactory.primary("Exit")
53     btnExit.setSize(CAR_SELECTOR.EXIT_BUTTON_WIDTH, CAR_SELECTOR.EXIT_BUTTON_HEIGHT)
54     btnExit.setPosition(CAR_SELECTOR.SIDE_PADDING, Gdx.graphics.getHeight -
55     btnExit.getHeight - CAR_SELECTOR.TOP_BOTTOM_PADDING)
56     btnExit.onClick(() => GameStateMachine.handle(BackToMenu))
57
58     override def onInit(): Unit = {
59         AudioManager.playTrack(MusicTrack.Menu)
60         Gdx.input.setInputProcessor(stage)
61         stage.addActor(btnPrevious)
62         stage.addActor(btnNext)
63         stage.addActor(btnSelect)
64         stage.addActor(btnExit)
65     }
66
67     override def onGraphicRender(g: GdxGraphics): Unit = {
68         g.clear()
69     }
70 }

```

```

64     drawBackground(g)
65     drawCar(g)
66     g.drawStringCentered(CAR_SELECTOR.TOP_BOTTOM_PADDING +
CAR_SELECTOR.SELECT_BUTTON_HEIGHT + 100, currentSkin.carName, titleFont)
67
68     if (Gdx.input.isKeyJustPressed(Input.Keys.ESCAPE)) {
69         GameStateMachine.handle(BackToMenu)
70     }
71
72     renderStage(g, Gdx.graphics.getDeltaTime)
73 }
74
75 private def previous(): Unit = {
76     currentIndex = (currentIndex - 1 + CarSkins.all.length) % CarSkins.all.length
77 }
78
79 private def next(): Unit = {
80     currentIndex = (currentIndex + 1) % CarSkins.all.length
81 }
82
83 private def drawBackground(g: GdxGraphics): Unit = {
84     val screenW = g.getScreenWidth.toFloat
85     val screenH = g.getScreenHeight.toFloat
86     val imgW = background.getImage.getWidth.toFloat
87     val imgH = background.getImage.getHeight.toFloat
88
89     val scale = math.min(screenW / imgW, screenH / imgH)
90     val drawW = imgW * scale
91     val drawH = imgH * scale
92
93     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
2f, background)
94 }
95
96 private def drawCar(g: GdxGraphics): Unit = {
97     val carImage = carImages(currentIndex)
98     val screenW = g.getScreenWidth.toFloat
99     val screenH = g.getScreenHeight.toFloat
100     val imgW = carImage.getImage.getWidth.toFloat
101     val imgH = carImage.getImage.getHeight.toFloat
102
103     g.drawTransformedPicture(screenW / 2f, screenH / 2f +
CAR_SELECTOR.CAR_VERTICAL_OFFSET, 0f, imgW / 2f, imgH / 2f, carImage)
104 }
105
106 private def onSkinSelected(skin: CarSkin): Unit = {
107     World.INSTANCE.selectedSkin = skin
108     GameStateMachine.handle(CarSkinSelected(skin))
109 }
110 }

```

9.2.75 – CinematicScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens.cinematic
3
4  import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5  import ch.hevs.fastandmudry.core.state._
6  import ch.hevs.fastandmudry.core.world.World
7  import ch.hevs.fastandmudry.screens.AbstractScreen
8  import ch.hevs.fastandmudry.utils.Constant.CINEMATIC
9  import ch.hevs.gdx2d.components.bitmaps.BitmapImage
10 import ch.hevs.gdx2d.lib.GdxGraphics
11 import com.badlogic.gdx.Gdx
12
13 class CinematicScreen extends AbstractScreen {

```

```

14 private var timeElapsed: Float = 0f
15 private val DURATION: Float = 5f
16 private val FINAL_DURATION: Float = 15f
17 private val carImage: BitmapImage = new
18 BitmapImage(World.INSTANCE.selectedSkin.sideImagePath)
19 private val frontFinalBackground: BitmapImage = new
20 BitmapImage(CINEMATIC.FRONT_FINAL_BACKGROUND_IMAGE)
21 private val backFinalBackground: BitmapImage = new
22 BitmapImage(CINEMATIC.BACK_FINAL_BACKGROUND_IMAGE)
23 private val mudryImage: BitmapImage = new
24 BitmapImage(CINEMATIC.MUDRY_BACKGROUND_IMAGE)
25
26 override def onInit(): Unit = {
27     // keep last biome music for final cinematic
28     if (GameStateMachine.getGameState != GameState.FinalCinematic)
29         AudioManager.playTrack(MusicTrack.Cinematic)
30     timeElapsed = 0f
31 }
32
33 override def onGraphicRender(g: GdxGraphics): Unit = {
34     g.clear()
35     timeElapsed += Gdx.graphics.getDeltaTime
36
37     GameStateMachine.getGameState match {
38         case GameState.StartDayCinematic(day) =>
39             drawBackground(g, day.startDayCinematicBackground)
40             drawCar(g, g.getScreenWidth.toFloat / 2,
41 carImage.getImage.getHeight.toFloat, 0f, 0f)
42             if (timeElapsed >= DURATION)
43                 GameStateMachine.handle(StartDayCinematicEnded)
44
45         case GameState.EndDayCinematic(day) =>
46             drawBackground(g, day.endDayCinematicBackground)
47             drawCar(g, 0f, 0f, g.getScreenWidth.toFloat / 2,
48 carImage.getImage.getHeight.toFloat)
49             if (timeElapsed >= DURATION) GameStateMachine.handle(EndDayCinematicEnded)
50
51         case GameState.FinalCinematic =>
52             drawFinalBackground(g, frontFinalBackground, backFinalBackground,
53 mudryImage, g.getScreenWidth.toFloat / 2, 1f, g.getScreenHeight / 1.5f)
54             drawCar(g, 0f, 0f, g.getScreenWidth.toFloat / 1.5f, g.getScreenHeight / 5f)
55             if (timeElapsed >= FINAL_DURATION)
56                 GameStateMachine.handle(FinalCinematicEnded)
57
58         case _ =>
59             }
60     }
61
62 private def drawBackground(g: GdxGraphics, background: BitmapImage): Unit = {
63     val screenW = g.getScreenWidth.toFloat
64     val screenH = g.getScreenHeight.toFloat
65     val imgW = background.getImage.getWidth.toFloat
66     val imgH = background.getImage.getHeight.toFloat
67
68     val scale = math.min(screenW / imgW, screenH / imgH)
69     val drawW = imgW * scale
70     val drawH = imgH * scale
71
72     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
73 2f, background)
74 }
75
76 private def drawFinalBackground(g: GdxGraphics, frontBackground: BitmapImage,
77 backBackground: BitmapImage, mudry: BitmapImage, x: Float, y: Float, endY: Float):
78 Unit = {
79     val screenW = g.getScreenWidth.toFloat

```

```

68     val screenH = g.getScreenHeight.toFloat
69     val imgW = frontBackground.getImage.getWidth.toFloat
70     val imgH = frontBackground.getImage.getHeight.toFloat
71
72     val scale = math.min(screenW / imgW, screenH / imgH)
73     val drawW = imgW * scale
74     val drawH = imgH * scale
75
76     val mudryW = mudry.getImage.getWidth.toFloat
77     val mudryH = mudry.getImage.getHeight.toFloat
78     var yT = y * timeElapsed*100f
79
80     yT = if(yT ≤ endY) yT else endY
81
82     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
2f, backBackground)
83     g.drawTransformedPicture(x, yT, 0f, mudryW / 2f, mudryH / 2f, mudry)
84     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
2f, frontBackground)
85 }
86
87 private def drawCar(g: GdxGraphics, startX: Float, startY: Float, endX: Float,
endY: Float): Unit = {
88     val imgW = carImage.getImage.getWidth.toFloat
89     val imgH = carImage.getImage.getHeight.toFloat
90
91     val dirX = if(startX > endX) -1 else 1
92     val dirY = if(startY > endY) -1 else 1
93
94     var x = dirX * timeElapsed*400f
95     var y = dirY * timeElapsed*200f
96
97     x = if(x ≤ endX) x else endX
98     y = if(y ≤ endY) y else endY
99
100     g.drawTransformedPicture(startX + x, startY + y, 0f, imgW / 2f, imgH / 2f,
carImage)
101 }
102 }

```

9.2.76 – CustomScreenManager.scala

```

1  package ch.hevs.fastandmudry
2  package screens
3
4  import ch.hevs.fastandmudry.screens.carSelector.CarSelectorScreen
5  import ch.hevs.fastandmudry.screens.cinematic.CinematicScreen
6  import ch.hevs.fastandmudry.screens.death.DeathScreen
7  import ch.hevs.fastandmudry.screens.debug.CarDebugScreen
8  import ch.hevs.fastandmudry.screens.game.GameScreen
9  import ch.hevs.fastandmudry.screens.garage.GarageScreen
10 import ch.hevs.fastandmudry.screens.loading.LoadingScreen
11 import ch.hevs.fastandmudry.screens.menu.MenuScreen
12 import ch.hevs.fastandmudry.screens.quiz.QuizScreen
13 import ch.hevs.gdx2d.lib.ScreenManager
14
15 object CustomScreenManager {
16     private val s = new ScreenManager
17     val LOADING = 0
18     val MENU = 1
19     val GAME = 2
20     val CINEMATIC = 3
21     val QUIZ = 4
22     val DEATH = 5
23     val GARAGE = 6
24     val CAR_SELECTOR = 7

```

```

25 val CAR_DEBUG = 8
26
27 def getInstance: ScreenManager = s
28
29 // Register Screen
30 s.registerScreen(classOf[LoadingScreen])
31 s.registerScreen(classOf[MenuScreen])
32 s.registerScreen(classOf[GameScreen])
33 s.registerScreen(classOf[CinematicScreen])
34 s.registerScreen(classOf[QuizScreen])
35 s.registerScreen(classOf[DeathScreen])
36 s.registerScreen(classOf[GarageScreen])
37 s.registerScreen(classOf[CarSelectorScreen])
38 s.registerScreen(classOf[CarDebugScreen])
39 }

```

9.2.77 – DeathScreen.scala

```

1 package ch.hevs.fastandmudry
2 package screens.death
3
4 import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5 import ch.hevs.fastandmudry.core.state.{BackToMenu, GameStateMachine}
6 import ch.hevs.fastandmudry.core.world.World
7 import ch.hevs.fastandmudry.screens.AbstractScreen
8 import ch.hevs.fastandmudry.utils.Constant.DEATH
9 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
10 import ch.hevs.gdx2d.lib.GdxGraphics
11 import com.badlogic.gdx.{Gdx, Input}
12
13 class DeathScreen extends AbstractScreen {
14     private val background: BitmapImage = new BitmapImage(DEATH.BACKGROUND_IMAGE)
15     private val carImage: BitmapImage = new
16     BitmapImage(World.INSTANCE.selectedSkin.openHoodImagePath)
17
18     override def onInit(): Unit = {
19         AudioManager.playTrack(MusicTrack.Menu)
20     }
21
22     override def onGraphicRender(g: GdxGraphics): Unit = {
23         g.clear()
24         drawBackground(g, background)
25         drawCar(g)
26
27         if (Gdx.input.isKeyJustPressed(Input.Keys.SPACE)) {
28             GameStateMachine.handle(BackToMenu)
29         }
30     }
31
32     private def drawBackground(g: GdxGraphics, background: BitmapImage): Unit = {
33         val screenW = g.getScreenWidth.toFloat
34         val screenH = g.getScreenHeight.toFloat
35         val imgW = background.getImage.getWidth.toFloat
36         val imgH = background.getImage.getHeight.toFloat
37
38         val scale = math.min(screenW / imgW, screenH / imgH)
39         val drawW = imgW * scale
40         val drawH = imgH * scale
41
42         g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
43         2f, background)
44     }
45
46     private def drawCar(g: GdxGraphics): Unit = {
47         val screenW = g.getScreenWidth.toFloat
48         val screenH = g.getScreenHeight.toFloat
49         val imgW = carImage.getImage.getWidth.toFloat
50
51     }

```



```

47     val imgH = carImage.getImage.getHeight.toFloat
48
49     val drawW = imgW * 0.35f
50     val drawH = imgH * 0.35f
51
52     g.drawTransformedPicture(screenW/1.7f, screenH/10f, 0f, drawW / 2f, drawH / 2f,
53     carImage)
54 }
55 }

```

9.2.78 – CarDebugScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens.debug
3
4  import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5  import ch.hevs.fastandmudry.core.state.{BackToMenu, GameStateMachine}
6  import ch.hevs.fastandmudry.core.world.World
7  import ch.hevs.fastandmudry.screens.AbstractScreen
8  import ch.hevs.fastandmudry.ui.hud.{CarHUD, SpeedometerHUD, TemperatureHUD}
9  import ch.hevs.gdx2d.lib.GdxGraphics
10 import com.badlogic.gdx.{Gdx, Input}
11
12
13 class CarDebugScreen extends AbstractScreen {
14     private val car = World.INSTANCE.CAR
15
16     override def onInit(): Unit = {
17         AudioManager.playTrack(MusicTrack.Menu)
18         Gdx.input.setInputProcessor(stage)
19     }
20
21     override def onGraphicRender(g: GdxGraphics): Unit = {
22         g.clear()
23         CarHUD.drawInterior(g)
24         SpeedometerHUD.draw(g, car)
25         CarHUD.drawSteeringWheel(g, car)
26         TemperatureHUD.draw(g, car)
27
28         if (Gdx.input.isKeyJustPressed(Input.Keys.ESCAPE)) {
29             GameStateMachine.handle(BackToMenu)
30         }
31     }
32 }

```

9.2.79 – GameScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens.game
3
4  import ch.hevs.fastandmudry.core.audio.AudioManager
5  import ch.hevs.fastandmudry.core.world.World
6  import ch.hevs.fastandmudry.input.DebugInput
7  import ch.hevs.fastandmudry.render.WorldRenderer
8  import ch.hevs.fastandmudry.screens.AbstractScreen
9  import ch.hevs.gdx2d.lib.GdxGraphics
10 import com.badlogic.gdx.Gdx
11
12 class GameScreen extends AbstractScreen {
13     val WorldRenderer = new WorldRenderer
14
15     override def onInit(): Unit = {
16         AudioManager.playTrack(World.INSTANCE.TRACK.biome.musicTrack)
17         AudioManager.startEngine()

```



```

18     WorldRenderer.onInit()
19 }
20
21 override def dispose(): Unit = {
22     AudioManager.stopEngine()
23 }
24
25 override def onGraphicRender(g: GdxGraphics): Unit = {
26     g.clear()
27
28     val ELAPSED_TIME = Gdx.graphics.getDeltaTime;
29
30     World.INSTANCE.onGameLoop(ELAPSED_TIME)
31
32     WorldRenderer.onGraphicRender(g)
33
34     g.end()
35
36     DebugInput.debuggingKeyProcess()
37 }
38 }

```

9.2.80 – GarageScreen.scala

```

1 package ch.hevs.fastandmudry
2 package screens.garage
3
4 import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5 import ch.hevs.fastandmudry.core.state.{GameStateMachine, GarageReady, Wallet}
6 import ch.hevs.fastandmudry.core.world.World
7 import ch.hevs.fastandmudry.screens.AbstractScreen
8 import ch.hevs.fastandmudry.ui.UISkin
9 import ch.hevs.fastandmudry.ui.components.ListItemRow
10 import ch.hevs.fastandmudry.ui.hud.WalletHUD
11 import ch.hevs.fastandmudry.utils.Constant.GARAGE
12 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
13 import ch.hevs.gdx2d.lib.GdxGraphics
14 import com.badlogic.gdx.scenes.scene2d.ui.{ScrollPane, Table}
15 import com.badlogic.gdx.{Gdx, Input}
16
17 class GarageScreen extends AbstractScreen {
18     val BOTTOM_MESSAGE: String = "Press SPACE to continue"
19     private val background: BitmapImage = new BitmapImage(GARAGE.BACKGROUND_IMAGE)
20     private val carImage: BitmapImage = new
21     BitmapImage(World.INSTANCE.selectedSkin.openHoodImagePath)
22
23     override def onInit(): Unit = {
24         AudioManager.playTrack(MusicTrack.Garage)
25         Gdx.input.setInputProcessor(stage)
26         buildList()
27     }
28
29     override def onGraphicRender(g: GdxGraphics): Unit = {
30         g.clear()
31         drawBackground(g)
32         drawCar(g)
33         g.drawStringCentered(g.getScreenHeight * 0.06f, BOTTOM_MESSAGE)
34
35         if (Gdx.input.isKeyJustPressed(Input.Keys.SPACE)) {
36             GameStateMachine.handle(GarageReady)
37         }
38
39         renderStage(g, Gdx.graphics.getDeltaTime)
40         WalletHUD.draw(g)
41     }
42
43     private def drawBackground(g: GdxGraphics): Unit = {

```

```

43     val screenW = g.getScreenWidth.toFloat
44     val screenH = g.getScreenHeight.toFloat
45     val imgW = background.getImage.getWidth.toFloat
46     val imgH = background.getImage.getHeight.toFloat
47
48     val scale = math.min(screenW / imgW, screenH / imgH)
49     val drawW = imgW * scale
50     val drawH = imgH * scale
51
52     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
53 2f, background)
54 }
55
56 private def drawCar(g: GdxGraphics): Unit = {
57     val screenW = g.getScreenWidth.toFloat
58     val screenH = g.getScreenHeight.toFloat
59     val imgW = carImage.getImage.getWidth.toFloat
60     val imgH = carImage.getImage.getHeight.toFloat
61
62     g.drawTransformedPicture((screenW+screenW/2) / 2f, (screenH-screenH/4) / 2f,
63 0f, imgW / 2f, imgH / 2f, carImage)
64 }
65
66 private def buildList(): Unit = {
67     val repairsList = new Table(UISkin.skin)
68
69     World.INSTANCE.CAR.getReparableProblems.foreach(item => {
70         val row = ListItemRow.create(item.IconPath, item.Title, s"${item.ButtonText}
71 - ${item.ReparationPrice}.-")
72         row.onClick(() => {
73             if(item.ReparationPrice <= Wallet.coins) {
74                 item.IsDefected = false
75                 Wallet.spend(item.ReparationPrice)
76                 row.setDisabled(!item.IsDefected)
77             }
78         })
79         row.setDisabled(!item.IsDefected)
80         repairsList.add(row).growX().pad(GARAGE.ROW_GAP)
81         repairsList.row() // line break
82     })
83
84     val scrollPane = new ScrollPane(repairsList, UISkin.skin)
85     scrollPane.setFadeScrollBars(false)
86     scrollPane.setScrollingDisabled(true, false)
87     scrollPane.setSize(GARAGE.LIST_WIDTH, Gdx.graphics.getHeight - 2 *
88 GARAGE.LIST_PADDING)
89     scrollPane.setPosition(GARAGE.LIST_PADDING, GARAGE.LIST_PADDING)
90
91     stage.addActor(scrollPane)
92 }
93
94 private def onItemClick(text: String): Unit = {
95     println(s"Item: $text clicked")
96 }
97 }

```

9.2.81 – LoadingScreen.scala

```

1 package ch.hevs.fastandmudry
2 package screens.loading
3
4 import ch.hevs.fastandmudry.core.state.{GameState, GameStateMachine, MapLoaded}
5 import ch.hevs.fastandmudry.core.world.World
6 import ch.hevs.fastandmudry.screens.AbstractScreen
7 import ch.hevs.gdx2d.lib.GdxGraphics
8 import com.badlogic.gdx.Gdx

```

```

9  import com.badlogic.gdx.graphics.Pixmap
10 import com.badlogic.gdx.math.Interpolation
11
12 class LoadingScreen extends AbstractScreen {
13     private var isFinishedLoading = false
14     private var tempPixmap: Pixmap = _
15
16     /**
17      * Some animation related variables
18      */
19     private var direction: Int = 1
20     private var currentTime: Float = 0
21     final private val ANIMATION_LENGTH: Float = 2f // Animation length (in seconds)
22     final private val MIN_ANGLE: Float = -20
23     final private val MAX_ANGLE: Float = 20
24
25     override def onInit(): Unit = {
26         isFinishedLoading = false
27         val track = World.INSTANCE.TRACK
28         GameStateMachine.getGameState match {
29             case GameState.Loading(day) => track.setBiome(day.newBiome)
30             case _ =>
31         }
32         val worker = new Thread(() => {
33             tempPixmap = track.buildPixmap()
34         })
35         worker.setDaemon(true)
36         worker.start()
37     }
38
39     override def onGraphicRender(g: GdxGraphics): Unit = {
40         g.clear()
41         val t = computePercentage
42         val angle: Float = Interpolation.sine.apply(MIN_ANGLE, MAX_ANGLE, t)
43
44         g.drawTransformedPicture(g.getScreenWidth / 2.0f, g.getScreenHeight / 2.0f,
45         angle, 0.7f, LOGO)
46         g.drawStringCentered(g.getScreenHeight * 0.9f, "Loading... ")
47
48         if (!isFinishedLoading && tempPixmap != null) {
49             World.INSTANCE.TRACK.installPixmap(tempPixmap)
50             tempPixmap = null
51             isFinishedLoading = true
52         }
53
54         if (isFinishedLoading) {
55             Gdx.app.log("LoadingScreen", "Lancement du jeu !")
56             GameStateMachine.handle(MapLoaded)
57         }
58     }
59
60     /**
61      * Compute time percentage for making a looping animation
62      * @return the current normalized time
63      */
64     protected def computePercentage: Float = {
65         if (direction == 1) {
66             currentTime += Gdx.graphics.getDeltaTime
67             if (currentTime > ANIMATION_LENGTH) {
68                 currentTime = ANIMATION_LENGTH
69                 direction *= -1
70             }
71         }
72         else {
73             currentTime -= Gdx.graphics.getDeltaTime
74             if (currentTime < 0) {

```

```

75     currentTime = 0
76     direction *= -1
77 }
78 }
79     currentTime / ANIMATION_LENGTH
80 }
81 }

```

9.2.82 – MenuScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens.menu
3
4  import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5  import ch.hevs.fastandmudry.core.quiz.QuizData
6  import ch.hevs.fastandmudry.core.state._
7  import ch.hevs.fastandmudry.screens.AbstractScreen
8  import ch.hevs.fastandmudry.ui.components.ButtonFactory
9  import ch.hevs.fastandmudry.ui.dialogs.{DialogFactory, SettingsDialog}
10 import ch.hevs.fastandmudry.utils.Constant.MENU
11 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
12 import ch.hevs.gdx2d.lib.GdxGraphics
13 import com.badlogic.gdx.{Gdx, Input}
14
15 class MenuScreen extends AbstractScreen {
16     private val background: BitmapImage = new BitmapImage(MENU.BACKGROUND_IMAGE)
17     private val BUTTON_WIDTH = 300
18     private val BUTTON_HEIGHT = 80
19     private val xButtonPosition = (Gdx.graphics.getWidth - BUTTON_WIDTH) / 2
20     private val yButtonPosition = (Gdx.graphics.getHeight - BUTTON_HEIGHT) / 3
21
22     private val btnPlay = ButtonFactory.primary("Play")
23     btnPlay.setSize(BUTTON_WIDTH, BUTTON_HEIGHT)
24     btnPlay.setPosition(xButtonPosition, yButtonPosition)
25     btnPlay.onClick(() => {
26         QuizData.reset()
27         Wallet.reset()
28         GameStateMachine.handle(StartGame)
29     })
30
31     private val settingsDialog: SettingsDialog =
32     DialogFactory.createSettingsDialog("Settings")
33
34     private val btnSettings = ButtonFactory.primary("Settings")
35     btnSettings.setSize(BUTTON_WIDTH, BUTTON_HEIGHT)
36     btnSettings.setPosition(xButtonPosition, yButtonPosition - 100)
37     btnSettings.onClick(() => settingsDialog.show(stage))
38
39     private val btnCarSelector = ButtonFactory.primary("Car selector")
40     btnCarSelector.setSize(BUTTON_WIDTH, BUTTON_HEIGHT)
41     btnCarSelector.setPosition(xButtonPosition, yButtonPosition - 200)
42     btnCarSelector.onClick(() => GameStateMachine.handle(OpenCarSelector))
43
44     override def onInit(): Unit = {
45         AudioManager.playTrack(MusicTrack.Menu)
46         Gdx.input.setInputProcessor(stage) // send inputs events to the stage
47         stage.addActor(btnPlay)
48         stage.addActor(btnSettings)
49         stage.addActor(btnCarSelector)
50     }
51
52     override def onGraphicRender(g: GdxGraphics): Unit = {
53         g.clear()
54         drawBackground(g)
55
56         if (Gdx.input.isKeyJustPressed(Input.Keys.F1)) {

```

```

56     GameStateMachine.handle(OpenCarDebug)
57 }
58
59 renderStage(g, Gdx.graphics.getDeltaTime)
60 }
61
62 private def drawBackground(g: GdxGraphics): Unit = {
63     val screenW = g.getScreenWidth.toFloat
64     val screenH = g.getScreenHeight.toFloat
65     val imgW = background.getImage.getWidth.toFloat
66     val imgH = background.getImage.getHeight.toFloat
67
68     val scale = math.min(screenW / imgW, screenH / imgH)
69     val drawW = imgW * scale
70     val drawH = imgH * scale
71
72     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
73 2f, background)
74 }

```

9.2.83 – QuizScreen.scala

```

1  package ch.hevs.fastandmudry
2  package screens.quiz
3
4  import ch.hevs.fastandmudry.core.audio.{AudioManager, MusicTrack}
5  import ch.hevs.fastandmudry.core.quiz.{Quiz, QuizData, QuizPhase}
6  import ch.hevs.fastandmudry.core.state.{GameStateMachine, QuizCompleted, Wallet}
7  import ch.hevs.fastandmudry.screens.AbstractScreen
8  import ch.hevs.fastandmudry.ui.components.{ButtonFactory, CustomButton}
9  import ch.hevs.fastandmudry.ui.hud.WalletHUD
10 import ch.hevs.fastandmudry.utils.Constant.QUIZ
11 import ch.hevs.gdx2d.components.bitmaps.BitmapImage
12 import ch.hevs.gdx2d.lib.GdxGraphics
13 import com.badlogic.gdx.graphics.Color
14 import com.badlogic.gdx.graphics.g2d.BitmapFont
15 import com.badlogic.gdx.{Gdx, Input}
16
17 class QuizScreen extends AbstractScreen {
18     private var quiz: Quiz = _
19     private val background: BitmapImage = new BitmapImage(QUIZ.BACKGROUND_IMAGE)
20     private var revealTimer: Float = 0f
21     private var buttons: Array[CustomButton] = Array.empty
22     private val titleFont: BitmapFont = new BitmapFont()
23     titleFont.getData.setScale(1.5f)
24     titleFont.setColor(Color.BLACK)
25
26     override def onInit(): Unit = {
27         AudioManager.playTrack(MusicTrack.Quiz)
28         Gdx.input.setInputProcessor(stage)
29         quiz = new Quiz(QuizData.nextSession())
30         revealTimer = 0f
31     }
32
33     override def onGraphicRender(g: GdxGraphics): Unit = {
34         g.clear()
35         drawBackground(g)
36         g.drawStringCentered(g.getScreenHeight * 0.85f, quiz.currentQuestion.text,
37 titleFont)
38
39         quiz.getPhase match {
40             case QuizPhase.ShowingQuestion => {
41                 g.drawStringCentered(g.getScreenHeight * 0.08f, "Press SPACE")
42                 if (Gdx.input.isKeyJustPressed(Input.Keys.SPACE)) {
43                     quiz.revealAnswers()
44                 }
45             }
46         }
47     }
48 }

```

```

43     buildAnswerButtons()
44     }
45 }
46 case QuizPhase.ShowingAnswers => // nothing, the click on button does the job
47 case QuizPhase.Revealing => {
48     revealTimer += Gdx.graphics.getDeltaTime
49     if (revealTimer ≥ QUIZ.REVEAL_DURATION) {
50         clearAnswerButtons()
51         if (!quiz.doContinueQuestions()) GameStateMachine.handle(QuizCompleted)
52     }
53 }
54 }
55 WalletHUD.draw(g)
56 renderStage(g, Gdx.graphics.getDeltaTime)
57 }
58
59 private def drawBackground(g: GdxGraphics): Unit = {
60     val screenW = g.getScreenWidth.toFloat
61     val screenH = g.getScreenHeight.toFloat
62     val imgW = background.getImage.getWidth.toFloat
63     val imgH = background.getImage.getHeight.toFloat
64
65     val scale = math.min(screenW / imgW, screenH / imgH)
66     val drawW = imgW * scale
67     val drawH = imgH * scale
68
69     g.drawTransformedPicture(screenW / 2f, screenH / 2f, 0f, drawW / 2f, drawH /
70 2f, background)
71 }
72
73 private def buildAnswerButtons(): Unit = {
74     val answers = quiz.currentQuestion.answers
75     val totalWidth = 2 * QUIZ.BUTTON_WIDTH + QUIZ.GRID_GAP
76     val totalHeight = 2 * QUIZ.BUTTON_HEIGHT + QUIZ.GRID_GAP
77     val startX = (Gdx.graphics.getWidth - totalWidth) / 2.2f
78     val startY = (Gdx.graphics.getHeight - totalHeight) / 1.5f
79
80     buttons = new Array[CustomButton](answers.length)
81
82     for (i ← answers.indices) {
83         val col = i % 2
84         val row = i / 2
85
86         val button = ButtonFactory.primary(answers(i))
87         button.setSize(QUIZ.BUTTON_WIDTH, QUIZ.BUTTON_HEIGHT)
88         button.setPosition(
89             startX + col * (QUIZ.BUTTON_WIDTH + QUIZ.GRID_GAP),
90             startY + (1 - row) * (QUIZ.BUTTON_HEIGHT + QUIZ.GRID_GAP)
91         )
92         button.onClick(() => onAnswerClicked(i))
93
94         stage.addActor(button)
95         buttons(i) = button
96     }
97 }
98
99 private def onAnswerClicked(index: Int): Unit = {
100     if (quiz.getPhase ≠ QuizPhase.ShowingAnswers) return
101
102     val correct = quiz.answer(index)
103     if (correct) Wallet.add(QUIZ.COINS_PER_CORRECT_ANSWER)
104
105     val correctIndex = quiz.currentQuestion.correctIndex
106     for (i ← buttons.indices) {
107         buttons(i).setDisabled(i ≠ correctIndex)
108         val c = if (i == correctIndex) Color.GREEN else Color.DARK_GRAY
109         buttons(i).setColor(c)

```

```

109     }
110
111     revealTimer = 0f
112 }
113
114 private def clearAnswerButtons(): Unit = {
115     stage.clear()
116     buttons = Array.empty
117 }
118 }

```

9.2.84 – ButtonGroup.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import com.badlogic.gdx.scenes.scene2d.ui.ButtonGroup
5
6 class CustomButtonGroup(buttons: Array[CustomButton]) extends
7 ButtonGroup[CustomButton]() {
8     buttons.foreach(b => {
9         add(b)
10     })
11 }
12
13 object ButtonGroupFactory {
14     def create(buttons: Array[CustomButton]): CustomButtonGroup = new
15 CustomButtonGroup(buttons)
16 }

```

9.2.85 – Button.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.InputEvent
6 import com.badlogic.gdx.scenes.scene2d.ui.{Skin, TextButton}
7 import com.badlogic.gdx.scenes.scene2d.utils.ClickListener
8
9 class CustomButton(text: String, skin: Skin) extends TextButton(text, skin) {
10     def onClick(action: () => Unit): Unit = {
11         addListener(new ClickListener {
12             override def clicked(event: InputEvent, x: Float, y: Float): Unit = action()
13         })
14     }
15 }
16
17 object ButtonFactory {
18     def primary(text: String): CustomButton = new CustomButton(text, UISkin.skin)
19 }

```

9.2.86 – CheckBox.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.Actor
6 import com.badlogic.gdx.scenes.scene2d.ui.{CheckBox, Skin}
7 import com.badlogic.gdx.scenes.scene2d.utils.Changelistener
8
9 class CustomCheckBox(text: String, skin: Skin) extends CheckBox(text, skin) {

```

```

10     def onChange(action: Boolean ⇒ Unit): Unit = {
11         addListener(new ChangeListener {
12             override def changed(changeEvent: ChangeListener.ChangeEvent, actor: Actor):
Unit = action(isChecked)
13         })
14     }
15 }
16
17 object CheckBoxFactory {
18     def create(text: String): CustomCheckBox = new CustomCheckBox(text, UISkin.skin)
19 }

```

9.2.87 – Image.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.ui.{Image, Skin}
6
7 class CustomImage(drawableName: String, skin: Skin) extends Image(skin,
8     drawableName){
9 }
10
11 object ImageFactory {
12     def create(drawableName: String): CustomImage = new CustomImage(drawableName,
13         UISkin.skin)
14 }

```

9.2.88 – Label.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.ui.{Label, Skin}
6
7 class CustomLabel(text: String, skin: Skin) extends Label(text, skin){
8 }
9
10 object LabelFactory {
11     def create(text: String): CustomLabel = new CustomLabel(text, UISkin.skin)
12 }

```

9.2.89 – ListItemRow.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.graphics.g2d.TextureRegion
6 import com.badlogic.gdx.graphics.{Color, Texture}
7 import com.badlogic.gdx.scenes.scene2d.InputEvent
8 import com.badlogic.gdx.scenes.scene2d.ui.{Image, Skin, Table}
9 import com.badlogic.gdx.scenes.scene2d.utils.{ClickListener, TextureRegionDrawable}
10
11 class ListItemRow(imagePath: String, text: String, buttonText: String, skin: Skin)
12     extends Table(skin) {
13     private val texture = new Texture(com.badlogic.gdx.Gdx.files.internal(imagePath))
14     private val image = new Image(new TextureRegionDrawable(new
15         TextureRegion(texture)))
16     private val button = new CustomButton(buttonText, skin)

```



```

16 add(image).size(ListItemRow.IMAGE_SIZE).pad(ListItemRow.PADDING)
17 add(text, "default").expandX().left().pad(ListItemRow.PADDING)
18 add(button).pad(ListItemRow.PADDING)
19
20
21 def onClick(action: () => Unit): Unit = {
22     button.addListener(new ClickListener {
23         override def clicked(event: InputEvent, x: Float, y: Float): Unit = {
24             if(!button.isDisabled) action()
25         }
26     })
27 }
28
29 def setDisabled(value: Boolean): Unit = {
30     println(s"$text - diable : $value")
31     button.setDisabled(value)
32     button.setColor(if (value) Color.DARK_GRAY else Color.LIGHT_GRAY)
33 }
34 }
35
36 object ListItemRow {
37     val IMAGE_SIZE: Float = 64f
38     val PADDING: Float = 10f
39
40     def create(imagePath: String, text: String, buttonText: String): ListItemRow =
41         new ListItemRow(imagePath, text, buttonText, UISkin.skin)
42 }

```

9.2.90 – ProgressBar.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.Actor
6 import com.badlogic.gdx.scenes.scene2d.ui.{ProgressBar, Skin}
7 import com.badlogic.gdx.scenes.scene2d.utils.ChangeListener
8
9 class CustomProgressBar(min: Float, max: Float, stepSize: Float, vertical: Boolean,
10 skin: Skin) extends ProgressBar(min, max, stepSize, vertical, skin) {
11     def onChange(action: Float => Unit): Unit = {
12         addListener(new ChangeListener {
13             override def changed(changeEvent: ChangeListener.ChangeEvent, actor: Actor):
14 Unit = action(getValue)
15         })
16     }
17 }
18
19 object ProgressBarFactory {
20     def create(min: Float, max: Float, stepSize: Float, vertical: Boolean):
21 CustomProgressBar = new CustomProgressBar(min, max, stepSize, vertical,
22 UISkin.skin)
23 }

```

9.2.91 – SelectBox.scala

```

1 package ch.hevs.fastandmudry
2 package ui.components
3
4 import ch.hevs.fastandmudry.ui.UISkin
5 import com.badlogic.gdx.scenes.scene2d.Actor
6 import com.badlogic.gdx.scenes.scene2d.ui.{SelectBox, Skin}
7 import com.badlogic.gdx.scenes.scene2d.utils.ChangeListener
8

```

```

9  class CustomSelectBox(styleName: String, skin: Skin) extends SelectBox(skin,
10 styleName) {
11      def onChange[T](action: T ⇒ Unit): Unit = {
12          addListener(new ChangeListener {
13              override def changed(changeEvent: ChangeListener.ChangeEvent, actor: Actor):
14              Unit = action(getSelected)
15          })
16      }
17  }
18
19  object SelectBoxFactory {
20      def create(styleName: String): CustomSelectBox = new CustomSelectBox(styleName,
21      UISkin.skin)
22  }

```

9.2.92 – Slider.scala

```

1  package ch.hevs.fastandmudry
2  package ui.components
3
4  import ch.hevs.fastandmudry.ui.UISkin
5  import com.badlogic.gdx.scenes.scene2d.Actor
6  import com.badlogic.gdx.scenes.scene2d.ui.{Skin, Slider}
7  import com.badlogic.gdx.scenes.scene2d.utils.ChangeListener
8
9  class CustomSlider(min: Float, max: Float, stepSize: Float, vertical: Boolean,
10 skin: Skin) extends Slider(min, max, stepSize, vertical, skin) {
11      def onChange(action: Float ⇒ Unit): Unit = {
12          addListener(new ChangeListener {
13              override def changed(changeEvent: ChangeListener.ChangeEvent, actor: Actor):
14              Unit = action(getValue)
15          })
16      }
17  }
18
19  object SliderFactory {
20      def create(min: Float, max: Float, stepSize: Float, vertical: Boolean):
21      CustomSlider = new CustomSlider(min, max, stepSize, vertical, UISkin.skin)
22  }

```

9.2.93 – Dialog.scala

```

1  package ch.hevs.fastandmudry
2  package ui.dialogs
3
4  import ch.hevs.fastandmudry.ui.UISkin
5  import com.badlogic.gdx.scenes.scene2d.ui.{Dialog, Skin}
6
7  abstract class CustomDialog(title: String, skin: Skin) extends Dialog(title, skin)
8  {
9  }
10
11 object DialogFactory {
12     def createSettingsDialog(title: String): SettingsDialog = new
13     SettingsDialog(title, UISkin.skin)
14 }

```

9.2.94 – SettingsDialog.scala

```

1  package ch.hevs.fastandmudry
2  package ui.dialogs

```

```

3  import ch.hevs.fastandmudry.ui.components.{ButtonFactory, LabelFactory,
4  SliderFactory}
5  import ch.hevs.fastandmudry.utils.Constant.UI
6  import com.badlogic.gdx.Gdx
7  import com.badlogic.gdx.scenes.scene2d.ui.Skin
8
9  class SettingsDialog(title: String, skin: Skin) extends CustomDialog(title, skin) {
10     private val lblVolumeText = LabelFactory.create("Volume")
11     private val lblVolumeValue = LabelFactory.create("50")
12     private val sldVolume = SliderFactory.create(0f, 100f, 1f, false)
13     sldVolume.setValue(50f)
14     sldVolume.onChange(volume => lblVolumeValue.setText(volume.toInt.toString))
15
16     pad(20)
17
18     private val content = getContentTable
19     content.clear()
20     content.center()
21     content.add(lblVolumeText).padRight(15)
22     content.add(sldVolume).expandX().fillX().padRight(15)
23     content.add(lblVolumeValue).width(40)
24
25     private val btnSave = ButtonFactory.primary("Save")
26     private val btnCancel = ButtonFactory.primary("Cancel")
27     btnSave.onClick(() => hide())
28     btnCancel.onClick(() => hide())
29     button(btnCancel)
30     button(btnSave)
31
32     override def getPrefWidth: Float = Gdx.graphics.getWidth *
33     UI.Dialog.SettingsDialog.SIZE_PERCENTAGE
34     override def getPrefHeight: Float = Gdx.graphics.getHeight *
35     UI.Dialog.SettingsDialog.SIZE_PERCENTAGE
36 }

```

9.2.95 – CarHUD.scala

```

1  package ch.hevs.fastandmudry
2  package ui.hud
3
4
5  import ch.hevs.fastandmudry.core.ecs.systems.Car
6  import ch.hevs.fastandmudry.core.world.World
7  import ch.hevs.fastandmudry.utils.Constant.GAME.CAR.FACTOR
8  import ch.hevs.fastandmudry.utils.Constant.RENDERING
9  import ch.hevs.gdx2d.components.bitmaps.BitmapImage
10 import ch.hevs.gdx2d.lib.GdxGraphics
11
12 object CarHUD {
13     private val skin = World.INSTANCE.selectedSkin
14     private val INTERIOR_IMAGE = new BitmapImage(skin.interiorImagePath)
15     private val STEERING_WHEEL_IMAGE = new BitmapImage(skin.steeringWheelImagePath)
16
17     def draw(g: GdxGraphics, car: Car): Unit = {
18         drawInterior(g)
19         drawSteeringWheel(g, car)
20     }
21
22     def drawInterior(g: GdxGraphics): Unit = {
23         val screenW = g.getScreenWidth.toFloat
24         val screenH = g.getScreenHeight.toFloat
25         val imgW = INTERIOR_IMAGE.getImage.getWidth.toFloat
26         val imgH = INTERIOR_IMAGE.getImage.getHeight.toFloat
27     }

```

```

28     val scale = math.max(screenW / imgW, screenH / imgH) *
RENDERING.CAR.INTERIOR_SCALE
29     val anchorY = screenH * RENDERING.CAR.INTERIOR_VERTICAL_ANCHOR
    g.drawTransformedPicture(screenW / 2f, screenH / 2f - anchorY, 0f, scale,
30     INTERIOR_IMAGE)
31 }
32
33 def drawSteeringWheel(g: GdxGraphics, car: Car): Unit = {
34     val wheelRatio = car.WheelAngle / FACTOR.WHEEL_MAX_ANGLE
35     val zoom = RENDERING.CAR.INTERIOR_SCALE
36     val cx = g.getScreenWidth / 2f
37     val cy = g.getScreenHeight / 2f
38     val anchorY = g.getScreenHeight * RENDERING.CAR.INTERIOR_VERTICAL_ANCHOR
39     val posX = cx + (g.getScreenWidth * skin.wheelPosition.x - cx) * zoom
40     val posY = cy + (g.getScreenHeight * skin.wheelPosition.y - cy) * zoom -
anchorY
41     g.drawAlphaPicture(posX, posY, wheelRatio * -RENDERING.CAR.WHEEL_MAX_ROTATION,
RENDERING.CAR.WHEEL_SCALE * zoom, 1f, STEERING_WHEEL_IMAGE)
42 }
43 }

```

9.2.96 – DebugHUD.scala

```

1  package ch.hevs.fastandmudry
2  package ui.hud
3
4  import ch.hevs.fastandmudry.utils.Common
5  import com.badlogic.gdx.Gdx
6  import com.badlogic.gdx.graphics.g2d.{BitmapFont, SpriteBatch}
7
8  import java.util
9
10 object DebugHUD {
11     val batch = new SpriteBatch()
12     val font = new BitmapFont()
13
14     private val logs = new util.HashMap[String, Any]()
15
16     def draw(): Unit = {
17         if(!Common.Debugging.IsDebugEnabled) return
18         batch.begin()
19         var i = 30;
20         font.draw(batch, s"FPS: ${Gdx.graphics.getFramesPerSecond}", 10,
Gdx.graphics.getHeight - 10)
21         logs.forEach((k, v) => {
22             font.draw(batch, s"$k: ${v.toString}", 10, Gdx.graphics.getHeight - i)
23             i += 20
24         })
25         batch.end()
26     }
27
28     def dispose(): Unit = {
29         batch.dispose()
30         font.dispose()
31     }
32
33     def setLogVar(name: String, value: Any): Unit = {
34         logs.put(name, value)
35     }
36 }

```

9.2.97 – SpeedometerHUD.scala

```

1  package ch.hevs.fastandmudry

```

```

2 package ui.hud
3
4 import ch.hevs.fastandmudry.core.ecs.systems.Car
5 import ch.hevs.fastandmudry.core.world.World
6 import ch.hevs.fastandmudry.utils.Constant.GAME.CAR.FACTOR
7 import ch.hevs.fastandmudry.utils.Constant.{Hud, RENDERING}
8 import ch.hevs.gdx2d.lib.GdxGraphics
9 import com.badlogic.gdx.graphics.Color
10
11 object SpeedometerHUD {
12     def draw(g: GdxGraphics, car: Car): Unit = {
13         val config = World.INSTANCE.selectedSkin.speedometer
14         val zoom = RENDERING.CAR.INTERIOR_SCALE
15         val radius = config.radius * zoom
16         val ticksLength = Hud.Speedometer.TICKS_LENGTH * zoom
17         val needleWidth = Hud.Speedometer.NEEDLE_WIDTH * zoom
18         val labelInset = 15f * zoom
19         val angleEvery = (math.abs(Hud.Speedometer.SPEEDOMETER_START_ANGLE) +
20 math.abs(Hud.Speedometer.SPEEDOMETER_END_ANGLE)) / Hud.Speedometer.TICKS_EVERY
21
22         val screenCx = g.getScreenWidth / 2f
23         val screenCy = g.getScreenHeight / 2f
24         val anchorY = g.getScreenHeight * RENDERING.CAR.INTERIOR_VERTICAL_ANCHOR
25         val cx = screenCx + (g.getScreenWidth * config.x - screenCx) * zoom
26         val cy = screenCy + (g.getScreenHeight * config.y - screenCy) * zoom - anchorY
27
28         // Speedometer background
29         g.drawFilledCircle(cx, cy, radius, config.backgroundColor)
30
31         // Speedometer ticks
32         var speedLabel = Hud.Speedometer.MAX_SPEED
33         for(i ← Hud.Speedometer.SPEEDOMETER_END_ANGLE to
34 Hud.Speedometer.SPEEDOMETER_START_ANGLE by angleEvery) {
35             val angle = Math.toRadians(i)
36             val mx = math.cos(angle) * (radius - ticksLength/2) // drawFilledRectangle
37             require the center x,y
38             val my = math.sin(angle) * (radius - ticksLength/2)
39
40             val xLabel = math.cos(angle)*(radius-ticksLength-labelInset)
41             val yLabel = math.sin(angle)*(radius-ticksLength-labelInset)
42
43             g.drawFilledRectangle(cx + mx.toFloat, cy + my.toFloat, ticksLength,
44 ticksLength/2, i.toFloat, config.ticksColor)
45
46             g.drawString(cx + xLabel.toFloat, cy + 5 + yLabel.toFloat,
47 speedLabel.toString, 1) // "1" value is to align it correctly
48             speedLabel -= Hud.Speedometer.MAX_SPEED / Hud.Speedometer.TICKS_EVERY
49         }
50
51         // Speedometer needle
52         val carSpeedInKmH = (Hud.Speedometer.MAX_SPEED * car.Speed) / FACTOR.MAX_SPEED
53         val angleDeg = Hud.Speedometer.SPEEDOMETER_START_ANGLE + (carSpeedInKmH *
54 (Hud.Speedometer.SPEEDOMETER_END_ANGLE -
55 Hud.Speedometer.SPEEDOMETER_START_ANGLE)) / Hud.Speedometer.MAX_SPEED
56         val angleRad = Math.toRadians(angleDeg)
57
58         val xStart = math.cos(angleRad)*radius
59         val yStart = math.sin(angleRad)*radius
60
61         val mx = cx + xStart / 2
62         val my = cy + yStart / 2
63
64         val needleLength = radius - ticksLength
65         g.drawFilledRectangle(mx.toFloat, my.toFloat, needleLength, needleWidth,
66 angleDeg, config.needleColor)
67         g.drawFilledCircle(cx, cy, 20 * zoom, Color.GRAY)
68     }
69 }

```

61 }

9.2.98 – TemperatureHUD.scala

```

1 package ch.hevs.fastandmudry
2 package ui.hud
3
4 import ch.hevs.fastandmudry.core.ecs.systems.Car
5 import ch.hevs.fastandmudry.utils.Constant.GAME
6 import ch.hevs.gdx2d.lib.GdxGraphics
7 import com.badlogic.gdx.Gdx
8 import com.badlogic.gdx.graphics.{Color, Texture}
9
10 object TemperatureHUD {
11     val thermometerOuter = new Texture("data/images/thermometer.png")
12     val PADDING = 20f
13
14     def draw(g: GdxGraphics, car: Car): Unit = {
15         val x = Gdx.graphics.getWidth / 1.05f
16         val y = Gdx.graphics.getHeight / 20f
17
18         val imgW = thermometerOuter.getWidth.toFloat / 4f
19         val imgH = thermometerOuter.getHeight.toFloat / 4f
20
21         val max = math.abs(GAME.CAR.FACTOR.MAX_TEMPERATURE) +
22             math.abs(GAME.CAR.FACTOR.MIN_TEMPERATURE)
23         val currentTemp = car.Temperature + max/2
24
25         val tempW = imgW
26         val tempH = if(max != 0) currentTemp * imgH / max else 0
27
28         if (tempH > imgH - PADDING) tempH -= PADDING/2
29
30         val color: Color = {
31             if(currentTemp < max / 4) Color.BLUE
32             else if(currentTemp < max / 2) Color.GREEN
33             else if(currentTemp > max - max / 4) Color.RED
34             else if(currentTemp > max / 2) Color.YELLOW
35             else Color.PURPLE
36         }
37
38         g.drawFilledRectangle(x + tempW/2, y + tempH/2, imgW - PADDING, tempH, 0f,
39             color)
40         g.draw(thermometerOuter, x, y, imgW, imgH)
41     }
42 }

```

9.2.99 – WalletHUD.scala

```

1 package ch.hevs.fastandmudry.ui.hud
2
3 import ch.hevs.fastandmudry.core.state.Wallet
4 import ch.hevs.gdx2d.lib.GdxGraphics
5 import com.badlogic.gdx.graphics.Texture
6 import com.badlogic.gdx.graphics.g2d.BitmapFont
7 import com.badlogic.gdx.utils.Align
8
9 object WalletHUD {
10     val mudryCoinImg = new Texture("data/images/MudryCoin.png")
11     val font = new BitmapFont()
12     font.getData.setScale(2f)
13     val margin = 30
14     val pictureWidth = 200
15     val pictureHeight = 100
16     val between = 10

```

```

17     def draw(g: GdxGraphics): Unit = {
18         g.drawString(g.getScreenWidth - pictureWidth - margin - between,
19         g.getScreenHeight - pictureHeight/2 - margin + font.getXHeight/2 ,
20         Wallet.coins.toString, font, Align.bottomRight)
21         g.draw(mudryCoinImg, g.getScreenWidth - pictureWidth - margin,
22         g.getScreenHeight - pictureHeight - margin, pictureWidth, pictureHeight)
23     }
24 }

```

9.2.100 – UISkin.scala

```

1 package ch.hevs.fastandmudry
2 package ui
3
4 import com.badlogic.gdx.Gdx
5 import com.badlogic.gdx.scenes.scene2d.ui.Skin
6
7 object UISkin {
8     lazy val skin: Skin = new Skin(Gdx.files.internal("data/ui/components/button/
9     uiskin.json"))
10 }

```

9.2.101 – Common.scala

```

1 package ch.hevs.fastandmudry
2 package utils
3
4 import ch.hevs.fastandmudry.input.DebugInput
5
6 object Common {
7     object Debugging {
8         def IsDebugEnabled: Boolean = {
9             DebugInput.IsDebugEnabled
10         }
11     }
12 }

```

9.2.102 – Constant.scala

```

1 package ch.hevs.fastandmudry
2 package utils
3
4 import com.badlogic.gdx.Input
5
6 object Constant {
7     object Window {
8         val WIDTH: Int = 1920
9         val HEIGHT: Int = 1080
10     }
11     object MENU {
12         val BACKGROUND_IMAGE = "data/images/menu_bg.png"
13     }
14     object Hud {
15         object Speedometer {
16             val MAX_SPEED: Int = 100
17             val TICKS_EVERY: Int = 10
18             val SPEEDOMETER_START_ANGLE: Int = 210
19             val SPEEDOMETER_END_ANGLE: Int = -30
20
21             val TICKS_LENGTH: Float = 10f
22             val NEEDLE_WIDTH: Float = 3f
23         }
24     }
25 }

```

```

25 object UI {
26     object Dialog {
27         object SettingsDialog {
28             val SIZE_PERCENTAGE: Float = 0.6f
29         }
30     }
31 }
32 object MapTexture {
33     val MAP_PADDING: Int = 200
34     val HALF_ROAD_WIDTH: Float = 30f
35     val SHOULDER_WIDTH: Float = 15f
36     val HALF_SHOULDER_WIDTH: Float = HALF_ROAD_WIDTH + SHOULDER_WIDTH
37     val HALF_LINE_WIDTH: Float = 2f
38     val SHOULDER_JITTER: Float = 32f
39     val SHOULDER_JITTER_NOISE_CELL: Int = 32
40
41     object ITEMS {
42         object FOREST {
43             object SIMPLE_TREE {
44                 val RENDERING_FACTOR = 80
45                 val IMAGE_SOURCE = "data/images/item/forest/Tree"
46             }
47
48             object SIMPLE_ROCK {
49                 val RENDERING_FACTOR = 20
50                 val IMAGE_SOURCE = "data/images/item/forest/Rock"
51             }
52
53             object SIMPLE_BUSH {
54                 val RENDERING_FACTOR = 30
55                 val IMAGE_SOURCE = "data/images/item/forest/Bush"
56             }
57         }
58
59         object DESERT {
60             object CACTUS {
61                 val RENDERING_FACTOR = 50
62                 val IMAGE_SOURCE = "data/images/item/desert/Cactus"
63             }
64
65             object DESERTIC_ROCK {
66                 val RENDERING_FACTOR = 35
67                 val IMAGE_SOURCE = "data/images/item/desert/DeserticRock"
68             }
69
70             object DESERTIC_SKELETON {
71                 val RENDERING_FACTOR = 15
72                 val IMAGE_SOURCE = "data/images/item/desert/DeserticSkeleton"
73             }
74         }
75
76         object SNOW {
77             object PIN_TREE {
78                 val RENDERING_FACTOR = 50
79                 val IMAGE_SOURCE = "data/images/item/snow/PinTree"
80             }
81
82             object SNOWY_ROCK {
83                 val RENDERING_FACTOR = 35
84                 val IMAGE_SOURCE = "data/images/item/snow/SnowyRock"
85             }
86
87             object SNOW_PILE {
88                 val RENDERING_FACTOR = 15
89                 val IMAGE_SOURCE = "data/images/item/snow/SnowPile"
90             }
91         }

```



```

92     object HES {
93         val RENDERING_FACTOR = 500
94         val IMAGE_SOURCE = "data/images/item/HES.png"
95     }
96 }
97 }
98 }
99 object KEYS {
100     val ENABLING_DEBUG_KEY = Input.Keys.F12
101 }
102 object QUIZ {
103     val COINS_PER_CORRECT_ANSWER: Int = 100
104     val REVEAL_DURATION: Float = 3f
105     val BUTTON_WIDTH: Float = 300
106     val BUTTON_HEIGHT: Float = 80
107     val GRID_GAP: Float = 30
108     val BACKGROUND_IMAGE: String = "data/images/quiz/bg.png"
109 }
110 object GARAGE {
111     val LIST_WIDTH: Float = 600
112     val LIST_PADDING: Float = 40
113     val ROW_GAP: Float = 15
114     val BACKGROUND_IMAGE: String = "data/images/garage/bg.png"
115 }
116 object CAR_SELECTOR {
117     val BACKGROUND_IMAGE: String = "data/images/car_selector_bg.png"
118     val BUTTON_WIDTH: Float = 120
119     val BUTTON_HEIGHT: Float = 120
120     val SIDE_PADDING: Float = 80
121     val TOP_BOTTOM_PADDING: Float = 60
122     val SELECT_BUTTON_WIDTH: Float = 300
123     val SELECT_BUTTON_HEIGHT: Float = 80
124     val EXIT_BUTTON_WIDTH: Float = 200
125     val EXIT_BUTTON_HEIGHT: Float = 80
126     val CAR_VERTICAL_OFFSET: Float = 150
127     val FONT_SIZE: Float = 3f
128 }
129 object CINEMATIC {
130     val FRONT_FINAL_BACKGROUND_IMAGE: String = "data/images/cinematic/final/
front_bg.png"
131     val BACK_FINAL_BACKGROUND_IMAGE: String = "data/images/cinematic/final/
back_bg.png"
132     val MUDRY_BACKGROUND_IMAGE: String = "data/images/cinematic/final/mudry.png"
133 }
134 object DEATH {
135     val BACKGROUND_IMAGE: String = "data/images/death_bg.png"
136 }
137 object GAME {
138     object CAR {
139         object FACTOR {
140             val ACCELERATION = 15f
141             val DECELERATION = 7f
142             val SPEED = 5f
143             val WHEEL_ROTATION = 0.05f
144             val WHEEL_RETURN = 0.01f
145             val WHEEL_MAX_ANGLE = 0.05f
146             val MAX_SPEED = 60f
147             val MAX_TEMPERATURE = 100f
148             val MIN_TEMPERATURE = -100f
149         }
150     }
151 }
152 object AUDIO {
153     val MUSIC_VOLUME: Float = 0.2f
154     object MUSIC {
155         val MENU = "data/sounds/music/nfs.mp3"

```

```

156     val CINEMATIC = "data/sounds/music/car-arriving.mp3"
157     val QUIZ = "data/sounds/music/millionaire.mp3"
158     val GARAGE = "data/sounds/music/garage.mp3"
159     val FOREST = "data/sounds/music/tokyo-drift.mp3"
160     val DESERT = "data/sounds/music/free-bird.mp3"
161     val SNOW = "data/sounds/music/drive.mp3"
162   }
163   object SFX {
164     val ENGINE_VOLUME: Float = 0.5f
165     val ENGINE = "data/sounds/sfx/car-engine.mp3"
166     val ENGINE_LOW_PITCH: Float = 0.6f
167     val ENGINE_TOP_PITCH: Float = 2.0f
168
169     val BACKFIRE = "data/sounds/sfx/pop.mp3"
170     val BACKFIRE_VOLUME: Float = 0.6f
171     val BACKFIRE_SPEED_RATIO: Float = 0.7f
172     val BACKFIRE_MIN_POPS: Int = 4
173     val BACKFIRE_MAX_POPS: Int = 5
174     val BACKFIRE_MIN_GAP: Float = 0.1f
175     val BACKFIRE_MAX_GAP: Float = 0.35f
176     val BACKFIRE_MIN_PITCH: Float = 0.8f
177     val BACKFIRE_MAX_PITCH: Float = 1.2f
178
179     val TIRE_EXPLOSION = "data/sounds/sfx/tire-explosion.mp3"
180     val TIRE_EXPLOSION_VOLUME: Float = 0.8f
181
182     val COLLISION = "data/sounds/sfx/crash.mp3"
183     val COLLISION_VOLUME: Float = 0.4f
184   }
185 }
186 object RENDERING {
187   object CAR {
188     val WHEEL_MAX_ROTATION = 180
189     val WHEEL_SCALE = 1.75f
190     var INTERIOR_SCALE = 1.3f
191     var INTERIOR_VERTICAL_ANCHOR = 0.2f
192   }
193 }
194 }

```

9.2.103 – Generator.scala

```

1 package ch.hevs.fastandmudry
2 package utils.noise
3
4 import scala.util.Random
5
6 // The code comes from this repo: https://github.com/yatsukha/perlin-noise
7 object Generator {
8   def gradientStream(rng: Random): Stream[(Double, Double)] = {
9     val x = rng.between(-1.0, 1.0)
10    val y = rng.between(-1.0, 1.0)
11
12    val len = math.sqrt(x * x + y * y)
13
14    // avoid corner bias
15    if (len > 1.0)
16      gradientStream(rng)
17    else
18      (x / len, y / len) #:: gradientStream(rng)
19  }
20
21 // guaranteed O(1) indexed access
22 def gradientField(rng: Random, dim: (Int, Int))
23 : IndexedSeq[IndexedSeq[(Double, Double)]] = {
24

```

```

25     val stream = gradientStream(rng)
26     Vector((for (i ← 0 until dim._1) yield stream.drop(i *
27 dim._2).take(dim._2).toVector): _*)
28 }

```

9.2.104 – Noise.scala

```

1  package ch.hevs.fastandmudry
2  package utils.noise
3
4  // The code comes from this repo: https://github.com/yatsukha/perlin-noise
5  object Noise {
6    def noise(g: IndexedSeq[IndexedSeq[(Double, Double)]], p: (Double, Double)):
7    Double = {
8      // floors and ceilings of the point, aka bounds
9      val bx = (p._1.toInt, (p._1 + 1).toInt)
10     val by = (p._2.toInt, (p._2 + 1).toInt)
11
12     // distances to the bounds
13     val dx = (p._1 - bx._1, p._1 - bx._2)
14     val dy = (p._2 - by._1, p._2 - by._2)
15
16     // smooth step of the upperleft bound is used as weight
17     val w = (Util.smoothStep(dx._1), Util.smoothStep(dy._1))
18
19     // dot product for upper bounds
20     var vx0 = dot(g(bx._1)(by._1), (dx._1, dy._1))
21     var vx1 = dot(g(bx._2)(by._1), (dx._2, dy._1))
22
23     val vy0 = scale(vx0, vx1, w._1)
24
25     // dot product for lower bounds
26     vx0 = dot(g(bx._1)(by._2), (dx._1, dy._2))
27     vx1 = dot(g(bx._2)(by._2), (dx._2, dy._2))
28
29     val vy1 = scale(vx0, vx1, w._1)
30
31     scale(vy0, vy1, w._2)
32   }
33
34   private def dot(a: (Double, Double), b: (Double, Double)): Double =
35     a._1 * b._1 + a._2 * b._2
36
37   private def scale(x: Double, y: Double, w: Double): Double =
38     (1.0 - w) * x + w * y
39
40   def smooth(grid: Array[Double], rows: Int, cols: Int, x: Double, y: Double):
41   Double = {
42     val ix = x.toInt
43     val iy = y.toInt
44     val fx = x - ix
45     val fy = y - iy
46
47     val x0 = ix % rows
48     val x1 = (ix + 1) % rows
49     val y0 = iy % cols
50     val y1 = (iy + 1) % cols
51
52     val c00 = grid(x0 * cols + y0)
53     val c01 = grid(x0 * cols + y1)
54     val c10 = grid(x1 * cols + y0)
55     val c11 = grid(x1 * cols + y1)
56
57     c00 * (1 - fx) * (1 - fy) +

```

```

57     c01 * (1 - fx) * fy +
58     c10 * fx * (1 - fy) +
59     c11 * fx * fy
60 }
61
62 def turbulence(grid: Array[Double], rows: Int, cols: Int, s: Int, x: Int, y:
63 Int): Double = {
64     var counter = s
65     var value = 0.0
66     while (counter ≥ 1) {
67         value += smooth(grid, rows, cols, x.toDouble / counter, y.toDouble / counter)
68     }
69     value / (2 * s.toDouble)
70 }
71 }

```

9.2.105 – Util.scala

```

1  package ch.hevs.fastandmudry
2  package utils.noise
3
4  // The code comes from this repo: https://github.com/yatsukha/perlin-noise
5  object Util {
6      def smoothStep(x: Double): Double =
7          x * x * (3 - 2 * x)
8
9      def exaggerate(x: Double, a: Double = 1.85, limits: (Double, Double) = (-1.0,
10 1.0))
11      : Double = math.max(math.min(x * a, limits._2), limits._1)
12
13      def marble(noise: Double)
14      (xy: (Double, Double))
15      (noiseStrength: Double = 2.0, period: (Double, Double) = (5.0, 10.0)):
16  Double =
17      (1 + math.sin((noise * noiseStrength + xy._1 * period._1 + xy._2 * period._2) *
18  math.Pi)) / 2
19  }

```

9.2.106 – mode7.glsl

```

1  // in from the java side
2  uniform sampler2D backbuffer;
3  uniform vec2 resolution;
4  uniform float time;
5  uniform int enabled;
6  uniform vec3 cameraPosition;
7  uniform vec3 cameraAxis;
8  uniform float cameraAngle;
9  uniform float screenPlanDistance;
10 uniform float pitch;
11 uniform vec2 renderingFactor;
12 uniform vec2 mapOrigin;
13
14 mat3 rotationMatrix(vec3 axis, float angle)
15 {
16     axis = normalize(axis);
17     float s = sin(angle);
18     float c = cos(angle);
19     float oc = 1.0 - c;
20
21     return mat3(oc * axis.x * axis.x + c,          oc * axis.x * axis.y - axis.z *
s, oc * axis.z * axis.x + axis.y * s,

```

```

22         oc * axis.x * axis.y + axis.z * s, oc * axis.y * axis.y + c,
23         oc * axis.y * axis.z - axis.x * s,
24         oc * axis.z * axis.x - axis.y * s, oc * axis.y * axis.z + axis.x *
25         s, oc * axis.z * axis.z + c);
26     }
27     void main() {
28         if(enabled == 1){
29             //Calculate the vector coming out from the camera to the pixel of the screen in
30             the 3D space.
31             vec3 pixelRay;
32             pixelRay.xz = (gl_FragCoord.xy / resolution.xy)*vec2(1.0, -1.0) + vec2(-0.5,
33             0.5);
34             pixelRay.z += pitch;
35             pixelRay.y = screenPlanDistance;
36             pixelRay *= rotationMatrix(cameraAxis,cameraAngle);
37
38             if (pixelRay.z > 0.0) {
39                 vec2 hitPosition = cameraPosition.xy + pixelRay.xy *
40                 (cameraPosition.z / pixelRay.z) + vec2(0.5, 0);
41
42                 hitPosition = (hitPosition - mapOrigin) * renderingFactor;
43
44                 if (hitPosition.x ≥ 0.0 && hitPosition.x ≤ 1.0 &&
45                 hitPosition.y ≥ 0.0 && hitPosition.y ≤ 1.0)
46                 {
47                     gl_FragColor = texture2D(backbuffer, hitPosition);
48                 }
49             }
50             else {
51                 gl_FragColor = texture2D(backbuffer, gl_FragCoord.xy / resolution.xy);
52             }
53         }
54     }

```